



UNIVERSITÀ DEGLI STUDI DI SALERNO

Relazione Progettuale

Food Delivery Platform

Antony Storti

a.storti2@studenti.unisa.it

Matricola: 0622702353

DIEM

Paola Saggiomo

p.saggiomo1@studenti.unisa.it

Matricola: 0622702380

DIEM

10 Febbraio 2025

Abstract

Il progetto *Food Delivery Platform* è un sistema distribuito che ottimizza il processo di ordinazione e consegna di cibo, collegando clienti, ristoranti e rider. I clienti possono esplorare i menu, effettuare ordini personalizzati, tracciare le consegne e condividere recensioni. I ristoranti gestiscono i loro menu, ricevono ordini in tempo reale e monitorano le performance. I rider ricevono ordini ottimizzati e gestiscono più consegne simultaneamente. Il sistema amministrativo supporta la gestione degli account e la generazione di report. La piattaforma utilizza **API RESTful** per la comunicazione tra i componenti, **RabbitMQ** per la gestione degli ordini e **Apache Kafka** per sincronizzare le richieste di consegna tra ristoranti e rider, assicurando scalabilità, alta disponibilità e performance ottimali. Il sistema è progettato per gestire picchi di carico e garantire la sicurezza dei dati e una reportistica accurata. Inoltre, grazie alla sua architettura distribuita, la piattaforma è in grado di offrire un'esperienza utente fluida e reattiva, anche durante le ore di punta. La progettazione modulare e l'uso di tecnologie moderne consentono di adattare facilmente il sistema a nuove funzionalità e scalare orizzontalmente man mano che la base di utenti cresce. Infine, l'approccio orientato agli eventi e l'uso di sistemi di messaggistica avanzati garantiscono una gestione efficiente e in tempo reale di tutti i flussi di lavoro, migliorando l'affidabilità e la resilienza del sistema.

Indice

1	Introduzione	1
2	Specifica dei Requisiti del Sistema (SRS)	4
2.1	Scopo	4
2.1.1	Visione generale del sistema	4
2.1.2	Per i clienti	5
2.1.3	Per i ristoranti	6
2.1.4	Per i rider	7
2.1.5	Per gli amministratori	9
2.1.6	Benefici complessivi del sistema	10
2.1.7	Impatti strategici	12
2.2	Descrizione Generale del Sistema	15
2.2.1	Funzionalità per i clienti	15
2.2.2	Funzionalità per i ristoranti	16
2.2.3	Funzionalità per i rider	18
2.2.4	Funzionalità per gli amministratori	19
2.2.5	Gestione degli ordini	21
2.2.6	Gestione delle consegne	23
2.2.7	Scalabilità e prestazioni	24
2.2.8	Sicurezza e privacy	26
2.3	Requisiti Specifici	29
2.3.1	Requisiti Funzionali	29
2.3.2	Requisiti non Funzionali	38
3	Architettura del Sistema	41
3.1	Architettura Generale	41
3.2	Componenti Principali del Sistema	43
3.2.1	Model	44
3.2.2	View	46
3.2.3	Controller	48
3.2.4	Database	50
3.3	Interazione tra i Componenti	53
3.3.1	Interazione tra Utente e View	54
3.3.2	Interazione tra View e Controller	55
3.3.3	Interazione tra Controller e Model	57
3.3.4	Interazione tra Model e Database	58
4	Framework, Toolkit e Protocolli Utilizzati	61
4.1	Flask	61
4.1.1	Caratteristiche Principali	61
4.1.2	Architettura	61
4.1.3	Vantaggi e Svantaggi	63
4.2	AJAX	65
4.2.1	Funzionamento	65
4.2.2	Formati dei Dati	67
4.2.3	Vantaggi e Svantaggi	68
4.3	SQLAlchemy	71

4.3.1	Caratteristiche Principali	71
4.3.2	Architettura	71
4.3.3	Vantaggi e Svantaggi	72
4.4	SQLite	75
4.4.1	Caratteristiche Principali	75
4.4.2	Vantaggi e Svantaggi	77
4.5	API RESTful	79
4.5.1	Caratteristiche Principali	79
4.5.2	Vantaggi e Svantaggi	81
4.6	RabbitMQ	84
4.6.1	Caratteristiche Principali	84
4.6.2	Advanced Message Queuing Protocol (AMQP)	85
4.6.3	Vantaggi e Svantaggi	86
4.7	Kafka	89
4.7.1	Architettura	89
4.7.2	Flusso di Dati	91
4.7.3	Vantaggi e Svantaggi	93
5	Descrizione del Codice	96
5.0.1	__init__.py	96
5.0.2	extensions.py	96
5.0.3	config.py	97
5.0.4	run.py	97
5.1	Templates	98
5.1.1	admin_dashboard.html	98
5.1.2	admin_manage_accounts.html	98
5.1.3	admin_monitor_performance.html	99
5.1.4	customer_browse_restaurants.html	100
5.1.5	customer_dashboard.html	101
5.1.6	customer_place_order.html	102
5.1.7	customer_search_foods.html	103
5.1.8	customer_track_orders.html	104
5.1.9	customer_view_menu.html	105
5.1.10	customer_view_order_details.html	106
5.1.11	customer_view_orders.html	107
5.1.12	customer_view_profile.html	108
5.1.13	home.html	109
5.1.14	login.html	109
5.1.15	register.html	110
5.1.16	restaurant_dashboard.html	111
5.1.17	restaurant_edit_item.html	112
5.1.18	restaurant_manage_menu.html	113
5.1.19	restaurant_order_details.html	113
5.1.20	restaurant_view_orders.html	114
5.1.21	restaurant_view_profile.html	115
5.1.22	rider_dashboard.html	116
5.1.23	rider_delivery_requests.html	116
5.1.24	rider_track_optimal_routes.html	117

5.1.25	rider_update_availability.html	117
5.1.26	rider_update_status.html	118
5.1.27	rider_view_profile.html	118
5.2	Static	120
5.2.1	admin_manage_accounts.js	120
5.2.2	admin_monitor_performance.js	120
5.2.3	customer_track_orders.js	121
5.2.4	home.js	122
5.2.5	login.js	122
5.2.6	restaurant_edit_item.js	123
5.2.7	restaurant_manage_menu.js	124
5.2.8	restaurant_view_orders.js	126
5.2.9	rider_delivery_requests.js	127
5.2.10	rider_track_optimal_routes.js	128
5.2.11	rider_update_availability.js	130
5.2.12	rider_update_status.js	131
5.3	Models	132
5.3.1	delivery.py	132
5.3.2	menu.py	133
5.3.3	order.py	133
5.3.4	restaurant.py	134
5.3.5	user.py	135
5.4	Routes	137
5.4.1	administrator.py	137
5.4.2	auth.py	138
5.4.3	customer.py	141
5.4.4	delivery.py	142
5.4.5	home.py	144
5.4.6	restaurant.py	144
5.5	Utils	146
5.5.1	consumer_kafka.py	146
5.5.2	consumer_rabbitmq.py	147
5.5.3	get_coordinates.py	148
5.5.4	haversine_distance.py	148
5.5.5	producer_kafka.py	149
5.5.6	producer_rabbitmq.py	149
5.5.7	session_handler.py	150
5.5.8	trigger.py	151

1 Introduzione

Nel contesto attuale, la crescente domanda di soluzioni rapide e comode per l'acquisto e la consegna di cibo ha trasformato radicalmente il panorama della ristorazione. Con il rapido aumento della digitalizzazione, le piattaforme di food delivery sono diventate un punto di riferimento essenziale per connettere i consumatori, i ristoranti e i rider, in un ecosistema che deve garantire un'elevata efficienza e una gestione ottimale dei flussi di dati. Le aspettative dei clienti sono aumentate notevolmente: non si accontentano più solo di ricevere il cibo a domicilio, ma desiderano una user experience impeccabile, che include la possibilità di ordinare in modo semplice e veloce, monitorare lo stato delle consegne in tempo reale, comunicare direttamente con i ristoranti e i rider, e lasciare recensioni e valutazioni per condividere la propria esperienza. Al contempo, i ristoranti devono essere in grado di gestire ordini, modificare menu, ottimizzare la preparazione dei piatti e avere visibilità sulle performance, in modo da migliorare continuamente la qualità del servizio offerto. I rider, infine, devono affrontare la sfida di gestire più consegne simultaneamente, ricevere ordini in tempo reale e ottimizzare i percorsi di consegna, il tutto con l'obiettivo di garantire che ogni cliente riceva il proprio ordine in tempo utile e nelle condizioni desiderate.

Il progetto *Food Delivery Platform* si propone di rispondere a tutte queste esigenze, creando una piattaforma digitale avanzata che ottimizza l'interazione tra i vari attori e assicura il buon funzionamento del sistema complesso e distribuito che gestisce ordini, consegne e feedback. L'obiettivo primario è quello di fornire un'infrastruttura che sia in grado di supportare operazioni quotidiane ad alta intensità di traffico, con una gestione dinamica delle risorse e dei flussi, ma che allo stesso tempo garantisca una risposta rapida, sicura e precisa a ogni singola richiesta. La piattaforma è sviluppata per rispondere alle esigenze specifiche di tre categorie principali di utenti: i clienti, i ristoranti e i rider.

I **clienti** sono l'anima della piattaforma, e la loro esperienza deve essere il più fluida e intuitiva possibile. La piattaforma consente loro di esplorare una vasta gamma di ristoranti e menu, con la possibilità di ordinare in modo completamente personalizzato, selezionando piatti, aggiungendo preferenze alimentari, e completando l'ordine in pochi passi. Il sistema offre anche la possibilità di tracciare gli ordini in tempo reale, monitorando ogni fase del processo di consegna, dalla preparazione in cucina alla consegna del cibo direttamente a casa. Inoltre, i clienti possono comunicare facilmente con i ristoranti e i rider, indicando eventuali richieste speciali e ricevendo conferme tempestive. Non da ultimo, dopo aver ricevuto il pasto, i clienti possono lasciare recensioni e valutazioni, contribuendo così a migliorare la qualità del servizio e aiutando altri utenti a fare scelte più consapevoli.

I **ristoranti**, dall'altro lato, devono essere in grado di gestire un flusso costante di ordini e operazioni quotidiane in modo efficiente. La piattaforma offre loro la possibilità di caricare e aggiornare il menu, gestire i prezzi e le offerte speciali, monitorare lo stato degli ordini in tempo reale e analizzare le performance complessive. Ogni ordine che arriva alla cucina viene immediatamente notificato, con la possibilità di aggiornare lo stato dell'ordine (ad esempio, "in preparazione", "pronto per la consegna") e comunicare con i rider per la gestione ottimale della consegna. I ristoranti possono anche visualizzare report dettagliati riguardanti le loro performance, analizzare le recensioni dei clienti, e implementare cambiamenti mirati per migliorare il servizio e incrementare le vendite.

I **rider**, infine, rappresentano il cuore pulsante della piattaforma per quanto riguarda

la consegna. Devono essere in grado di gestire in modo efficace gli ordini che ricevono, ottimizzare i percorsi di consegna, e aggiornare lo stato di ogni ordine in tempo reale. I rider sono abilitati a ricevere ordini in base alla loro posizione geografica e disponibilità, e possono gestire più consegne contemporaneamente se si trovano sulla stessa rotta, migliorando così l'efficienza complessiva del sistema. La piattaforma fornisce anche funzionalità di tracciamento in tempo reale, che permettono sia ai clienti che ai ristoranti di seguire l'andamento della consegna e ricevere notifiche automatiche in caso di eventuali ritardi o modifiche.

Alla base di questo sistema complesso e distribuito, è stata costruita un'architettura scalabile e robusta, che permette di gestire in modo efficiente l'intero processo, dalla registrazione degli utenti, alla gestione degli ordini, fino alla consegna finale. Le **API RESTful** sono utilizzate per la comunicazione tra i vari componenti del sistema, garantendo una gestione modulare e facilmente estendibile delle funzionalità. La piattaforma fa uso di **RabbitMQ**, un sistema di messaggistica che consente di gestire in modo efficiente i flussi di messaggi tra i vari sistemi (come ordini, aggiornamenti e notifiche). Questo sistema è particolarmente utile per garantire la corretta gestione degli ordini e la sincronizzazione tra ristoranti, rider e clienti, in quanto permette di elaborare e smistare i messaggi in modo asincrono e scalabile.

Inoltre, per gestire in modo ottimizzato le richieste di consegna e le informazioni in tempo reale tra i ristoranti e i rider, è stato implementato **Apache Kafka**, che permette di gestire flussi di eventi in modo altamente performante. Kafka è utilizzato per inviare eventi in tempo reale relativi agli ordini e alle consegne, consentendo una gestione rapida ed efficiente delle informazioni tra i vari attori, e riducendo al minimo il rischio di ritardi o perdite di dati. Queste tecnologie, combinate con un'architettura microservizi ben definita, consentono alla piattaforma di scalare orizzontalmente in risposta a picchi di traffico, gestendo contemporaneamente centinaia o migliaia di ordini senza compromettere le performance.

Un altro aspetto fondamentale della piattaforma è la gestione della **sicurezza**. Per proteggere le informazioni sensibili degli utenti e garantire la privacy delle transazioni, sono stati adottati i più elevati standard di sicurezza. I dati sensibili, come le informazioni di pagamento, sono criptati utilizzando algoritmi avanzati, e vengono rispettate le normative internazionali in materia di protezione dei dati, come il GDPR. Inoltre, l'autenticazione degli utenti avviene attraverso metodi sicuri, come l'autenticazione a due fattori, per ridurre al minimo il rischio di accessi non autorizzati.

La piattaforma è anche progettata per essere facilmente **monitorata** e **gestita** da parte degli amministratori. Questi possono visualizzare report dettagliati sulla performance complessiva del sistema, sui tempi di consegna, sul numero di ordini per ristorante e rider, e sugli eventuali problemi riscontrati nel processo di ordinazione e consegna. Queste informazioni vengono utilizzate per prendere decisioni strategiche e migliorare l'efficienza complessiva della piattaforma.

Infine, la piattaforma è costruita per essere **futura-proof**, ovvero in grado di evolversi facilmente in base alle esigenze del mercato. L'architettura modulare e l'utilizzo di tecnologie moderne consentono di aggiungere nuove funzionalità o modificare quelle esistenti in modo semplice, senza dover riprogettare l'intero sistema. Con l'evoluzione continua del settore del food delivery, la piattaforma è pronta per adattarsi alle nuove sfide, dalle integrazioni con nuove modalità di pagamento e tecnologie di intelligenza artificiale, fino alla gestione di nuove tipologie di ordini e richieste personalizzate.

In sintesi, il progetto *Food Delivery Platform* rappresenta una soluzione innovativa e

altamente scalabile per il settore del food delivery, che mette al centro la soddisfazione dei clienti, l'efficienza operativa dei ristoranti e la gestione ottimale delle consegne da parte dei rider. Grazie all'utilizzo delle tecnologie più avanzate, la piattaforma è in grado di rispondere alle sfide odierne e future del mercato, creando un ecosistema di interazione fluida e ottimizzata tra tutti gli attori coinvolti.

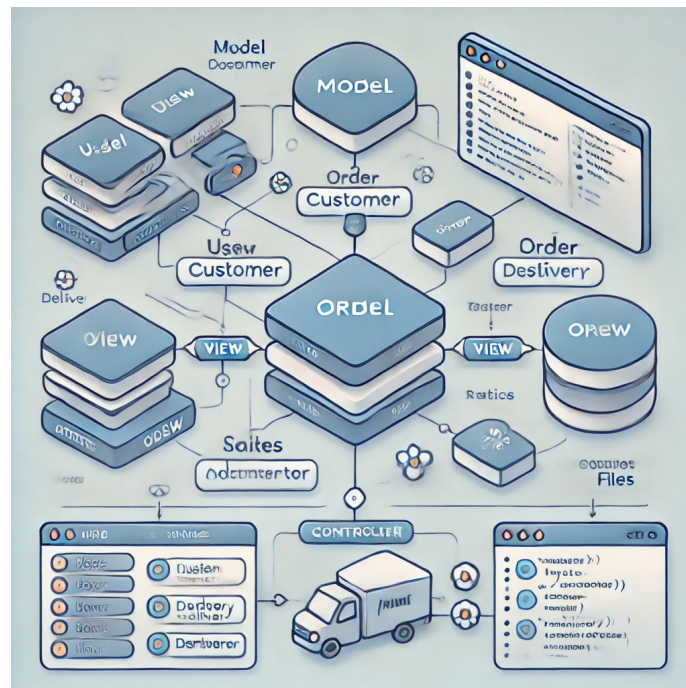


Figura 1: Food Delivery Platform

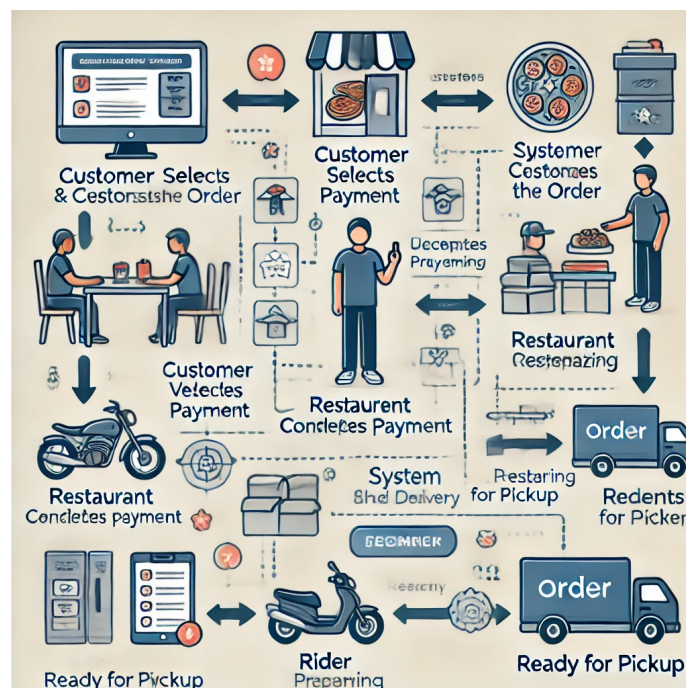


Figura 2: Processo di Ordinazione sulla Piattaforma

2 Specifica dei Requisiti del Sistema (SRS)

2.1 Scopo

La piattaforma *Food Delivery Platform* si propone come una soluzione tecnologica avanzata per rivoluzionare il settore delle consegne di cibo. Il suo scopo principale è quello di creare un ecosistema digitale che connetta in maniera fluida e ottimizzata i tre attori chiave del processo: i clienti, i ristoranti e i rider. In un contesto in cui la domanda di servizi di consegna è in costante aumento, la piattaforma mira a soddisfare le aspettative di velocità, precisione e trasparenza, superando le inefficienze dei metodi tradizionali.

Attraverso un sistema integrato e scalabile, *Food Delivery Platform* vuole garantire una gestione più efficiente degli ordini, dalla loro creazione fino alla consegna finale. Questo include il miglioramento dell'esperienza utente per i clienti, la semplificazione delle operazioni quotidiane per i ristoranti e l'ottimizzazione del lavoro per i rider. Il progetto intende non solo facilitare l'interazione tra le parti coinvolte, ma anche offrire strumenti avanzati per monitorare, analizzare e migliorare le performance complessive del sistema. La piattaforma si pone quindi come un punto di riferimento nel settore, rispondendo alle esigenze di un mercato sempre più digitale e competitivo, con l'obiettivo di garantire qualità, affidabilità e soddisfazione per tutti gli utenti coinvolti.

2.1.1 Visione generale del sistema

La *Food Delivery Platform* è progettata per essere una soluzione end-to-end che affronta in modo completo e innovativo le sfide quotidiane del settore della consegna di cibo. La sua visione è quella di creare un sistema altamente efficiente che faciliti la comunicazione e la coordinazione tra clienti, ristoranti e rider, ottimizzando ogni fase del processo di ordinazione, preparazione e consegna del cibo.

Il sistema si distingue per la sua capacità di adattarsi a diverse esigenze e contesti, permettendo a ciascun attore di avere una visibilità e un controllo completi sulle proprie operazioni. Per i **clienti**, la piattaforma offre un'interfaccia user-friendly che rende semplice la ricerca, l'ordinazione e il monitoraggio delle consegne, con un focus sulla personalizzazione dei piatti e sull'esperienza complessiva. I **ristoranti** possono utilizzare la piattaforma per gestire i loro menu, monitorare gli ordini in tempo reale e migliorare le loro performance attraverso analisi dettagliate. Infine, i **rider** beneficiano di una gestione intelligente delle consegne, ottimizzando il loro tempo e migliorando l'efficienza del servizio.

La visione del sistema non si limita alla semplice consegna di cibo, ma si estende alla creazione di un ecosistema che integra tecnologie avanzate per garantire il miglioramento continuo dei processi, un'esperienza utente eccellente e una gestione efficace e scalabile delle operazioni.

L'adozione di soluzioni come il monitoraggio in tempo reale, la gestione ottimizzata dei percorsi di consegna e la possibilità di personalizzare le opzioni per tutti gli utenti sono aspetti chiave che permettono al sistema di soddisfare in modo completo le necessità di ciascun attore. Questo approccio contribuisce non solo a migliorare la qualità del servizio, ma anche a rendere il sistema resiliente durante i periodi di picco, come le ore di punta o durante eventi speciali.

Inoltre, la piattaforma si prefigge di essere un punto di riferimento per l'innovazione nel settore delle consegne, in grado di raccogliere feedback, monitorare performance e adattarsi alle evoluzioni del mercato, rendendo ogni attore parte di un processo dinamico e in continua evoluzione.

2.1.2 Per i clienti

La *Food Delivery Platform* è stata progettata per offrire ai clienti un'esperienza di ordinazione e consegna di cibo comoda, veloce e completamente personalizzabile. L'obiettivo principale per i clienti è quello di semplificare ogni fase del processo, dalla scelta del cibo alla sua consegna, e rendere ogni interazione con la piattaforma il più semplice e piacevole possibile. La piattaforma mette a disposizione degli utenti una serie di funzionalità pensate per garantire un servizio di alta qualità e rispondere alle diverse esigenze dei consumatori.

Una delle funzionalità più importanti è la possibilità di **esplorare e scoprire ristoranti** nelle vicinanze, grazie a un sistema di ricerca avanzato che permette di filtrare i risultati per tipo di cucina, area geografica, valutazioni degli utenti e altro ancora. I clienti possono **sfogliare i menu** dei ristoranti, visualizzando informazioni dettagliate sui piatti, inclusi ingredienti, allergeni, e opzioni vegetariane, vegane o senza glutine. Ogni piatto è descritto con cura, dando al cliente la possibilità di prendere decisioni informate. Inoltre, il sistema consente di **personalizzare ogni ordine** in modo facile, con la possibilità di aggiungere o rimuovere ingredienti, scegliere preferenze di cottura e altre specifiche, rendendo l'esperienza ancora più adatta ai gusti individuali.

La piattaforma supporta anche una **facile gestione dell'ordine**, consentendo ai clienti di aggiungere più articoli al carrello, modificare l'ordine in tempo reale e visualizzare un riepilogo completo prima della conferma. Una volta completato l'ordine, i clienti possono tracciare il loro pasto in **tempo reale**, monitorando ogni fase del processo, dalla preparazione in cucina alla consegna. Questa funzione di monitoraggio migliora l'esperienza dell'utente, poiché fornisce una visione chiara dello stato dell'ordine, riducendo l'incertezza e aumentando la fiducia nel sistema.

Un altro elemento chiave per i clienti è la possibilità di **comunicare direttamente con i ristoranti e i rider** tramite la piattaforma. I clienti possono inviare istruzioni speciali, come richieste di modifiche dell'ordine o preferenze particolari, e anche comunicare direttamente con il rider per domande relative alla consegna. Questo sistema di comunicazione bidirezionale aumenta il livello di personalizzazione e permette di gestire situazioni impreviste in modo efficiente.

Per migliorare l'esperienza complessiva, la piattaforma offre anche la funzionalità di **salvare le preferenze** degli utenti, inclusi i ristoranti preferiti, i piatti più ordinati e le modalità di pagamento, facilitando così il riordino futuro. I clienti possono consultare facilmente la **storico degli ordini precedenti** e ripetere un ordine con pochi click. Inoltre, il sistema offre un sistema di **valutazioni e recensioni**, dove i clienti possono lasciare feedback sui ristoranti e sui rider. Questi feedback non solo aiutano gli altri clienti a prendere decisioni informate, ma forniscono anche preziose informazioni ai ristoranti e ai rider per migliorare il loro servizio.

Un altro punto fondamentale per il cliente è la **facilità di pagamento**. La piattaforma offre diverse modalità di pagamento sicuro, inclusi carte di credito, carte di debito, portafogli digitali e altre opzioni locali. Questo garantisce una gestione fluida e sicura delle

transazioni, con la possibilità di applicare sconti, promozioni e codici voucher, quando disponibili. La piattaforma è anche in grado di gestire **problematiche legate ai pagamenti**, come rimborsi o errori nelle transazioni, in modo rapido ed efficiente, riducendo al minimo le preoccupazioni per gli utenti.

Infine, la piattaforma si distingue per la **sicurezza e privacy dei dati**. Ogni informazione relativa agli ordini, ai pagamenti e ai dati personali viene trattata con la massima attenzione, adottando misure avanzate di protezione dei dati e garantendo che tutte le operazioni vengano svolte in modo sicuro. La piattaforma è conforme alle normative vigenti sulla protezione dei dati personali, assicurando che i clienti possano utilizzare il sistema in totale fiducia.

In sintesi, per i clienti, la *Food Delivery Platform* rappresenta una soluzione che va oltre la semplice ordinazione di cibo. È una piattaforma che facilita la scoperta di nuovi ristoranti, rende l'ordinazione e la personalizzazione dei piatti rapide e semplici, e offre un monitoraggio in tempo reale della consegna. Grazie a una gestione intuitiva dell'ordine, alla possibilità di comunicare con ristoranti e rider, e a opzioni di pagamento sicure, il sistema punta a garantire un'esperienza utente soddisfacente e senza intoppi. La piattaforma si impegna a rispondere in modo flessibile alle esigenze dei clienti, assicurando che ogni ordine venga gestito in modo tempestivo e preciso.

2.1.3 Per i ristoranti

La *Food Delivery Platform* offre ai ristoranti una serie di funzionalità pensate per ottimizzare la gestione degli ordini, migliorare l'efficienza operativa e ampliare la propria base di clienti. Il sistema è progettato per semplificare le attività quotidiane dei ristoratori, rendendo più facile la gestione del menu, la ricezione degli ordini e la comunicazione con i clienti e i rider, in modo da garantire un'esperienza fluida e senza intoppi.

Una delle funzionalità principali per i ristoranti è la possibilità di **gestire facilmente il menu**. I ristoratori possono aggiungere, modificare o rimuovere i piatti in modo rapido e intuitivo attraverso un'interfaccia user-friendly. Ogni piatto può essere corredato di dettagli come descrizione, ingredienti, allergeni, e informazioni relative alla preparazione, permettendo ai clienti di avere tutte le informazioni necessarie per fare scelte informate. Inoltre, i ristoranti possono aggiornare i prezzi, includere opzioni per piatti del giorno o menu stagionali, e offrire personalizzazioni come porzioni extra o modifiche ai piatti per soddisfare le preferenze dietetiche dei clienti. Questo consente ai ristoranti di adattarsi rapidamente alle richieste di mercato, alle tendenze gastronomiche e alle esigenze specifiche dei clienti.

La piattaforma permette ai ristoranti di **ricevere e gestire gli ordini in tempo reale**. Ogni ordine effettuato viene immediatamente notificato al ristorante, che può accettarlo, modificarlo o rifiutarlo in base alla disponibilità o a eventuali problemi con gli ingredienti. Una volta che l'ordine è stato accettato, il ristorante può **aggiornare lo stato dell'ordine** (ad esempio, "in preparazione", "pronto per il ritiro", "in consegna"), mantenendo così il cliente sempre aggiornato sullo stato della sua ordinazione. Questo sistema di aggiornamento in tempo reale riduce il rischio di errori e migliora la comunicazione con il cliente, aumentando la soddisfazione generale.

Un altro aspetto fondamentale per i ristoranti è la **gestione delle richieste speciali**. I clienti possono inserire istruzioni particolari per ogni ordine, come allergie o preferenze alimentari. I ristoranti possono gestire facilmente queste richieste e garantire che ogni

piatto sia preparato in modo adeguato, riducendo al minimo il rischio di errori o problemi di sicurezza alimentare. La piattaforma fornisce anche un sistema di **notifiche** che avvisa il ristorante di eventuali modifiche dell'ordine in tempo reale, così che possano reagire tempestivamente.

Per quanto riguarda la **gestione delle operazioni quotidiane**, la piattaforma offre ai ristoranti la possibilità di **monitorare le proprie performance** attraverso report dettagliati e analisi delle vendite, delle preferenze dei clienti e dei tempi di preparazione. Questi dati sono fondamentali per ottimizzare l'efficienza e migliorare i servizi offerti. Inoltre, i ristoranti possono visualizzare e rispondere alle **recensioni dei clienti**, utilizzando i feedback per apportare miglioramenti alla qualità del servizio e dell'offerta gastronomica. Le recensioni aiutano i ristoratori a comprendere meglio le esigenze dei loro clienti e a perfezionare l'esperienza complessiva, mantenendo alti standard di qualità.

Per facilitare la gestione delle operazioni quotidiane, i ristoranti possono **gestire la loro disponibilità e orari di apertura**. La piattaforma consente ai ristoratori di impostare orari fissi o modificabili per ogni giorno, gestendo al meglio la disponibilità del personale e riducendo al minimo il rischio di sovraccarichi durante le ore di punta. Inoltre, i ristoranti possono aggiornare facilmente il proprio profilo e le proprie informazioni, come indirizzo, numero di telefono, e politiche di servizio, garantendo che i clienti abbiano sempre informazioni accurate.

Un altro elemento cruciale è la gestione delle **consegne**. I ristoranti possono monitorare lo stato di ogni ordine, aggiornare lo stato di preparazione e comunicare direttamente con i rider per coordinare il ritiro e la consegna degli ordini. La piattaforma integra anche un sistema che permette ai ristoranti di ottimizzare il flusso di lavoro in cucina, assegnando priorità agli ordini in base al tempo di preparazione e alle necessità specifiche del cliente, come piatti speciali o urgenti.

Infine, la piattaforma fornisce ai ristoranti la possibilità di **gestire i pagamenti** in modo sicuro e trasparente, ricevendo i pagamenti direttamente tramite la piattaforma, riducendo i rischi associati alla gestione del denaro contante. I ristoranti possono visualizzare e gestire le transazioni, controllare gli incassi giornalieri e avere accesso a report finanziari dettagliati.

In sintesi, per i ristoranti, la *Food Delivery Platform* è uno strumento completo che semplifica la gestione quotidiana, migliorando l'efficienza operativa, la comunicazione con i clienti e la qualità del servizio. Grazie a funzionalità avanzate per la gestione del menu, degli ordini, delle richieste speciali, delle recensioni e delle consegne, i ristoranti sono in grado di offrire un'esperienza migliore ai clienti e di ottimizzare le loro operazioni interne.

2.1.4 Per i rider

La *Food Delivery Platform* è pensata per supportare i rider, i protagonisti mobili del sistema di consegna, offrendo una serie di strumenti pensati per ottimizzare la gestione delle consegne e migliorare l'efficienza del loro lavoro quotidiano. La piattaforma garantisce che i rider possano operare in modo semplice, sicuro ed efficiente, riducendo al minimo il rischio di errori e migliorando l'esperienza del cliente. Il sistema consente ai rider di ricevere e gestire le richieste di consegna in tempo reale, monitorare le proprie performance, comunicare facilmente con i clienti e i ristoranti, e mantenere un controllo completo sulla loro disponibilità e guadagni.

Una delle funzionalità chiave per i rider è la **ricezione delle richieste di consegna ottimizzate**. La piattaforma assegna automaticamente gli ordini ai rider in base alla loro posizione geografica e alla loro disponibilità, ottimizzando il percorso di consegna per ridurre i tempi di attesa e migliorare l'efficienza. I rider possono visualizzare dettagli come l'indirizzo del cliente, l'orario stimato di consegna e le istruzioni speciali, se presenti, direttamente sul loro dispositivo mobile, facilitando così l'esecuzione dell'ordine. Questo approccio ottimizzato consente ai rider di gestire più consegne simultaneamente, riducendo il tempo di viaggio e aumentando il numero di consegne effettuate.

La piattaforma consente anche ai rider di **aggiornare lo stato delle consegne** in tempo reale, permettendo loro di segnare l'ordine come "in transito", "consegnato" o "non consegnato" in base all'andamento della consegna. Questi aggiornamenti vengono immediatamente comunicati al cliente e al ristorante, consentendo loro di essere sempre informati sull'avanzamento della consegna. In caso di problemi o imprevisti durante il percorso, i rider hanno la possibilità di **comunicare direttamente con il cliente e il ristorante**, garantendo una comunicazione fluida e tempestiva che aiuta a risolvere qualsiasi problema in modo rapido e senza intoppi.

Un altro aspetto cruciale per i rider è la **gestione della loro disponibilità**. La piattaforma consente ai rider di impostare la propria disponibilità, indicando quando sono pronti per ricevere nuove consegne. I rider possono aggiornare lo stato di disponibilità in qualsiasi momento, ad esempio segnalandosi come "disponibili", "occupati" o "non disponibili", in base alla loro situazione e alle loro preferenze. Questo sistema flessibile consente ai rider di organizzare il loro tempo e bilanciare il lavoro con le altre esigenze quotidiane.

La piattaforma fornisce anche ai rider uno **strumento di monitoraggio delle performance e dei guadagni**. I rider possono visualizzare in tempo reale il totale delle consegne effettuate, le distanze percorse e i guadagni guadagnati per ogni singola consegna. La piattaforma calcola anche i pagamenti giornalieri o settimanali, permettendo ai rider di avere un quadro completo delle loro entrate e di gestire in modo trasparente la propria attività lavorativa. Inoltre, è possibile visualizzare le statistiche sulle consegne completate, il tempo medio di consegna, la valutazione media dei clienti e altre informazioni utili per monitorare il proprio rendimento e migliorare le performance nel tempo.

Un'altra funzionalità importante è la gestione delle **consegne multiple**. La piattaforma permette ai rider di accettare più ordini contemporaneamente, se questi sono nella stessa area geografica o in una direzione simile. In questo modo, i rider possono ottimizzare i loro percorsi, riducendo i tempi di consegna e aumentando l'efficienza. La piattaforma fornisce anche un sistema di **navigazione integrato**, che aiuta i rider a scegliere il percorso più veloce per ogni consegna, tenendo conto del traffico, delle strade chiuse e di altri fattori in tempo reale.

Inoltre, i rider possono **gestire il proprio profilo e le proprie impostazioni** sulla piattaforma. Possono aggiornare informazioni personali, come i dati di contatto, il metodo di pagamento preferito, e le impostazioni di comunicazione. In caso di problemi, i rider possono anche richiedere supporto tramite il sistema di assistenza integrato, che fornisce risposte a domande frequenti o consente di inviare richieste specifiche.

Infine, la piattaforma si preoccupa della **sicurezza dei rider**, offrendo strumenti che consentono loro di seguire le linee guida per una consegna sicura, come raccomandazioni sulle attrezzature da utilizzare (caschi, giubbotti riflettenti, ecc.) e opzioni per segnalare incidenti o problemi durante il percorso. La sicurezza dei rider è una priorità, e il sistema

è progettato per ridurre al minimo i rischi durante la fase di consegna.

In sintesi, per i rider, la *Food Delivery Platform* offre un insieme di strumenti e funzionalità che ottimizzano la gestione delle consegne, migliorano la comunicazione con i clienti e i ristoranti, e consentono una gestione flessibile e sicura della propria attività. Grazie a una gestione efficiente delle richieste, del percorso, dei guadagni e della disponibilità, i rider possono concentrarsi sulla parte operativa del loro lavoro, con un maggiore controllo e una migliore qualità del servizio.

2.1.5 Per gli amministratori

La *Food Delivery Platform* fornisce agli **amministratori** una serie di strumenti potenti e funzionalità per gestire l'intero ecosistema della piattaforma, monitorare le performance, e garantire il corretto funzionamento del sistema. Gli amministratori hanno il compito di supervisionare le operazioni, prendere decisioni strategiche basate sui dati e garantire che tutte le parti coinvolte (clienti, ristoranti, rider) interagiscano in modo efficiente. La piattaforma è progettata per supportare gli amministratori in queste attività, assicurando che possano monitorare, analizzare e intervenire in tempo reale, quando necessario.

Una delle principali responsabilità degli amministratori è la **gestione degli account**. Gli amministratori possono creare, aggiornare e rimuovere account per i clienti, i ristoranti e i rider, garantendo che i dati siano correttamente archiviati e che gli utenti abbiano accesso alle funzionalità appropriate in base al loro ruolo. Questo sistema centralizzato permette una gestione facile e sicura degli accessi, assicurando che solo gli utenti autorizzati possano modificare le informazioni sensibili o compiere azioni critiche. Ad esempio, l'amministratore può verificare che i ristoranti abbiano fornito informazioni accurate e che i rider siano correttamente registrati per effettuare le consegne.

Inoltre, la piattaforma permette agli amministratori di **monitorare le performance della flotta di rider** e delle operazioni dei ristoranti. Attraverso un dashboard centralizzato, gli amministratori possono visualizzare una panoramica dettagliata delle attività quotidiane: ordini in tempo reale, stato delle consegne, performance dei rider (ad esempio, il numero di consegne completate, il tempo medio di consegna, la valutazione media da parte dei clienti) e altre metriche operative. Questo monitoraggio continuo consente agli amministratori di identificare rapidamente eventuali inefficienze o problematiche, come rallentamenti nelle consegne, ritardi o ristoranti che non rispettano gli orari di preparazione.

La **generazione di report e analisi avanzate** è un'altra funzione fondamentale per gli amministratori. Utilizzando i dati raccolti dalla piattaforma, gli amministratori possono creare report personalizzati per analizzare vari aspetti delle operazioni, come il volume degli ordini per ristorante, i guadagni totali, la performance dei rider, la soddisfazione dei clienti e altro ancora. Questi report possono essere utilizzati per prendere decisioni strategiche, come l'ottimizzazione delle aree di consegna, la pianificazione di promozioni per i ristoranti più popolari, o il miglioramento dell'efficienza della flotta di rider. Le analisi possono anche includere previsioni sulle tendenze di mercato, aiutando gli amministratori a pianificare meglio le risorse e a rispondere proattivamente alle fluttuazioni della domanda.

Gli amministratori hanno anche il compito di **gestire le problematiche e i conflitti** tra i diversi attori della piattaforma. In caso di reclami da parte dei clienti, disguidi con i rider o problemi legati agli ordini, l'amministratore può intervenire direttamente per

risolvere le situazioni. Può essere necessario emettere rimborsi, prendere provvedimenti disciplinari nei confronti di un ristorante o di un rider, o risolvere problematiche legate alla qualità del servizio. La piattaforma offre strumenti che consentono di tracciare ogni interazione, garantendo che ogni problema venga risolto in modo tempestivo e trasparente. Un altro aspetto importante è la **gestione della sicurezza e della conformità**. Gli amministratori sono responsabili del rispetto delle normative legali e delle politiche aziendali, come la protezione dei dati sensibili degli utenti e il rispetto delle normative sul lavoro per i rider. La piattaforma è progettata per garantire che tutte le transazioni siano sicure, che le informazioni siano trattate in modo conforme alle leggi sulla privacy, e che tutti gli attori rispettino gli standard richiesti. Gli amministratori possono anche monitorare e gestire i permessi di accesso per garantire che solo gli utenti autorizzati possano visualizzare o modificare informazioni sensibili.

Gli **strumenti di comunicazione integrati** offrono agli amministratori un mezzo per rimanere in contatto con tutti gli attori della piattaforma. Possono inviare messaggi a singoli ristoranti, rider o clienti, oppure trasmettere avvisi e notifiche generali per informare gli utenti di modifiche, aggiornamenti o eventi speciali. Questo sistema aiuta a mantenere una comunicazione fluida e diretta, riducendo i rischi di fraintendimenti e migliorando la coordinazione tra le diverse parti coinvolte.

Infine, gli amministratori hanno accesso a un sistema di **gestione delle emergenze** che consente loro di affrontare rapidamente eventi imprevisti, come la chiusura temporanea di un ristorante, problemi con la flotta di rider, o interruzioni del servizio. In queste situazioni, gli amministratori possono prendere decisioni rapide per mantenere il servizio operativo, ad esempio riassegnando ordini ai rider disponibili, avvisando i clienti dei cambiamenti o sospendendo temporaneamente la disponibilità di un ristorante.

In sintesi, per gli amministratori, la *Food Delivery Platform* offre un ampio set di funzionalità per garantire la gestione efficace e sicura del sistema, mantenendo il controllo su tutte le operazioni, migliorando la qualità del servizio e ottimizzando le performance. Grazie a una dashboard intuitiva, strumenti di monitoraggio avanzati e la possibilità di gestire situazioni in tempo reale, gli amministratori possono assicurare che la piattaforma funzioni senza intoppi, offrendo un'esperienza di alta qualità sia per i clienti, i ristoranti e i rider.

2.1.6 Benefici complessivi del sistema

La *Food Delivery Platform* offre una serie di benefici che si riflettono positivamente sia sugli utenti finali che sull'efficienza operativa della piattaforma stessa. Questi vantaggi sono il risultato dell'adozione di un sistema distribuito, scalabile e ottimizzato per supportare tutti i partecipanti, dai clienti ai ristoranti, fino ai rider e agli amministratori. I benefici complessivi del sistema sono evidenti sotto diversi aspetti: miglioramento dell'esperienza utente, ottimizzazione delle operazioni, efficienza nelle consegne e potenziamento della gestione.

2.1.6.1 Esperienza utente migliorata

Uno dei principali benefici per i **clienti** è l'esperienza di ordinazione semplificata e personalizzata. La piattaforma consente agli utenti di esplorare facilmente un ampio assortimento di ristoranti e piatti, risparmiando tempo e fatica nella ricerca del cibo desiderato. Le funzionalità di personalizzazione degli ordini, come la possibilità di aggiungere note

speciali o modificare gli ingredienti, offrono una maggiore flessibilità. Inoltre, la funzione di tracciamento in tempo reale degli ordini permette ai clienti di monitorare l'intero processo, dal momento in cui l'ordine viene confermato dal ristorante fino alla consegna finale, migliorando la trasparenza e la sicurezza.

I **ristoranti** beneficiano di un sistema che permette una gestione efficiente delle loro operazioni quotidiane. Possono ricevere ordini in tempo reale, gestire il flusso di lavoro in cucina, e monitorare le performance in modo continuo. Questo non solo aiuta a ridurre gli errori nei processi di ordinazione, ma consente anche di ottimizzare la produzione, evitando inutili sprechi di risorse. La visibilità in tempo reale sugli ordini e le richieste speciali dei clienti permette una gestione più mirata e reattiva, aumentando la soddisfazione dei clienti e la fidelizzazione.

Per i **rider**, il sistema migliora la gestione delle consegne, grazie a ottimizzazioni in tempo reale per la pianificazione dei percorsi e l'assegnazione degli ordini. I rider possono ricevere informazioni dettagliate sugli orari di consegna e i percorsi più efficienti, il che contribuisce a ridurre i tempi di attesa e migliorare l'efficienza operativa. La possibilità di monitorare in tempo reale le loro performance e guadagni rende il lavoro più trasparente, motivante e ben remunerato, contribuendo alla loro soddisfazione.

2.1.6.2 Ottimizzazione delle operazioni

Uno dei principali vantaggi per l'**amministratore** della piattaforma è la possibilità di monitorare tutte le operazioni in tempo reale attraverso una dashboard centralizzata. Ciò consente di prendere decisioni informate e tempestive, affrontando eventuali problemi operativi come ritardi nelle consegne o errori negli ordini. La possibilità di generare report analitici offre inoltre dati cruciali per migliorare le performance e ottimizzare le risorse. Gli amministratori possono analizzare i trend delle vendite, il comportamento dei clienti, e l'efficienza delle consegne, utilizzando questi dati per introdurre miglioramenti continui al servizio. Il sistema distribuito consente inoltre una **gestione scalabile e resiliente**. In caso di picchi di domanda, ad esempio durante i periodi di alta affluenza come i weekend o le festività, la piattaforma è progettata per gestire un grande volume di ordini simultanei senza compromettere la qualità del servizio. L'uso di tecnologie come RabbitMQ e Kafka assicura che le comunicazioni tra i vari componenti del sistema avvengano in modo fluido e tempestivo, senza rallentamenti. Questo aumenta l'affidabilità della piattaforma e riduce i tempi di attesa per i clienti.

2.1.6.3 Efficienza nella gestione delle consegne

Un altro beneficio importante riguarda l'**efficienza delle consegne**. Grazie all'uso di algoritmi avanzati per l'ottimizzazione dei percorsi, i rider possono completare le consegne più velocemente, riducendo il tempo complessivo di ogni transazione e aumentando il numero di ordini che possono gestire in una giornata. L'uso di tecnologie come il tracciamento in tempo reale e l'ottimizzazione delle rotte consente di ridurre i costi operativi per la piattaforma e migliorare l'efficienza dei rider, con un impatto positivo sulla loro produttività e soddisfazione. L'efficienza delle consegne è anche migliorata dalla possibilità di gestire **consegne multiple simultanee**. Quando i rider hanno più ordini che vanno nella stessa direzione, il sistema permette di ottimizzare il percorso in modo che possano completare più consegne contemporaneamente. Questo non solo riduce i costi operativi, ma contribuisce anche a ridurre l'impatto ambientale, riducendo il numero di veicoli necessari per gestire una certa quantità di ordini.

2.1.6.4 Miglioramento della gestione dei dati e della sicurezza

Un altro vantaggio significativo del sistema è la gestione avanzata dei **dati sensibili**. La piattaforma è progettata per garantire la sicurezza delle informazioni dei clienti, dei ristoranti e dei rider, proteggendo dati personali, transazioni finanziarie e dettagli sensibili degli utenti. Le informazioni vengono archiviate in modo sicuro, con accessi protetti da crittografia, e il sistema rispetta le normative sulla privacy e sulla protezione dei dati. Gli amministratori hanno strumenti per monitorare eventuali anomalie e rispondere prontamente a minacce alla sicurezza.

2.1.6.5 Adattabilità alle esigenze future

Infine, uno dei maggiori benefici della piattaforma è la sua **adattabilità e capacità di evolversi**. La struttura modulare del sistema permette l'integrazione di nuove funzionalità o l'adeguamento alle tendenze di mercato in continua evoluzione. Ciò significa che la piattaforma può essere facilmente aggiornata per aggiungere nuove funzionalità, come il supporto per diversi metodi di pagamento, l'integrazione con nuovi sistemi di localizzazione o l'introduzione di funzionalità di intelligenza artificiale per ottimizzare ulteriormente i processi. Questo rende la piattaforma altamente scalabile e pronta ad affrontare le sfide future. In sintesi, la *Food Delivery Platform* offre numerosi benefici che spaziano dall'esperienza utente migliorata, all'efficienza operativa, alla sicurezza dei dati, e alla gestione ottimizzata delle risorse. Questi vantaggi si riflettono in un servizio più fluido, rapido e sicuro per tutti gli attori coinvolti, dalla ristorazione alla consegna, fino alla gestione amministrativa.

2.1.7 Impatti strategici

L'introduzione della *Food Delivery Platform* ha un impatto significativo su diversi livelli strategici, sia a livello operativo che a livello competitivo. I benefici generati dal sistema non solo ottimizzano l'esperienza quotidiana di clienti, ristoranti, rider e amministratori, ma contribuiscono anche a rafforzare la posizione strategica della piattaforma nel mercato delle consegne di cibo, favorendo l'adozione e il consolidamento di una posizione di leadership. Di seguito vengono analizzati i principali impatti strategici.

2.1.7.1 Vantaggio competitivo e differenziazione nel mercato

La *Food Delivery Platform* è progettata per fornire un'esperienza utente unica, personalizzata e altamente efficiente, che consente di differenziarsi dalla concorrenza. La capacità di ottimizzare le consegne, migliorare la gestione degli ordini e garantire un servizio di alta qualità rappresenta un forte vantaggio competitivo. La piattaforma permette ai ristoranti di gestire in modo dinamico i propri menu e rispondere rapidamente alle esigenze dei clienti. Inoltre, la possibilità di ricevere feedback in tempo reale e di adattarsi in modo proattivo alle tendenze di consumo e alle richieste dei clienti rende i ristoranti più competitivi, rafforzando la loro posizione nel mercato locale. La capacità di ottimizzare la gestione delle consegne, con l'uso di tecnologie avanzate come l'ottimizzazione dei percorsi e il tracciamento in tempo reale, rende la piattaforma particolarmente vantaggiosa per i **rider**, che possono incrementare i guadagni grazie a una gestione più efficiente delle consegne e a un servizio che premia la velocità e l'affidabilità.

2.1.7.2 Espansione e scalabilità

Uno degli impatti strategici più significativi riguarda la **scalabilità del sistema**. La piattaforma è progettata per essere facilmente scalabile, il che consente di espandere rapidamente la portata del servizio su nuove aree geografiche. La modularità del sistema permette di integrare nuove funzionalità o adattarsi a normative locali in modo fluido, riducendo i costi e i tempi associati a questa espansione. Questo impatto strategico è particolarmente rilevante per le aziende che puntano a crescere in modo rapido e sostenibile in un mercato competitivo. La possibilità di aggiungere nuovi ristoranti, rider e clienti alla piattaforma senza compromettere l'efficienza operativa è una chiave per l'espansione a lungo termine.

La piattaforma può anche evolversi facilmente per supportare nuove tendenze di mercato, come la **domanda crescente per opzioni alimentari più sostenibili** o l'integrazione con nuove tecnologie come la **consegna tramite veicoli autonomi**. Tali adattamenti contribuiranno a posizionare la piattaforma come un leader del settore, non solo in termini di dimensioni, ma anche di innovazione.

2.1.7.3 Ottimizzazione delle risorse e riduzione dei costi

Un impatto strategico cruciale della piattaforma riguarda l'**ottimizzazione delle risorse**. La possibilità di monitorare in tempo reale gli ordini, le consegne e le performance, unita alla gestione efficiente dei percorsi per i rider, consente di ridurre significativamente i costi operativi, migliorando l'efficienza complessiva del sistema. La riduzione dei tempi di consegna e l'uso ottimizzato delle risorse (come il personale dei ristoranti e i rider) contribuiscono direttamente a migliorare i margini di profitto.

L'adozione di tecnologie come RabbitMQ e Kafka permette di gestire gli ordini e le richieste di consegna in modo fluido e con tempi di latenza minimi. Ciò significa che il sistema può gestire un numero crescente di ordini simultanei senza compromettere le prestazioni. Questo impatto è strategico non solo per ottimizzare i costi, ma anche per aumentare la capacità di gestione di un volume crescente di transazioni senza dover investire pesantemente in infrastrutture fisiche.

2.1.7.4 Fidelizzazione e soddisfazione del cliente

Un altro impatto strategico importante è la **fidelizzazione dei clienti**. La piattaforma crea un'esperienza utente personalizzata che favorisce la fidelizzazione attraverso funzionalità come il tracciamento in tempo reale degli ordini, la possibilità di salvare preferenze e ordini precedenti, e la gestione delle recensioni. Questi elementi aumentano il livello di soddisfazione dei clienti, riducendo il churn e migliorando il valore a lungo termine per i clienti. La raccolta e l'analisi dei feedback dei clienti consente ai ristoranti di migliorare costantemente i loro servizi, contribuendo alla loro crescita e al miglioramento dell'offerta. Inoltre, la piattaforma promuove una relazione diretta e trasparente tra i clienti e i ristoranti, aumentando la fiducia e la soddisfazione reciproca. Il sistema di recensioni e valutazioni crea un ambiente in cui le prestazioni vengono monitorate costantemente, favorendo una competizione sana e il miglioramento continuo dei servizi offerti.

2.1.7.5 Sostenibilità e impatti ambientali

La piattaforma, attraverso l'**ottimizzazione dei percorsi di consegna**, contribuisce anche a ridurre l'impatto ambientale delle operazioni di delivery. Con una gestione più efficiente dei percorsi e delle consegne multiple, il sistema riduce la distanza percorsa dai rider e, quindi, le emissioni di CO₂, contribuendo a rendere il servizio più sostenibile. Inoltre, le opzioni per la gestione delle richieste speciali, come l'adozione di confezioni ecologiche, possono essere integrate nella piattaforma, promuovendo scelte di consumo più sostenibili da parte dei clienti.

2.1.7.6 Adattamento alle normative e alle esigenze del mercato

Un altro impatto strategico è la capacità della piattaforma di **adattarsi alle normative locali** e alle richieste del mercato. La piattaforma è progettata per essere flessibile e in grado di rispondere ai cambiamenti normativi e alle esigenze specifiche dei diversi mercati in cui opera. Questo consente di mantenere la conformità alle leggi locali, come quelle relative alla protezione dei dati personali (GDPR), e di adattarsi rapidamente a nuove regolazioni, senza compromettere la continuità delle operazioni.

2.2 Descrizione Generale del Sistema

La *Food Delivery Platform* è una soluzione digitale innovativa progettata per soddisfare le crescenti esigenze del settore del food delivery. Il sistema si propone di ottimizzare l'intero processo di ordinazione e consegna di cibo, migliorando l'interazione tra i diversi attori coinvolti: clienti, ristoranti, rider e amministratori. L'architettura della piattaforma è stata sviluppata con l'obiettivo di garantire un servizio scalabile, sicuro ed efficiente, che risponda in modo dinamico a diverse condizioni di carico e a un numero crescente di utenti.

Attraverso l'integrazione di una serie di funzionalità avanzate e l'adozione di una struttura modulare, la piattaforma non solo consente la gestione degli ordini, ma ottimizza anche le operazioni di delivery, offrendo un'esperienza utente fluida e soddisfacente. L'intero ecosistema, che include strumenti per la gestione dei ristoranti, l'ottimizzazione dei percorsi per i rider e la visibilità completa per gli amministratori, è pensato per garantire che ogni parte del processo venga monitorata e migliorata in tempo reale, attraverso feedback e analisi delle performance.

Questa descrizione generale esplorerà le caratteristiche principali del sistema, evidenziando come ogni componente interagisce per garantire un flusso continuo e ottimizzato di ordini e consegne. Verranno descritti i vari moduli che compongono la piattaforma e i benefici che ogni attore può trarre dal suo utilizzo, così da comprendere appieno come la piattaforma supporta in modo efficiente ogni fase del processo, dalla selezione del ristorante fino alla consegna finale al cliente.

2.2.1 Funzionalità per i clienti

La piattaforma offre ai clienti un'ampia gamma di funzionalità che rendono l'esperienza di ordinazione semplice, rapida e altamente personalizzabile. L'obiettivo principale è quello di soddisfare ogni preferenza e necessità, migliorando l'interazione con i ristoranti e rendendo il processo di ordinazione il più fluido possibile. Le funzionalità principali sono le seguenti:

- **Esplorazione di Ristoranti e Menu:** I clienti possono navigare tra un'ampia selezione di ristoranti, visualizzare i menu dettagliati e selezionare i piatti che desiderano ordinare. È possibile sfogliare le opzioni per tipo di cucina, fascia di prezzo, valutazione e disponibilità.
- **Ricerca Avanzata:** La piattaforma consente ai clienti di cercare specifici cibi o ristoranti tramite parole chiave, filtri di ricerca avanzata o categorie predefinite, rendendo facile e veloce trovare esattamente ciò che si desidera. La funzione di ricerca offre anche suggerimenti in tempo reale per facilitare ulteriormente la selezione.
- **Personalizzazione degli Ordini:** I clienti possono personalizzare ogni ordine in base alle proprie preferenze, ad esempio scegliendo varianti, aggiungendo o rimuovendo ingredienti, e indicando eventuali richieste speciali per i ristoranti (ad esempio, piatti senza allergeni o cibi vegetariani).
- **Tracking in Tempo Reale:** Una delle caratteristiche più apprezzate della piattaforma è la possibilità per i clienti di tracciare l'intero processo di consegna in tempo reale. Dopo aver effettuato un ordine, il cliente può seguire il progresso,

dalla preparazione del piatto presso il ristorante, fino al momento della consegna, visualizzando l'andamento in una mappa interattiva.

- **Gestione delle Informazioni di Pagamento:** La piattaforma consente di memorizzare e gestire facilmente le informazioni di pagamento, con la possibilità di scegliere tra diversi metodi, come carte di credito, portafogli elettronici o pagamenti tramite app. I clienti possono completare l'acquisto in modo sicuro e veloce, senza dover reinserire i dati ogni volta.
- **Recensioni e Feedback:** Dopo aver ricevuto il proprio ordine, i clienti possono lasciare recensioni e valutazioni per il ristorante e per il rider, condividendo la loro esperienza con gli altri utenti. Questo aiuta a mantenere una community attiva e a migliorare i servizi offerti, grazie ai feedback ricevuti.
- **Storico degli Ordini:** I clienti hanno accesso a un archivio dettagliato dei loro ordini precedenti, che consente di visualizzare lo storico delle scelte fatte, ripetere ordini passati e mantenere un registro delle preferenze personali. Questo sistema semplifica anche il processo di riordinazione, offrendo una lista dei ristoranti e piatti più ordinati.
- **Gestione del Profilo e delle Impostazioni:** Ogni cliente può personalizzare il proprio profilo, modificare i propri dati personali, le preferenze di pagamento e le informazioni di contatto. È possibile anche configurare le impostazioni per ricevere notifiche personalizzate (ad esempio, notifiche per offerte speciali, nuovi piatti o sconto sui prossimi ordini).
- **Programmazione Ordini Futuri:** Per una maggiore comodità, la piattaforma consente ai clienti di programmare ordini futuri, scegliendo la data e l'orario in cui desiderano che il loro cibo venga consegnato. Questa funzionalità è utile per pianificare pasti a lungo termine, come pranzi aziendali o cene speciali.

L'obiettivo di queste funzionalità è di garantire un'esperienza utente altamente interattiva, efficiente e personalizzata, che risponda alle diverse esigenze di ogni cliente. La piattaforma è progettata per rendere l'ordinazione e la consegna del cibo un'esperienza semplice, veloce e senza frizioni, mettendo al centro la soddisfazione del cliente.

2.2.2 Funzionalità per i ristoranti

La piattaforma offre ai ristoranti una serie di funzionalità avanzate che permettono loro di gestire efficientemente il loro menu, le operazioni quotidiane e le interazioni con i clienti. Queste funzionalità sono pensate per semplificare il processo di ricezione e gestione degli ordini, ottimizzare le performance del ristorante e migliorare la visibilità online, contribuendo a una gestione più fluida e profittevole del ristorante. Le funzionalità principali sono le seguenti:

- **Gestione del Menu:** I ristoranti possono facilmente aggiungere, modificare o rimuovere piatti dal proprio menu attraverso una interfaccia intuitiva. Ogni piatto può essere corredato di dettagli come descrizione, prezzo, ingredienti e informazioni nutrizionali, dando così ai clienti tutte le informazioni necessarie per una scelta consapevole.

- **Aggiornamento dei Prezzi e delle Offerte Speciali:** La piattaforma consente di aggiornare rapidamente i prezzi e le offerte speciali in tempo reale. I ristoranti possono lanciare promozioni, sconti o piatti del giorno, comunicando direttamente con i clienti e incentivando acquisti aggiuntivi.
- **Gestione degli Ordini in Tempo Reale:** Una delle funzionalità chiave per i ristoranti è la gestione degli ordini in tempo reale. Ogni ordine effettuato dai clienti viene notificato immediatamente, permettendo al ristorante di rispondere tempestivamente e preparare i piatti secondo la richiesta. Lo stato di ciascun ordine (ad esempio, "In preparazione", "Pronto per il ritiro", ecc.) viene aggiornato automaticamente sulla piattaforma, consentendo una gestione ottimale delle tempistiche.
- **Notifiche e Comunicazione con i Clienti:** I ristoranti sono in grado di ricevere notifiche in tempo reale su nuove ordinazioni, modifiche agli ordini esistenti e richieste speciali dei clienti. È possibile comunicare direttamente con i clienti per eventuali aggiornamenti sugli ordini o per risolvere problemi in tempo reale, migliorando l'esperienza complessiva.
- **Gestione delle Richieste Speciali e Personalizzazioni:** I clienti possono personalizzare i loro ordini e il ristorante ha la possibilità di gestire queste modifiche (ad esempio, eliminare ingredienti, aggiungere extra o adattare piatti alle allergie alimentari). La piattaforma consente di tracciare e gestire queste richieste speciali, garantendo che ogni ordine rispetti le specifiche del cliente.
- **Monitoraggio delle Performance:** I ristoranti possono visualizzare in tempo reale dati e analisi dettagliate delle loro performance. Le statistiche fornite comprendono il numero di ordini giornalieri, i piatti più venduti, le recensioni dei clienti e altre metriche che aiutano a migliorare l'efficienza e la qualità del servizio offerto.
- **Gestione degli Orari di Apertura e Disponibilità:** La piattaforma permette ai ristoranti di definire e aggiornare facilmente gli orari di apertura e le disponibilità. I ristoranti possono impostare orari speciali per festività o eventi particolari, così da comunicare in modo chiaro con i clienti e evitare ordini fuori orario.
- **Gestione della Capacità della Cucina:** I ristoranti possono gestire la capacità della loro cucina, definendo un limite massimo di ordini da preparare in un dato periodo per evitare sovraccarichi. Questa funzionalità aiuta a garantire che i ristoranti possano gestire gli ordini in modo ottimale e ridurre i tempi di attesa.
- **Recensioni e Feedback dei Clienti:** I ristoranti possono visualizzare le recensioni e i feedback lasciati dai clienti, che sono cruciali per monitorare la qualità del servizio e fare miglioramenti mirati. Le recensioni forniscono anche una valutazione complessiva del ristorante, utile per attrarre nuovi clienti e migliorare la reputazione online.
- **Gestione degli Ordini di Ritiro e Consegna:** Oltre agli ordini per la consegna a domicilio, i ristoranti possono gestire ordini di ritiro, offrendo la possibilità ai clienti di ritirare il cibo direttamente al ristorante. Le modalità di pagamento e di comunicazione sono integrate per garantire che tutto avvenga in modo fluido e senza intoppi.

- **Generazione di Report e Analisi:** I ristoranti possono generare report dettagliati che forniscono informazioni su vari aspetti della gestione, come le vendite, la frequenza degli ordini, i piatti più popolari, le recensioni dei clienti e molto altro. Questi report consentono ai ristoranti di prendere decisioni basate sui dati e di ottimizzare le operazioni quotidiane.

Le funzionalità descritte sono progettate per migliorare l'efficienza operativa del ristorante e favorire una gestione ottimale degli ordini, contribuendo a creare un'esperienza soddisfacente per i clienti. Con un sistema di gestione semplice e intuitivo, i ristoranti sono in grado di migliorare la qualità del servizio, ridurre gli errori e ottimizzare le proprie operazioni quotidiane, con un impatto positivo sulle loro performance generali.

2.2.3 Funzionalità per i rider

La piattaforma offre un set completo di funzionalità progettate per supportare i rider nel gestire le consegne in modo efficiente e senza problemi. Ogni funzionalità è pensata per ottimizzare i percorsi, semplificare la comunicazione e migliorare l'esperienza lavorativa dei rider. Queste funzionalità garantiscono un equilibrio tra flessibilità operativa e massimizzazione delle opportunità di guadagno. Di seguito sono descritte le principali funzionalità dedicate ai rider:

- **Ricezione di Richieste di Consegna:** I rider ricevono notifiche in tempo reale ogni volta che c'è una nuova richiesta di consegna. Le richieste vengono assegnate sulla base della loro posizione attuale, disponibilità e capacità, garantendo una gestione efficiente delle risorse.
- **Accettazione o Rifiuto delle Consegne:** I rider hanno la possibilità di accettare o rifiutare una richiesta di consegna. Questa flessibilità consente loro di gestire il proprio carico di lavoro in base alla situazione attuale, assicurando che possano operare in modo efficiente.
- **Percorsi Ottimizzati:** La piattaforma fornisce indicazioni di percorso ottimizzate per ogni consegna. Attraverso algoritmi di calcolo avanzati, i rider ricevono istruzioni che minimizzano il tempo di percorrenza, considerando fattori come il traffico in tempo reale, la distanza e l'ordine delle consegne.
- **Gestione di Consegne Multiple:** In caso di ordini diretti verso aree vicine, i rider possono gestire consegne multiple nello stesso viaggio. La piattaforma ottimizza il percorso e fornisce istruzioni chiare per massimizzare l'efficienza del rider.
- **Aggiornamento dello Stato delle Consegne:** I rider possono aggiornare lo stato delle consegne in tempo reale, indicando quando l'ordine è stato ritirato dal ristorante, quando è in transito e quando è stato consegnato. Questo fornisce ai clienti informazioni precise sullo stato dei loro ordini.
- **Comunicazione Diretta:** La piattaforma consente ai rider di comunicare direttamente con i ristoranti e i clienti in caso di problemi o domande. Questa funzionalità riduce i tempi di risposta e migliora la qualità del servizio.

- **Gestione della Disponibilità:** I rider possono impostare i loro orari di disponibilità direttamente sulla piattaforma. Questo permette loro di scegliere quando essere operativi, favorendo un equilibrio tra lavoro e vita privata.
- **Monitoraggio dei Guadagni:** Attraverso un'interfaccia chiara e intuitiva, i rider possono monitorare i loro guadagni in tempo reale. Vengono forniti dettagli su ogni consegna effettuata, comprensivi di importo guadagnato, bonus o incentivi ricevuti.
- **Recensioni e Feedback:** I rider possono visualizzare il feedback ricevuto dai clienti, il quale rappresenta un'opportunità per migliorare il servizio. La piattaforma utilizza queste recensioni anche per calcolare la valutazione complessiva del rider, che può influire sulle sue opportunità future.
- **Supporto per Problemi o Incidenti:** La piattaforma offre strumenti per segnalare problemi o incidenti durante le consegne. I rider possono richiedere supporto immediato in caso di difficoltà, garantendo un intervento tempestivo da parte dell'amministrazione.
- **Tracciamento del Veicolo e Sicurezza:** La piattaforma consente ai rider di tracciare la propria posizione in tempo reale, sia per garantire una navigazione precisa che per motivi di sicurezza. Il tracciamento è anche utilizzato per ottimizzare l'assegnazione delle consegne.
- **Strumenti di Pianificazione:** I rider possono accedere a strumenti per pianificare i turni o visualizzare le consegne programmate, permettendo loro di gestire meglio il proprio tempo.

Le funzionalità per i rider sono progettate per offrire un ambiente di lavoro flessibile ed efficiente. Grazie a un'integrazione fluida tra tecnologia e operatività, i rider possono fornire un servizio di alta qualità, aumentando la soddisfazione dei clienti e ottimizzando il proprio rendimento.

2.2.4 Funzionalità per gli amministratori

Gli amministratori della piattaforma svolgono un ruolo cruciale nel mantenere il sistema operativo, sicuro ed efficiente. Per supportare queste responsabilità, la piattaforma fornisce una serie di strumenti avanzati progettati per facilitare la gestione degli utenti, il monitoraggio delle operazioni e l'analisi delle prestazioni. Di seguito sono descritte le principali funzionalità dedicate agli amministratori:

- **Gestione degli Account:** Gli amministratori hanno il controllo completo sugli account dei clienti, dei ristoranti e dei rider. Possono creare, modificare, sospendere o eliminare account, garantendo che solo utenti autorizzati abbiano accesso alla piattaforma.
- **Monitoraggio delle Prestazioni:** La piattaforma offre strumenti per monitorare le prestazioni complessive del sistema, inclusi ordini elaborati, tempi di consegna medi, attività dei ristoranti e dei rider, e feedback dei clienti. Questo permette agli amministratori di individuare eventuali colli di bottiglia o aree di miglioramento.

- **Gestione della Flotta di Consegna:** Gli amministratori possono monitorare l'attività dei rider in tempo reale, visualizzando la loro posizione, lo stato delle consegne e la disponibilità. Questa funzionalità consente di ottimizzare l'assegnazione degli ordini e migliorare l'efficienza operativa.
- **Generazione di Report e Analisi:** La piattaforma genera report dettagliati che aiutano gli amministratori a comprendere le tendenze del business. Questi report includono metriche chiave come il volume degli ordini, i guadagni dei ristoranti, le prestazioni dei rider e il feedback dei clienti.
- **Gestione dei Pagamenti:** Gli amministratori possono supervisionare i processi di pagamento, inclusi i trasferimenti ai ristoranti e ai rider. Hanno anche la possibilità di gestire rimborsi o risolvere controversie relative ai pagamenti.
- **Gestione delle Recensioni e dei Feedback:** Gli amministratori possono moderare le recensioni e i feedback inviati dai clienti, garantendo che siano appropriati e costruttivi. Possono anche intervenire per risolvere problematiche segnalate nei commenti.
- **Risoluzione dei Problemi:** La piattaforma include strumenti per rilevare e risolvere problemi tecnici o operativi. Gli amministratori possono intervenire rapidamente in caso di ordini bloccati, pagamenti non processati o problemi di comunicazione tra gli utenti.
- **Monitoraggio della Sicurezza:** Gli amministratori possono supervisionare le misure di sicurezza della piattaforma, garantendo che i dati degli utenti siano protetti e che le transazioni siano sicure. Possono anche rilevare e gestire attività sospette o accessi non autorizzati.
- **Configurazione delle Politiche del Sistema:** Gli amministratori possono impostare e modificare politiche relative a tariffe di consegna, commissioni per i ristoranti e incentivi per i rider, garantendo un equilibrio tra sostenibilità economica e soddisfazione degli utenti.
- **Pianificazione e Ottimizzazione Operativa:** Gli amministratori possono utilizzare strumenti avanzati per pianificare le operazioni, come il bilanciamento del carico durante i periodi di alta affluenza, e per ottimizzare le risorse disponibili.
- **Gestione delle Notifiche di Sistema:** La piattaforma consente agli amministratori di inviare notifiche ai clienti, ristoranti o rider in caso di aggiornamenti, problemi tecnici o modifiche ai termini di servizio.
- **Integrazione con Sistemi Esterni:** Gli amministratori possono configurare integrazioni con strumenti di terze parti, come sistemi di pagamento, CRM o software per l'analisi avanzata dei dati, migliorando ulteriormente l'efficienza del sistema.

Le funzionalità per gli amministratori sono progettate per garantire che il sistema rimanga robusto, scalabile e affidabile. Grazie a un set completo di strumenti, gli amministratori possono mantenere la qualità del servizio, migliorare l'esperienza utente e garantire il successo a lungo termine della piattaforma.

2.2.5 Gestione degli ordini

La gestione degli ordini è il cuore operativo della piattaforma, che coordina l'interazione tra clienti, ristoranti e rider. Questa funzionalità garantisce un flusso fluido dall'acquisizione di un ordine fino alla sua consegna, offrendo un'esperienza user-friendly e altamente efficiente. La piattaforma implementa un sistema strutturato che supporta ogni fase del ciclo di vita degli ordini. Di seguito sono descritti i principali aspetti della gestione degli ordini:

1. Acquisizione degli Ordini

- I clienti possono effettuare ordini direttamente dai menu dei ristoranti, personalizzando le loro richieste con opzioni come ingredienti extra, variazioni di porzione e richieste speciali.
- La piattaforma verifica la validità dell'ordine, compresi i dati forniti, la disponibilità dei prodotti richiesti e il pagamento.

2. Notifica agli Attori Coinvolti

- Una volta confermato un ordine, il sistema invia notifiche istantanee ai ristoranti per l'avvio della preparazione e ai clienti come conferma dell'acquisto.
- I rider ricevono notifiche quando un ordine è pronto per la consegna, permettendo loro di accettare la richiesta.

3. Monitoraggio dello Stato dell'Ordine

- Gli utenti possono tracciare lo stato dell'ordine in tempo reale, visualizzando aggiornamenti su ogni fase, come *"Ordine ricevuto"*, *"In preparazione"*, *"Pronto per il ritiro"* e *"In consegna"*.
- Un'interfaccia intuitiva offre una visualizzazione chiara del progresso, sia per i clienti che per i ristoranti e i rider.

4. Integrazione del Pagamento

- Il sistema supporta pagamenti sicuri attraverso metodi diversificati, inclusi carte di credito, portafogli digitali e contanti alla consegna (se applicabile).
- I pagamenti vengono confermati prima di avviare l'elaborazione dell'ordine per garantire l'affidabilità del sistema.

5. Assegnazione degli Ordini ai Rider

- Una volta che un ordine è pronto per la consegna, la piattaforma assegna automaticamente il compito al rider più vicino e disponibile, utilizzando algoritmi di ottimizzazione.
- I rider possono accettare o rifiutare l'assegnazione; in caso di rifiuto, il sistema riassegna rapidamente l'ordine a un altro rider.

6. Gestione delle Modifiche agli Ordini

- I clienti possono apportare modifiche ai loro ordini fino a un certo punto del processo, ad esempio prima che il ristorante inizi la preparazione.

- Il sistema si occupa di aggiornare le informazioni e notificare tutte le parti coinvolte in caso di modifiche.

7. Risoluzione di Problemi e Cancellazioni

- La piattaforma gestisce eventuali problematiche, come cancellazioni dell'ultimo minuto, ordini non disponibili o ritardi imprevisti.
- Gli utenti ricevono notifiche con aggiornamenti e, se necessario, opzioni per il rimborso o la sostituzione.

8. Archivio e Storico degli Ordini

- Tutti gli ordini vengono archiviati nel sistema, consentendo ai clienti di visualizzare lo storico dei loro acquisti.
- I ristoranti e i rider possono accedere ai dettagli degli ordini passati per analisi o verifica.

9. Analisi e Reportistica

- La gestione degli ordini include funzionalità per analizzare i dati operativi, come tempi di consegna medi, frequenza degli ordini e performance dei ristoranti.
- Gli amministratori possono utilizzare queste informazioni per identificare aree di miglioramento e ottimizzare il sistema.

10. Scalabilità e Resilienza

- Il sistema è progettato per gestire picchi di carico durante periodi di alta domanda, garantendo tempi di risposta rapidi e un'elaborazione senza interruzioni.
- Meccanismi di backup assicurano che nessun ordine venga perso in caso di problemi tecnici.

Grazie a un'infrastruttura ben progettata e a processi automatizzati, la gestione degli ordini garantisce che ogni attore coinvolto nella piattaforma riceva un servizio affidabile ed efficiente.

2.2.6 Gestione delle consegne

La gestione delle consegne è una delle funzionalità chiave della piattaforma, responsabile del collegamento tra i ristoranti, i rider e i clienti finali. Questa funzionalità si concentra sull'ottimizzazione delle operazioni di consegna, garantendo che gli ordini vengano recapitati in modo tempestivo, sicuro ed efficiente. La piattaforma utilizza algoritmi avanzati e strumenti di monitoraggio in tempo reale per coordinare l'intero processo. Ecco i principali aspetti della gestione delle consegne:

1. Assegnazione Intelligente delle Consegne:

- La piattaforma assegna automaticamente gli ordini ai rider disponibili, tenendo conto di fattori come posizione attuale, carico di lavoro e tempi stimati di consegna.
- In caso di rifiuto di un ordine, il sistema assegna rapidamente la consegna a un altro rider idoneo, riducendo i ritardi.

2. Ottimizzazione dei Percorsi:

- I rider ricevono indicazioni precise per raggiungere il ristorante e successivamente il cliente, utilizzando percorsi ottimizzati per risparmiare tempo e ridurre i costi.
- La piattaforma considera condizioni del traffico, distanze e possibilità di combinare più consegne lungo un tragitto comune.

3. Monitoraggio in Tempo Reale:

- La posizione dei rider viene monitorata in tempo reale attraverso un sistema di tracciamento GPS integrato.
- I clienti possono visualizzare l'avanzamento della consegna su una mappa, con aggiornamenti regolari sullo stato dell'ordine.

4. Gestione delle Tempistiche:

- Il sistema sincronizza le tempistiche di preparazione degli ordini con l'arrivo dei rider, minimizzando i tempi di attesa nei ristoranti.
- Previsioni accurate dei tempi di consegna vengono comunicate ai clienti, migliorando la trasparenza e le aspettative.

5. Comunicazione tra Rider, Ristoranti e Clienti:

- I rider possono comunicare direttamente con i ristoranti per eventuali chiarimenti sugli ordini e con i clienti per dettagli di consegna specifici, come variazioni dell'indirizzo o codici di accesso.
- Un sistema di messaggistica integrato facilita queste interazioni in modo rapido ed efficace.

6. Gestione della Disponibilità dei Rider:

- I rider possono aggiornare il loro stato di disponibilità tramite l'app dedicata, consentendo alla piattaforma di adattarsi in tempo reale alle variazioni della forza lavoro.

- Questo meccanismo aiuta a garantire che ci siano rider sufficienti per gestire la domanda, soprattutto durante i picchi di attività.

7. Supporto per Consegne Multiple:

- La piattaforma consente ai rider di gestire più consegne contemporaneamente quando i percorsi lo consentono, migliorando l'efficienza operativa.
- Gli ordini vengono raggruppati in base alla posizione geografica e alla priorità, garantendo consegne rapide e precise.

8. Gestione delle Problematiche:

- In caso di imprevisti, come ritardi, indirizzi errati o rider non disponibili, il sistema implementa soluzioni rapide per ridurre al minimo i disagi.
- I clienti vengono informati in tempo reale di eventuali modifiche o ritardi nelle consegne.

9. Archiviazione e Analisi dei Dati:

- Tutte le informazioni sulle consegne vengono archiviate per analisi successive, consentendo agli amministratori di monitorare metriche chiave come tempi di consegna, successo delle consegne e feedback dei clienti.
- Questi dati supportano decisioni strategiche per migliorare l'efficienza e la qualità del servizio.

10. Sicurezza e Affidabilità:

- La piattaforma garantisce la sicurezza dei dati personali di clienti e rider durante la gestione delle consegne.
- I meccanismi di backup e il monitoraggio costante assicurano un servizio affidabile anche in caso di problemi tecnici.

Grazie a questi elementi, la gestione delle consegne offre un'esperienza fluida e soddisfacente per tutti gli utenti della piattaforma, assicurando che gli ordini vengano recapitati nel minor tempo possibile e con elevati standard di qualità.

2.2.7 Scalabilità e prestazioni

La scalabilità e le prestazioni sono aspetti fondamentali della *Food Delivery Platform*, progettata per gestire un alto volume di richieste simultanee e garantire un'esperienza fluida e reattiva agli utenti. Considerando la natura dinamica delle attività sulla piattaforma, come picchi di ordini durante l'ora di pranzo o cena, il sistema è stato ottimizzato per adattarsi alle variazioni di carico in modo efficiente. Di seguito, i principali aspetti relativi a scalabilità e prestazioni:

1. Architettura Scalabile:

- La piattaforma utilizza un'architettura distribuita, che consente l'aggiunta o la rimozione di risorse a seconda delle necessità.

- I componenti del sistema sono stati separati in microservizi, consentendo una gestione indipendente e una scalabilità verticale e orizzontale.
- La separazione dei carichi tra i servizi di gestione ordini, gestione rider, e amministrazione riduce i colli di bottiglia.

2. Bilanciamento del Carico:

- I bilanciatori di carico distribuiscono le richieste degli utenti tra più server, garantendo che nessun server venga sovraccaricato.
- Questo approccio riduce i tempi di risposta e aumenta la tolleranza ai guasti.

3. Ottimizzazione della Consegna dei Dati:

- La piattaforma utilizza meccanismi di caching per ridurre il numero di richieste ripetitive ai database e migliorare le prestazioni complessive.
- Le informazioni frequentemente richieste, come i menu dei ristoranti o lo stato delle consegne, vengono memorizzate temporaneamente nei sistemi di cache distribuiti.

4. Gestione dei Picchi di Carico:

- Durante i periodi di picco, come le festività o gli orari di punta, il sistema si adatta automaticamente aumentando le risorse di calcolo grazie al supporto del cloud computing.
- La coda dei messaggi (RabbitMQ) gestisce l'instradamento degli ordini, assicurando che tutte le richieste vengano elaborate senza perdita di dati.

5. Bassi Tempi di Latenza:

- L'uso di Apache Kafka per l'aggiornamento in tempo reale degli stati degli ordini e delle consegne riduce i tempi di propagazione delle informazioni tra i servizi.
- I tempi di latenza sono monitorati costantemente e ottimizzati per garantire un'esperienza utente fluida.

6. Resilienza e Tolleranza ai Guasti:

- La piattaforma è progettata per continuare a funzionare anche in presenza di guasti hardware o software. Meccanismi di fallback assicurano che i servizi critici restino operativi.
- Backup e sistemi di replica garantiscono che i dati non vengano persi in caso di interruzioni.

7. Test delle Prestazioni:

- Il sistema è sottoposto regolarmente a test di carico per simulare scenari di alta affluenza e individuare potenziali colli di bottiglia.
- I risultati dei test vengono utilizzati per ottimizzare ulteriormente il sistema e pianificare l'espansione futura.

8. Metriche e Monitoraggio:

- Strumenti avanzati di monitoraggio tengono traccia delle prestazioni del sistema, come i tempi medi di risposta, il numero di richieste al secondo e l'utilizzo delle risorse.
- Le anomalie vengono rilevate in tempo reale, consentendo interventi immediati per risolvere eventuali problemi.

9. Riduzione dei Tempi Morti:

- Gli aggiornamenti del sistema vengono implementati utilizzando strategie di deployment continuo, minimizzando i tempi di inattività per gli utenti.
- Le funzionalità chiave rimangono attive anche durante gli aggiornamenti grazie a sistemi di failover.

10. Espansione Futura:

- La piattaforma è progettata con un'ottica di lungo termine, pronta a integrare nuove funzionalità e a espandersi in nuovi mercati geografici senza compromettere le prestazioni.
- La modularità del sistema garantisce che l'espansione possa avvenire senza modificare significativamente i componenti esistenti.

Grazie a questi meccanismi, la *Food Delivery Platform* è in grado di soddisfare la crescente domanda e offrire un servizio affidabile, reattivo e scalabile per tutti gli utenti coinvolti.

2.2.8 Sicurezza e privacy

La sicurezza e la privacy sono priorità fondamentali per la *Food Delivery Platform*, in quanto trattano dati sensibili relativi a clienti, ristoranti, rider e amministratori. La protezione di questi dati è essenziale per garantire la fiducia degli utenti e la conformità alle normative legali, come il GDPR per la gestione dei dati personali. La piattaforma è progettata con un'architettura robusta e con misure di sicurezza avanzate per proteggere le informazioni e le transazioni. Di seguito sono illustrati i principali aspetti relativi alla sicurezza e alla privacy.

1. Autenticazione e Autorizzazione Sicura:

- Per garantire l'accesso ai dati e alle funzionalità sensibili solo agli utenti autorizzati, la piattaforma implementa meccanismi di autenticazione avanzati, come l'autenticazione a due fattori (2FA) e il Single Sign-On (SSO) per semplificare l'accesso degli utenti senza compromettere la sicurezza.
- I privilegi degli utenti sono gestiti tramite controlli di accesso basati sui ruoli (RBAC), assicurando che ogni attore (cliente, ristorante, rider, amministratore) abbia accesso solo alle informazioni e funzionalità di cui è responsabile.

2. Crittografia dei Dati:

- Tutti i dati sensibili, inclusi i dati di pagamento, le informazioni sugli ordini e i dati personali degli utenti, sono criptati sia in transito che a riposo utilizzando tecnologie di crittografia avanzate (ad esempio, TLS per la protezione delle comunicazioni in transito e AES per la crittografia dei dati a riposo).
- Questo assicura che i dati non possano essere letti o manipolati da soggetti non autorizzati durante il trasferimento o la memorizzazione.

3. Protezione delle Transazioni Finanziarie:

- Per garantire la sicurezza delle transazioni, la piattaforma integra soluzioni di pagamento sicure e conformi agli standard di sicurezza delle carte di pagamento (PCI DSS).
- Le informazioni sensibili relative ai pagamenti non vengono memorizzate nei sistemi della piattaforma, ma vengono gestite da provider di pagamento esterni che offrono soluzioni sicure e certificate.

4. Monitoraggio delle Attività e Registrazione degli Eventi:

- Il sistema registra tutte le attività critiche, come login, modifiche ai dati degli utenti e operazioni sugli ordini, in log sicuri e immutabili.
- I log vengono monitorati in tempo reale per rilevare comportamenti sospetti o attività malevoli, come tentativi di accesso non autorizzato o anomalie nelle transazioni.

5. Protezione contro Attacchi:

- La piattaforma è protetta contro i principali tipi di attacchi informatici, tra cui attacchi DDoS, iniezioni SQL, attacchi XSS (Cross-Site Scripting) e CSRF (Cross-Site Request Forgery), attraverso l'adozione di firewall applicativi, filtri di contenuto e controlli di validazione rigorosi.
- Vengono inoltre implementati sistemi di rilevamento delle intrusioni (IDS) per identificare e rispondere a potenziali minacce in tempo reale.

6. Gestione della Privacy degli Utenti:

- La piattaforma raccoglie solo i dati necessari per il funzionamento dei servizi, come l'indirizzo di consegna e le preferenze alimentari, e garantisce che questi dati siano trattati con la massima riservatezza.
- Gli utenti hanno il controllo sui propri dati personali e possono modificarli, eliminarli o richiedere una copia in qualsiasi momento. Inoltre, vengono forniti strumenti per gestire le preferenze relative alla privacy, come la possibilità di disabilitare la raccolta di alcuni dati di tracciamento.

7. Conformità alle Leggi sulla Privacy:

- La piattaforma è progettata per essere conforme alle normative sulla protezione dei dati, come il GDPR (General Data Protection Regulation) in Europa e altre leggi simili in altre giurisdizioni.

- Vengono adottate politiche rigorose per garantire che i dati degli utenti siano trattati correttamente, con il consenso esplicito degli utenti per la raccolta e l'elaborazione delle informazioni.

8. Sicurezza dei Sistemi di Comunicazione:

- Le comunicazioni tra i vari componenti della piattaforma (client, server, ristoranti, rider) sono protette tramite canali sicuri (HTTPS, WebSockets sicuri) per evitare intercettazioni o manomissioni.
- L'integrazione con RabbitMQ e Kafka avviene in modo sicuro, con autenticazione e crittografia applicate ai messaggi inviati tra i diversi microservizi.

9. Formazione e Consapevolezza sulla Sicurezza:

- Il personale della piattaforma, inclusi ristoranti e rider, riceve formazione regolare sulle best practice di sicurezza, come la gestione delle password, il riconoscimento dei tentativi di phishing e l'importanza di mantenere i propri dispositivi sicuri.

10. Piani di Risposta agli Incidenti:

- La piattaforma dispone di un piano di risposta agli incidenti per affrontare tempestivamente eventuali violazioni della sicurezza o perdite di dati. Il piano include procedure di notifica agli utenti, misure per mitigare il danno e azioni correttive per prevenire futuri incidenti.

Con queste misure di sicurezza avanzate, la *Food Delivery Platform* è in grado di proteggere i dati sensibili degli utenti, garantire transazioni sicure e mantenere la fiducia degli attori coinvolti, assicurando un ambiente di utilizzo sicuro e conforme alle normative vigenti.

2.3 Requisiti Specifici

I requisiti specifici sono una parte fondamentale della progettazione e dello sviluppo della piattaforma di food delivery, poiché definiscono in modo dettagliato ciò che il sistema deve fare e come deve comportarsi. Questi requisiti sono suddivisi in due categorie principali: **requisiti funzionali** e **requisiti non funzionali**. I **requisiti funzionali** sono legati alle operazioni quotidiane e alle funzionalità che la piattaforma deve offrire per soddisfare le esigenze degli utenti. Questi includono la possibilità di visualizzare e ordinare menu, gestire ordini, tracciare le consegne in tempo reale, pagare, e personalizzare l'esperienza d'uso in base alle preferenze degli utenti. Ogni attore del sistema—che si tratti di un cliente, di un ristorante, di un rider o di un amministratore—ha requisiti funzionali specifici che devono essere implementati per garantire un'esperienza fluida ed efficiente. D'altro canto, i **requisiti non funzionali** riguardano le qualità complessive del sistema e descrivono caratteristiche essenziali come le prestazioni, la scalabilità, la sicurezza e l'affidabilità del sistema. Questi requisiti, seppur non direttamente legati a funzionalità specifiche, sono cruciali per garantire che la piattaforma possa supportare carichi di lavoro elevati, gestire grandi volumi di traffico, proteggere i dati degli utenti e offrire una risposta tempestiva anche durante i picchi di utilizzo. I requisiti non funzionali influenzano direttamente sull'usabilità e sulla sostenibilità a lungo termine del sistema, quindi sono altrettanto importanti quanto quelli funzionali.

In questa sezione, esploreremo in dettaglio entrambi i gruppi di requisiti, fornendo una base solida per la progettazione e lo sviluppo della piattaforma, che dovrà rispondere alle esigenze degli utenti finali e rispettare gli standard di qualità e performance previsti.

2.3.1 Requisiti Funzionali

2.3.1.1 Requisiti Funzionali per i Ristoranti

I requisiti funzionali per i ristoranti definiscono le funzionalità che la piattaforma deve offrire agli utenti appartenenti a questa categoria per consentire loro di gestire efficacemente la propria attività e interagire con gli altri attori del sistema. Questi requisiti mirano a semplificare il processo di gestione dei menu, degli ordini, delle disponibilità e delle performance, garantendo che i ristoranti possano offrire un servizio ottimale ai clienti e una gestione efficiente delle loro operazioni.

1. Gestione del Menu

- Il ristorante deve poter aggiungere, modificare o rimuovere gli articoli del proprio menu in tempo reale.
- Deve essere possibile configurare categorie di piatti (ad esempio, antipasti, piatti principali, dessert, bevande) e descrivere ogni articolo con dettagli come ingredienti, allergeni, prezzo e immagini.
- Il ristorante deve poter evidenziare offerte speciali o promozioni in modo visibile per i clienti.

2. Gestione degli Ordini

- I ristoranti devono ricevere notifiche in tempo reale ogni volta che un nuovo ordine viene effettuato da un cliente.

- Devono essere in grado di visualizzare i dettagli degli ordini, tra cui i piatti selezionati, le personalizzazioni richieste e le informazioni sul cliente (indirizzo, preferenze speciali, etc.).
- Il sistema deve consentire al ristorante di aggiornare lo stato degli ordini (ad esempio, accettato, in preparazione, pronto per il ritiro).
- Il ristorante deve avere la possibilità di aggiungere note o modifiche agli ordini se necessario, e di comunicare con il cliente per eventuali richieste speciali.

3. Gestione delle Disponibilità e degli Orari di Apertura

- Il ristorante deve poter impostare orari di apertura e chiusura, con la possibilità di modificare gli orari speciali per giorni festivi o altre occasioni.
- Deve esserci la possibilità di impostare la disponibilità per determinate fasce orarie, in modo da gestire al meglio l'afflusso di ordini durante le ore di punta.

4. Gestione delle Preferenze del Ristorante e delle Impostazioni del Profilo

- Il ristorante deve poter aggiornare e gestire le informazioni del proprio profilo, come il nome, la descrizione, l'indirizzo, il numero di telefono e altre informazioni di contatto.
- Deve essere in grado di caricare un logo e immagini rappresentative del ristorante.
- Deve poter configurare le preferenze relative alla gestione delle recensioni e dei commenti dei clienti.

5. Monitoraggio delle Performance e delle Statistiche

- Il ristorante deve poter visualizzare report dettagliati riguardanti le proprie performance, come il numero di ordini ricevuti, il fatturato, i piatti più ordinati, la valutazione media da parte dei clienti e i tempi di preparazione.
- Deve essere possibile visualizzare queste statistiche in tempo reale, con la possibilità di filtrare i dati per periodi specifici (giorno, settimana, mese, anno).

6. Gestione delle Richieste Speciali e Personalizzazioni

- I ristoranti devono essere in grado di gestire richieste speciali provenienti dai clienti, come preferenze dietetiche, allergie o personalizzazioni nei piatti.
- Il sistema deve consentire al ristorante di specificare se una determinata richiesta può essere soddisfatta o meno e di comunicare eventuali limitazioni al cliente.

7. Comunicazione con i Rider

- Il ristorante deve poter comunicare facilmente con i rider per coordinare la consegna degli ordini. Ciò include la possibilità di fornire indicazioni aggiuntive sui percorsi, modificare lo stato dell'ordine o risolvere eventuali problematiche relative alla consegna.

8. Gestione delle Recensioni

- Il ristorante deve poter visualizzare e rispondere alle recensioni lasciate dai clienti. Inoltre, deve poter monitorare il punteggio medio delle proprie recensioni.
- Deve avere la possibilità di segnalare recensioni inappropriate o non conformi alle politiche della piattaforma.

Questi requisiti permettono ai ristoranti di gestire in modo completo le proprie operazioni all'interno della piattaforma, garantendo un'esperienza ottimale sia per i clienti che per il personale del ristorante.

2.3.1.2 Requisiti Funzionali per i Clienti

I requisiti funzionali per i clienti descrivono le funzionalità che la piattaforma deve offrire per garantire un'esperienza di acquisto e interazione fluida, soddisfacente e personalizzata. I clienti devono poter esplorare i ristoranti, effettuare ordini personalizzati, monitorare lo stato delle consegne, e interagire con i ristoranti e i rider in modo semplice ed efficiente.

1. Esplorazione dei Ristoranti e dei Menu

- I clienti devono poter navigare facilmente tra i ristoranti disponibili nella piattaforma, visualizzando informazioni dettagliate come il tipo di cucina, le recensioni, gli orari di apertura e le offerte speciali.
- Devono essere in grado di filtrare e ordinare i ristoranti in base a parametri come la valutazione, la distanza, il prezzo e la disponibilità.
- Ogni ristorante deve avere un menu digitale che il cliente può sfogliare, con dettagli per ciascun piatto, come ingredienti, prezzo, e immagini.

2. Ricerca di Cibi o Ristoranti

- I clienti devono poter utilizzare una barra di ricerca per trovare specifici piatti o ristoranti.
- Devono essere in grado di cercare per nome, categoria di cibo (ad esempio, pizza, sushi, hamburger) o per ingredienti.
- Il sistema deve suggerire ristoranti o piatti basati su ricerche passate e preferenze.

3. Effettuare e Personalizzare Ordini

- I clienti devono poter aggiungere piatti al carrello e personalizzarli in base alle loro preferenze (ad esempio, scegliere ingredienti aggiuntivi, cambiare porzioni, rimuovere ingredienti non graditi).
- Devono poter aggiungere note speciali agli ordini, come richieste dietetiche (vegetariano, senza glutine, ecc.).
- Il sistema deve permettere di salvare ordini ricorrenti o personalizzati per riordinarli rapidamente.

4. Tracciamento degli Ordini in Tempo Reale

- Una volta effettuato un ordine, il cliente deve poter monitorare in tempo reale lo stato dell'ordine, visualizzando la fase di preparazione del piatto e la posizione del rider.
- Il sistema deve fornire aggiornamenti in tempo reale sullo stato di avanzamento dell'ordine e su eventuali modifiche (ritardi, cambiamenti nell'ordine, ecc.).

5. Pagamenti Sicuri

- I clienti devono poter pagare l'ordine tramite vari metodi di pagamento, inclusi carte di credito/debito, portafogli elettronici, o pagamenti in contante alla consegna.
- Il sistema deve garantire una gestione sicura delle transazioni finanziarie, con crittografia dei dati sensibili e protezione contro le frodi.
- Il cliente deve ricevere una conferma immediata del pagamento, con la possibilità di ottenere una ricevuta elettronica.

6. Recensioni e Valutazioni

- Dopo aver ricevuto l'ordine, il cliente deve poter lasciare una recensione sul ristorante e sul servizio di consegna.
- Deve essere possibile assegnare una valutazione in stelle e scrivere commenti riguardo la qualità del cibo, il servizio e l'esperienza complessiva.
- Il cliente deve poter visualizzare le recensioni degli altri utenti per fare scelte più informate.

7. Storico Ordini

- I clienti devono poter visualizzare un elenco completo degli ordini passati, con la possibilità di ripetere un ordine precedente con un solo clic.
- Ogni ordine deve essere visualizzabile nei dettagli, comprendendo la data, i piatti ordinati, il prezzo, e lo stato di consegna.

8. Gestione del Profilo e delle Impostazioni

- I clienti devono poter creare e gestire il proprio profilo, includendo informazioni personali come nome, indirizzo, numero di telefono e preferenze alimentari.
- Devono poter modificare le impostazioni dell'account, come password, metodi di pagamento e indirizzi di spedizione.
- Il sistema deve permettere ai clienti di configurare le notifiche (ad esempio, ricevere aggiornamenti sull'ordine via SMS o e-mail) e le preferenze di privacy.

9. Promozioni e Offerte Speciali

- I clienti devono poter usufruire di promozioni speciali o sconti disponibili attraverso la piattaforma, come codici promozionali o offerte per ordini ricorrenti.

- Devono ricevere notifiche su eventuali offerte personalizzate in base ai loro acquisti precedenti o alla loro posizione geografica.

10. Assistenza Clienti

- Il cliente deve poter entrare in contatto con il supporto clienti per risolvere eventuali problematiche relative agli ordini, ai pagamenti, o a qualsiasi altro aspetto dell'esperienza.
- Deve essere possibile richiedere assistenza tramite chat, e-mail o telefono, con risposte tempestive alle richieste urgenti.

Questi requisiti mirano a garantire che l'esperienza d'uso per i clienti sia quanto più semplice, sicura e personalizzata possibile, facilitando l'interazione con la piattaforma e aumentando la soddisfazione del cliente finale.

2.3.1.3 Requisiti Funzionali per i Rider

I requisiti funzionali per i rider descrivono le funzionalità che la piattaforma deve offrire per garantire un'efficace gestione delle consegne e ottimizzare il lavoro dei rider. I rider sono un attore fondamentale nel processo di consegna del cibo, e la piattaforma deve permettere loro di ricevere, gestire e completare gli ordini in modo efficiente.

1. Ricezione delle Richieste di Consegna

- I rider devono ricevere notifiche in tempo reale per ogni nuova richiesta di consegna, che includano informazioni cruciali come l'indirizzo di ritiro, l'indirizzo di destinazione e il tempo stimato di consegna.
- Le richieste di consegna devono essere assegnate in base alla posizione e disponibilità del rider, con un sistema di prioritizzazione che ottimizzi l'efficienza.

2. Accettazione o Rifiuto delle Richieste

- I rider devono poter accettare o rifiutare una richiesta di consegna in base alla loro disponibilità e posizione.
- Se un rider rifiuta una consegna, il sistema deve assegnare l'ordine ad un altro rider disponibile nelle vicinanze.

3. Gestione delle Consegne Multiple

- I rider devono poter ricevere e gestire più consegne contemporaneamente, se i percorsi e le destinazioni lo permettono.
- Il sistema deve ottimizzare l'assegnazione delle consegne in modo che i rider possano effettuare più consegne in un'unica corsa, riducendo i tempi e aumentando l'efficienza.

4. Navigazione e Pianificazione del Percorso

- I rider devono ricevere percorsi ottimizzati per le consegne, in base alla loro posizione attuale, al traffico e alla destinazione.
- Il sistema deve integrare mappe in tempo reale e aggiornamenti sul traffico per guidare i rider nel percorso più veloce e sicuro.

5. Aggiornamento dello Stato della Consegna

- I rider devono poter aggiornare lo stato delle consegne in tempo reale, indicando quando hanno preso in carico l'ordine, quando sono in viaggio e quando hanno completato la consegna.
- Ogni aggiornamento deve essere visibile al cliente e al ristorante, per garantire una comunicazione trasparente sullo stato dell'ordine.

6. Gestione della Disponibilità

- I rider devono poter aggiornare la loro disponibilità sulla piattaforma, indicando se sono disponibili per nuove consegne o se sono in pausa.
- Devono avere la possibilità di pianificare i loro turni di lavoro e gestire in tempo reale la loro disponibilità per le consegne.

7. Comunicazione con i Clienti e i Ristoranti

- I rider devono poter comunicare direttamente con i clienti e i ristoranti in caso di necessità, come per chiarire le istruzioni speciali, segnalare ritardi o chiedere indicazioni aggiuntive.
- La piattaforma deve garantire un sistema di messaggistica sicuro che permetta di mantenere la privacy delle informazioni di contatto.

8. Monitoraggio delle Prestazioni e delle Entrate

- I rider devono poter visualizzare il proprio storico delle consegne, compreso il numero di ordini completati, la distanza percorsa, il tempo di consegna e le entrate guadagnate.
- Il sistema deve generare report periodici che mostrano le statistiche sulle prestazioni, permettendo ai rider di monitorare e migliorare la loro efficienza.

9. Gestione del Profilo e delle Impostazioni

- I rider devono poter creare e aggiornare il loro profilo, includendo informazioni personali come il nome, il numero di telefono, l'indirizzo e il veicolo utilizzato per le consegne.
- Devono avere la possibilità di modificare le impostazioni del proprio account, come la modalità di pagamento delle entrate (ad esempio, pagamento settimanale o giornaliero).

10. Gestione delle Valutazioni e delle Recensioni

- I rider devono poter ricevere feedback dai clienti riguardo alla qualità del servizio di consegna.
- Le recensioni devono essere visibili ai rider, che possono utilizzarle per migliorare il loro servizio.

11. Gestione delle Situazioni Improvvisate

- I rider devono essere in grado di segnare eventuali situazioni impreviste che possano influire sulla consegna, come traffico imprevisto, incidenti, o problemi con l'indirizzo di consegna.
- Il sistema deve prevedere una gestione efficiente di tali situazioni, notificando il cliente e il ristorante in caso di modifiche o ritardi.

12. Ricezione delle Istruzioni Speciali

- I rider devono poter visualizzare le istruzioni speciali fornite dai clienti, come richieste specifiche sul tipo di confezionamento o preferenze nella consegna (ad esempio, lasciarlo alla porta, suonare il campanello, ecc.).

Questi requisiti permettono ai rider di interagire con la piattaforma in modo efficiente e di gestire tutte le fasi della consegna, dalla ricezione dell'ordine alla sua consegna finale al cliente, garantendo che l'esperienza sia fluida e soddisfacente per tutti gli attori coinvolti.

2.3.1.4 Requisiti Funzionali per gli Amministratori

Gli amministratori sono i responsabili del funzionamento generale della piattaforma. I loro requisiti funzionali riguardano la gestione dei dati degli utenti, la supervisione delle operazioni, il monitoraggio delle performance e la generazione di report. L'interfaccia amministrativa deve consentire di gestire il sistema in modo efficiente e garantire che la piattaforma operi senza interruzioni.

1. Gestione degli Account degli Utenti

- Gli amministratori devono poter visualizzare e gestire gli account di tutti gli utenti della piattaforma, inclusi clienti, ristoranti e rider.
- Devono essere in grado di creare, modificare, bloccare o eliminare account degli utenti.
- Devono avere la possibilità di resettare le password degli utenti in caso di necessità e di inviare notifiche tramite email per aggiornamenti importanti.

2. Monitoraggio delle Performance della Piattaforma

- Gli amministratori devono avere accesso a dashboard in tempo reale che mostrano le performance generali della piattaforma, come il numero di ordini elaborati, il numero di consegne completate, e la soddisfazione dei clienti.
- Devono poter monitorare le metriche di performance della flotta di rider, come il tempo medio di consegna, il numero di consegne per rider e il guadagno per rider.

3. Generazione di Report

- Gli amministratori devono poter generare report dettagliati su vari aspetti operativi, come le performance dei ristoranti, il numero di ordini giornalieri, le tendenze di consumo dei clienti, e le entrate complessive della piattaforma.

- Devono essere in grado di esportare i report in vari formati (es. PDF, Excel) per analisi approfondite.
- I report possono includere anche analisi sulla qualità del servizio, come la valutazione dei rider e dei ristoranti da parte dei clienti.

4. Gestione delle Transazioni e Pagamenti

- Gli amministratori devono poter monitorare tutte le transazioni effettuate sulla piattaforma, inclusi i pagamenti effettuati dai clienti e i pagamenti ai ristoranti e ai rider.
- Devono avere la possibilità di gestire eventuali richieste di rimborso, risolvendo situazioni di pagamento problematiche.
- Il sistema deve consentire la visualizzazione delle transazioni sospette e la gestione dei fondi, per garantire che i pagamenti siano completati correttamente e in tempo.

5. Gestione delle Impostazioni della Piattaforma

- Gli amministratori devono poter configurare le impostazioni generali della piattaforma, inclusi i parametri di sicurezza, la gestione dei permessi degli utenti e le politiche di pagamento.
- Devono essere in grado di impostare le politiche di servizio, come la gestione dei resi e la tempistica delle consegne.
- Devono avere la possibilità di personalizzare la piattaforma con nuovi servizi o modifiche alle funzionalità esistenti, mantenendo la compatibilità con le esigenze operative.

6. Supervisione delle Operazioni in Tempo Reale

- Gli amministratori devono poter monitorare in tempo reale le operazioni di consegna, con un'attenzione particolare al flusso degli ordini, alla disponibilità dei rider e alla gestione delle risorse nei ristoranti.
- Devono ricevere notifiche di anomalie nel sistema, come errori nei pagamenti, problemi con le consegne o reclami dei clienti.
- Devono avere un sistema di allerta che segnali qualsiasi interruzione del servizio, per poter intervenire prontamente.

7. Gestione dei Reclami e del Servizio Clienti

- Gli amministratori devono poter accedere e gestire i reclami dei clienti, ristoranti e rider, garantendo che le problematiche vengano risolte in modo tempestivo e soddisfacente.
- Devono essere in grado di visualizzare il feedback dei clienti e prendere provvedimenti per migliorare la qualità del servizio (es. modifiche ai processi o aggiustamenti alle politiche).

8. Controllo della Sicurezza e della Privacy dei Dati

- Gli amministratori devono avere accesso alle informazioni sensibili degli utenti solo se strettamente necessario per risolvere problemi operativi o amministrativi.
- Devono essere in grado di monitorare l'accesso ai dati e gestire la sicurezza della piattaforma, implementando controlli per prevenire accessi non autorizzati o abusi dei dati.
- Devono gestire la conformità alle normative sulla protezione dei dati personali, come il GDPR.

9. Gestione della Disponibilità e delle Operazioni dei Ristoranti

- Gli amministratori devono poter gestire l'elenco dei ristoranti presenti sulla piattaforma, approvare o rimuovere i ristoranti registrati e monitorare le operazioni quotidiane.
- Devono avere la possibilità di visualizzare e gestire la disponibilità dei ristoranti e l'orario di apertura, aggiornando i dettagli per evitare errori nelle operazioni.

10. Configurazione delle Promozioni e Offerte Speciali

- Gli amministratori devono poter creare e gestire campagne promozionali, come sconti, offerte speciali o programmi fedeltà per attrarre clienti.
- Devono essere in grado di configurare la durata delle promozioni, i criteri di applicazione e monitorare l'efficacia delle campagne promozionali.

11. Gestione dei Servizi Aggiuntivi

- Gli amministratori devono poter abilitare o disabilitare servizi aggiuntivi sulla piattaforma, come la possibilità per i ristoranti di personalizzare il menu con opzioni speciali o per i rider di scegliere tra diversi metodi di pagamento.
- Devono avere il controllo su eventuali nuovi servizi o modifiche che impattano sul funzionamento generale della piattaforma.

Questi requisiti funzionali permettono agli amministratori di monitorare e gestire l'intero ecosistema della piattaforma, garantendo che tutti gli attori coinvolti – clienti, ristoranti e rider – abbiano un'esperienza ottimale e che il sistema funzioni in modo sicuro, efficiente e conforme alle normative vigenti.

2.3.2 Requisiti non Funzionali

I requisiti non funzionali descrivono gli aspetti del sistema che influenzano il comportamento del sistema stesso, come prestazioni, sicurezza, usabilità e manutenibilità. Questi requisiti non riguardano direttamente le funzioni specifiche che il sistema deve eseguire, ma definiscono le caratteristiche generali necessarie per garantire un funzionamento ottimale e soddisfacente in diversi scenari. Di seguito sono riportati i principali requisiti non funzionali per la piattaforma di food delivery:

1. Prestazioni e Scalabilità

- **Tempo di risposta:** Il sistema deve garantire un tempo di risposta rapido per tutte le operazioni principali, come la visualizzazione dei menu, l'invio degli ordini e l'aggiornamento dello stato delle consegne. Le operazioni devono essere completate entro un tempo massimo di 3 secondi.
- **Scalabilità:** Il sistema deve essere in grado di gestire un aumento significativo del traffico durante i periodi di alta richiesta (es. ore di punta o festività). Deve supportare una scalabilità orizzontale per aumentare la capacità di elaborazione, consentendo di aggiungere più risorse (server, istanze) senza compromettere le prestazioni.
- **Gestione dei picchi di carico:** Il sistema deve essere progettato per gestire picchi di carico elevati senza compromettere la stabilità. Ad esempio, durante un evento di grande affluenza, la piattaforma dovrebbe essere in grado di continuare a elaborare ordini senza rallentamenti significativi.

2. Affidabilità e Disponibilità

- **Disponibilità:** Il sistema deve essere disponibile 24 ore su 24, 7 giorni su 7, con un tempo di inattività massimo consentito (es. per manutenzione) non superiore a 4 ore al mese.
- **Recupero dai guasti:** Il sistema deve essere in grado di recuperare rapidamente da guasti o interruzioni. Dev'essere implementata una strategia di backup e ripristino dei dati, in modo che il sistema possa riprendere le operazioni normali con il minimo impatto sugli utenti.
- **Tolleranza agli errori:** Il sistema deve essere tollerante agli errori, cioè deve continuare a funzionare anche in caso di malfunzionamenti parziali (es. un nodo di rete che si guasta o un server che smette di rispondere).

3. Sicurezza

- **Protezione dei dati sensibili:** I dati personali dei clienti, ristoranti e rider devono essere protetti in conformità alle normative sulla privacy, come il GDPR. Le informazioni sensibili (ad esempio, numeri di carta di credito, indirizzi di consegna) devono essere cifrate durante la trasmissione (SSL/TLS) e a riposo nel database.
- **Autenticazione e autorizzazione:** Il sistema deve implementare meccanismi di autenticazione sicura (ad esempio, tramite password o autenticazione a due fattori) e autorizzazione per garantire che gli utenti possano accedere solo alle risorse per le quali hanno i permessi.

- **Prevenzione degli attacchi:** Il sistema deve essere protetto contro gli attacchi informatici comuni, come attacchi DDoS, SQL injection e cross-site scripting (XSS). Inoltre, deve essere possibile monitorare e tracciare le attività sospette per prevenire accessi non autorizzati.

4. Usabilità

- **Interfaccia utente:** L'interfaccia utente deve essere intuitiva, semplice e facile da navigare per tutti gli utenti (clienti, ristoranti, rider e amministratori). Dev'essere accessibile sia tramite desktop che dispositivi mobili, ottimizzando la visualizzazione per schermi di diverse dimensioni.
- **Esperienza utente:** La piattaforma deve garantire una buona esperienza utente (UX), riducendo al minimo il numero di passaggi necessari per completare un ordine, accedere a informazioni e completare una consegna. I clienti e i rider devono poter interagire con il sistema senza difficoltà, anche se non sono esperti nell'uso di tecnologie.
- **Supporto multilingue:** Il sistema deve supportare almeno due lingue (ad esempio, italiano e inglese), permettendo agli utenti di scegliere la lingua preferita per l'interfaccia. L'utente deve essere in grado di cambiare facilmente la lingua durante l'uso della piattaforma.

5. Manutenibilità

- **Modularità:** Il sistema deve essere progettato in modo modulare, consentendo la facile aggiunta di nuove funzionalità o modifiche senza influire sulle operazioni correnti. Ad esempio, nuove funzionalità come offerte promozionali o metodi di pagamento alternativi devono poter essere integrate senza interruzioni.
- **Documentazione:** Deve essere fornita una documentazione completa e chiara per gli sviluppatori e gli amministratori del sistema. La documentazione deve includere informazioni sul design, la struttura del database, le API, le modalità di deploy e i processi operativi.
- **Monitoraggio e logging:** Il sistema deve implementare un sistema di monitoraggio per tracciare il funzionamento delle varie componenti e generare log dettagliati per l'analisi di performance e la risoluzione dei problemi.

6. Conformità alle Normative

- **Conformità alle normative sulla privacy:** Il sistema deve essere conforme alle normative relative alla protezione dei dati personali (ad esempio, GDPR per l'Unione Europea). Ciò include la gestione del consenso dell'utente, il diritto di accesso ai dati e la gestione delle richieste di cancellazione dei dati.
- **Normative fiscali e legali:** Il sistema deve rispettare le normative fiscali e legali applicabili, come le leggi locali sui pagamenti elettronici, la gestione delle tasse e la protezione dei consumatori. Dev'essere in grado di generare report fiscali corretti per supportare i ristoranti e gli amministratori nella gestione delle transazioni.

7. Compatibilità

- **Compatibilità cross-platform:** La piattaforma deve funzionare senza problemi su diversi sistemi operativi (Windows, macOS, Linux) e su dispositivi mobili (Android, iOS). Deve essere ottimizzata per funzionare correttamente su browser web moderni, come Chrome, Firefox, Safari ed Edge.
- **Integrazione con sistemi esterni:** La piattaforma deve essere progettata per integrarsi facilmente con altri sistemi esterni, come gateway di pagamento, sistemi di geolocalizzazione e servizi di notifica.

8. Efficienza Energetica

- **Ottimizzazione delle risorse:** Il sistema deve essere progettato per ridurre al minimo l'uso di risorse, come la memoria e la potenza di calcolo, per garantire un funzionamento efficiente e ridurre i costi operativi. Ad esempio, la gestione delle risorse server deve essere ottimizzata in modo che vengano utilizzati solo quando necessario.

Questi requisiti non funzionali sono fondamentali per garantire che la piattaforma di food delivery non solo soddisfi le aspettative degli utenti in termini di funzionalità, ma anche in termini di performance, sicurezza, usabilità e conformità alle normative. Essi permettono al sistema di operare in modo continuo, affidabile e sicuro, anche in scenari di alta domanda e in un contesto normativo in continua evoluzione.

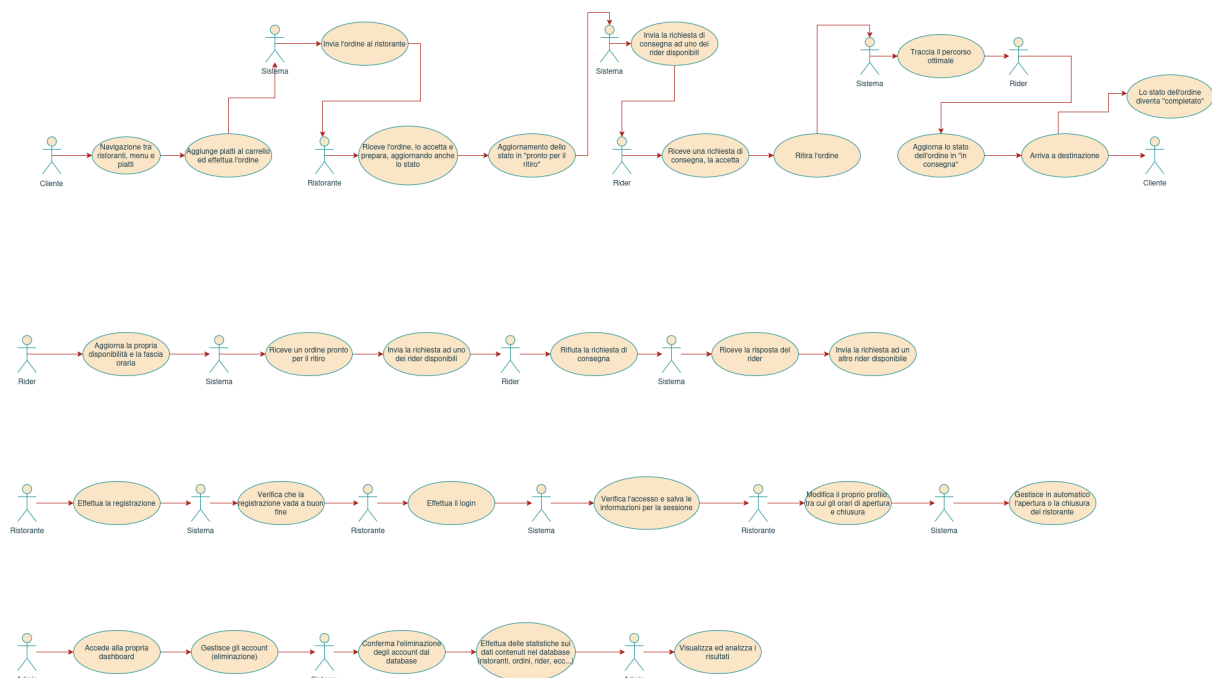


Figura 3: Diagrammi UML

3 Architettura del Sistema

L'architettura di un sistema complesso come una piattaforma di food delivery deve rispondere a molteplici esigenze, inclusi prestazioni elevate, scalabilità, sicurezza e affidabilità. La progettazione architetturale deve quindi tenere in considerazione la gestione dei flussi di dati in tempo reale, l'interoperabilità tra i diversi attori coinvolti (clienti, ristoranti, rider, amministratori), e la facilità di manutenzione e aggiornamento del sistema.

Nel caso della piattaforma di food delivery, è stata adottata un'architettura monolitica basata sul modello **MVC (Model-View-Controller)**. Questo approccio consente di separare chiaramente le responsabilità tra i dati e la logica di business (Model), la presentazione dell'interfaccia utente (View) e il controllo delle interazioni tra i due (Controller). Questa separazione facilita lo sviluppo, poiché consente di lavorare su singole componenti in modo indipendente, mantenendo comunque un buon livello di coesione all'interno del sistema.

Il modello MVC ha il vantaggio di favorire una gestione centralizzata delle funzionalità, semplificando il processo di integrazione tra i vari moduli del sistema. La struttura monolitica iniziale offre inoltre un'architettura più semplice da gestire e distribuire, riducendo la complessità del sistema nelle fasi iniziali di sviluppo e implementazione.

Uno degli aspetti chiave della piattaforma è la gestione efficace degli ordini e delle consegne, che richiedono una comunicazione fluida tra il sistema e i vari utenti in tempo reale. La piattaforma deve gestire grandi volumi di traffico, particolarmente durante le ore di punta, senza compromettere l'affidabilità del servizio. Per raggiungere questo obiettivo, il sistema è progettato in modo da garantire prestazioni ottimali, con l'utilizzo di tecnologie scalabili e con un'architettura in grado di crescere e adattarsi alle nuove necessità.

Inoltre, la sicurezza è una priorità fondamentale. La protezione dei dati sensibili degli utenti, come le informazioni di pagamento, e la gestione sicura delle comunicazioni tra ristoranti, rider e clienti, sono garantite attraverso rigorosi protocolli di sicurezza e cifratura.

In questo capitolo, verrà analizzata la struttura dell'architettura del sistema, esaminando in dettaglio come i vari componenti interagiscono tra di loro e come sono stati progettati per soddisfare le necessità operative e le aspettative degli utenti, mantenendo una buona performance, sicurezza e manutenibilità.

3.1 Architettura Generale

L'architettura della piattaforma di food delivery è progettata per rispondere a requisiti complessi, integrando diverse funzionalità che coinvolgono attori distinti come clienti, ristoranti, rider e amministratori. L'approccio architetturale scelto è quello di una **piattaforma centralizzata**, che integra tutti i processi in un unico sistema monolitico ma modulare, il quale consente una gestione efficiente di tutte le operazioni.

Una delle scelte principali riguarda la **separazione delle preoccupazioni** tra i vari livelli del sistema, al fine di ottimizzare la manutenibilità e la scalabilità dell'applicazione. Per fare ciò, è stato adottato il modello **MVC (Model-View-Controller)**, che permette di separare la logica applicativa (model) dalla gestione dell'interfaccia utente (view) e dal controllo dei flussi di interazione (controller). Questo approccio semplifica lo sviluppo e il debug del sistema, consentendo aggiornamenti e modifiche localizzate senza impattare l'intero sistema.

L'architettura è pensata per gestire in modo ottimale il flusso dei dati, che scorre tra i vari attori attraverso un'infrastruttura **centralizzata** ma altamente modulare, dove ogni componente (ristoranti, clienti, rider, amministratori) ha accesso a funzionalità dedicate. Questa architettura consente di centralizzare la logica di business in un'unica base di codice, pur mantenendo la flessibilità necessaria per l'aggiunta di nuove funzionalità o il miglioramento delle performance.

La **gestione delle comunicazioni tra i vari moduli** avviene tramite **API RESTful**, che rappresentano il cuore della piattaforma, garantendo un'integrazione fluida tra il backend e il frontend. Le comunicazioni via API sono leggere, facilmente estendibili e consentono di mantenere una struttura chiara e ben separata tra client e server. Questo approccio facilita la scalabilità orizzontale del sistema e l'adozione di tecnologie future senza dover ripensare l'intera architettura.

Un altro principio fondamentale nell'architettura riguarda la **sicurezza e l'affidabilità** del sistema. Per proteggere i dati degli utenti, sono stati adottati meccanismi di autenticazione e autorizzazione robusti, insieme alla cifratura dei dati sensibili. Inoltre, l'architettura è progettata per gestire eventuali malfunzionamenti o picchi di traffico, garantendo che la piattaforma sia sempre operativa, anche in situazioni di alta domanda. Nel seguito del capitolo, approfondiremo le singole **componenti** architetturali del sistema, come la gestione degli ordini, le modalità di interazione con i vari attori, e le tecnologie di supporto. Verranno anche esplorate le scelte progettuali relative alla **manutenibilità** del sistema, alla **scalabilità** e alla **sicurezza**, con l'obiettivo di garantire che la piattaforma possa crescere e adattarsi alle necessità future, rispondendo adeguatamente ai cambiamenti nel carico di lavoro e alle nuove richieste dei clienti.

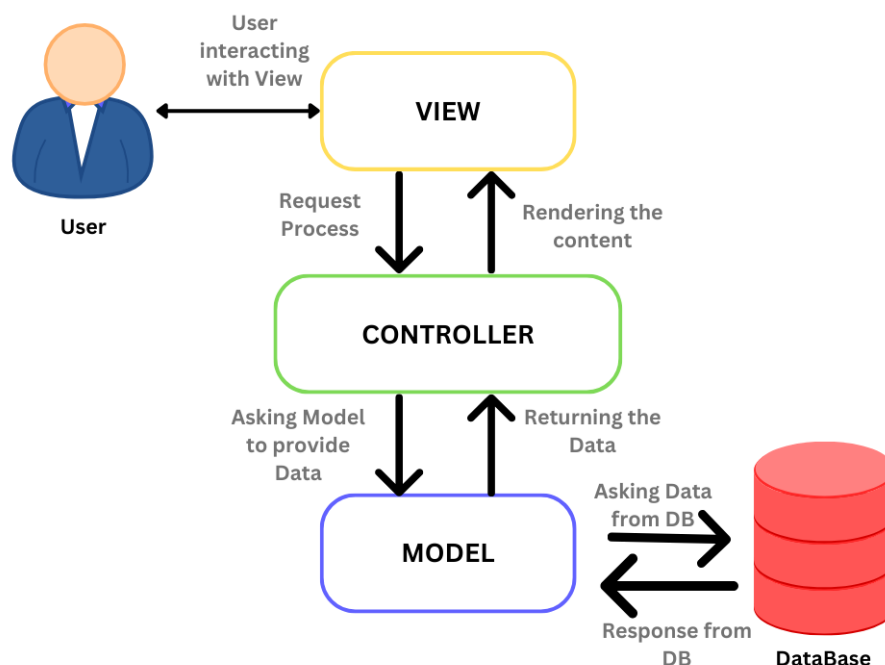


Figura 4: Pattern MVC

3.2 Componenti Principali del Sistema

L'architettura di un sistema è il fondamento che sostiene tutte le operazioni, le interazioni e i flussi di dati che permettono al sistema stesso di funzionare correttamente. In un'applicazione complessa come una piattaforma di food delivery, l'architettura deve essere progettata in modo tale da supportare una gestione efficiente delle risorse, garantire un'esperienza utente fluida e sicura, e permettere al sistema di scalare in base alla crescita del numero di utenti, ordini e ristoranti.

Nel caso della nostra piattaforma, l'architettura si compone di tre componenti principali che collaborano per garantire il buon funzionamento dell'intero sistema: **Frontend**, **Backend** e **Database**. Ciascuna di queste componenti ha un ruolo cruciale nel gestire un determinato aspetto del sistema, ma tutte interagiscono strettamente tra loro per soddisfare le esigenze di utenti, ristoratori, rider e amministratori.

Il **Frontend** è la parte dell'architettura che l'utente finale vede e con cui interagisce direttamente. È il canale attraverso il quale il cliente può navigare tra i ristoranti, esplorare i menu, fare ordini e tracciare lo stato delle consegne. Il ristorante utilizza il frontend per gestire il proprio menu, accettare ordini e monitorare lo stato delle preparazioni. I rider utilizzano il frontend per visualizzare le richieste di consegna e ottimizzare i loro percorsi. In pratica, il frontend è il punto di contatto tra l'utente e il sistema, ed è quindi essenziale che fornisca un'interfaccia intuitiva, reattiva e user-friendly.

Il **Backend**, d'altra parte, è il cuore del sistema, dove risiede la logica di business e dove vengono elaborate tutte le richieste provenienti dal frontend. Gestisce l'interazione con il database, elabora i dati e coordina le operazioni richieste dagli utenti, come la creazione di ordini, la gestione delle consegne e la visualizzazione delle informazioni. In un'applicazione come la nostra, il backend deve essere progettato per essere scalabile e performante, in modo da poter gestire un numero crescente di utenti e ordini simultanei, senza compromettere la velocità o la qualità del servizio.

Il **Database** funge da deposito centrale per tutti i dati del sistema. Esso conserva informazioni sugli utenti, i ristoranti, i menu, gli ordini e le transazioni. Un buon database deve garantire la sicurezza, l'integrità e la disponibilità dei dati in tempo reale, oltre a consentire operazioni rapide ed efficienti anche quando il volume di dati cresce. Il database è un elemento cruciale per il funzionamento del sistema, poiché memorizza e recupera continuamente i dati che alimentano il backend e il frontend.

In generale, la suddivisione dell'architettura in queste tre componenti principali consente di separare le preoccupazioni, facilitando la gestione e la manutenzione del sistema nel tempo. Inoltre, questa separazione consente di migliorare la scalabilità, poiché è possibile ottimizzare ciascuna componente in modo indipendente, in base ai requisiti e alle esigenze specifiche del sistema.

In sintesi, l'architettura della piattaforma di food delivery è stata progettata per essere modulare, scalabile, sicura e performante, con una chiara separazione tra la logica di presentazione (Frontend), la logica di elaborazione (Backend) e la gestione dei dati (Database), garantendo così una gestione fluida e coerente delle operazioni quotidiane del sistema.

3.2.1 Model

Il **Model** è una delle componenti principali del pattern **Model-View-Controller (MVC)**, ed è fondamentale per la gestione della logica di business e della struttura dei dati in un'applicazione. In un'applicazione di food delivery come quella descritta, il modello si occupa della gestione e della rappresentazione dei dati relativi agli utenti, ai ristoranti, ai menu, agli ordini e alle consegne.

3.2.1.1 Ruolo del Model

Il **Model** è la parte dell'architettura che gestisce tutte le operazioni di manipolazione dei dati. Si occupa di definire la struttura e le relazioni tra i vari oggetti, come i dati dei clienti, i menu dei ristoranti, gli ordini effettuati dai clienti e le informazioni sui rider e sulle consegne. È responsabile anche di implementare la logica applicativa, come ad esempio il calcolo dei costi di un ordine, la gestione dello stato di un ordine (ad esempio: "in preparazione", "in consegna", "completato"), e l'integrazione con altre componenti del sistema, come il database o i servizi esterni.

In pratica, il **Model** è il connettore tra il **Backend** e il **Database**, ed è responsabile per il recupero, la modifica e la persistenza dei dati. All'interno del Model, vengono implementate tutte le classi che rappresentano gli oggetti del dominio dell'applicazione.

3.2.1.2 Struttura del Model

Nel contesto di un'applicazione di food delivery, il Model è generalmente suddiviso in diverse entità, ciascuna con una propria logica e un proprio comportamento. Ecco alcuni degli oggetti principali che compongono il Model per la piattaforma descritta:

1. **Utente (User)**: L'entità **User** rappresenta i dati relativi agli utenti, che possono essere clienti, ristoratori o rider. Ogni tipo di utente avrà attributi differenti (ad esempio, un cliente avrà informazioni su indirizzo e ordini passati, un ristoratore avrà un menu e una lista di ordini da preparare, mentre un rider avrà informazioni sulla posizione e sulla disponibilità). La classe **User** può contenere metodi per la gestione delle credenziali, la modifica del profilo e la visualizzazione delle informazioni correlate.
2. **Ristorante (Restaurant)**: L'entità **Restaurant** rappresenta i ristoranti registrati sulla piattaforma. Contiene informazioni sul nome, indirizzo, menu, orari di apertura e chiusura e altre configurazioni specifiche per ogni ristorante. Il ristorante ha anche una lista di ordini ricevuti e una serie di operazioni per gestirli (ad esempio, accettare, rifiutare o preparare gli ordini).
3. **Menu (Menu)**: L'entità **Menu** è associata a un ristorante e contiene l'elenco degli articoli disponibili. Ogni articolo può avere attributi come il nome, la descrizione, il prezzo, la disponibilità e gli ingredienti. La classe **Menu** consente di aggiungere, modificare e rimuovere gli articoli dal menu.
4. **Ordine (Order)**: L'entità **Order** è un elemento centrale del sistema, poiché rappresenta un ordine effettuato da un cliente. Ogni ordine ha una serie di attributi, come gli articoli ordinati, il cliente che ha effettuato l'ordine, lo stato dell'ordine (ad esempio, "in preparazione", "pronto per la consegna"), l'indirizzo di consegna e le informazioni relative al pagamento. Il modello di ordine potrebbe includere

metodi per la gestione del flusso dell'ordine, come l'aggiornamento dello stato, il calcolo dei costi e l'associazione dell'ordine al rider che lo consegnerà.

5. **Consegna (Delivery)**: L'entità **Delivery** si riferisce alla consegna effettiva dell'ordine da parte del rider. Essa contiene informazioni sulla posizione del rider, lo stato della consegna (ad esempio, "in corso", "completato"), l'orario stimato di arrivo e la connessione all'ordine che deve essere consegnato. Inoltre, può contenere metodi per ottimizzare il percorso del rider, calcolare i tempi di arrivo e comunicare con il cliente.

3.2.1.3 Relazioni tra le Entità

Una delle caratteristiche fondamentali del **Model** è la gestione delle relazioni tra le entità. Ad esempio:

- Un **cliente** può effettuare più **ordini**.
- Un **ristorante** può avere più **menu** e può ricevere più **ordini**.
- Un **rider** può essere associato a più **consegne**.
- Un **ordine** è associato a un **ristorante**, un **cliente** e, alla fine, a un **rider** che lo consegnerà.

Queste relazioni devono essere gestite in modo efficiente all'interno del modello per consentire una navigazione fluida dei dati e una corretta esecuzione delle operazioni. Il **Model**, infatti, non solo definisce i dati, ma stabilisce anche la logica di come questi dati possono essere modificati o elaborati.

3.2.1.4 Integrazione con il Database

Il **Model** funge da interfaccia tra la logica applicativa e il database. Ogni entità del modello (come **User**, **Restaurant**, **Order**, ecc.) è tipicamente associata a una tabella nel database, e il **Model** si occupa di persistere e recuperare i dati da queste tabelle. In un'applicazione basata su un database relazionale, ad esempio, le operazioni CRUD (Create, Read, Update, Delete) sono implementate all'interno delle classi del **Model**. Le operazioni possono essere eseguite tramite un ORM (Object-Relational Mapping) che mappa gli oggetti Python alle tabelle del database. In questo caso, strumenti come SQLAlchemy, che è spesso utilizzato con Flask, consentono al **Model** di interagire con il database in modo trasparente, mantenendo il codice semplice e leggibile.

3.2.1.5 Comportamento e Logica di Business

Il **Model** non si limita solo a gestire i dati, ma implementa anche la logica di business che guida il comportamento dell'applicazione. Ad esempio, il calcolo dei costi di un ordine, l'assegnazione di un rider a un ordine, la gestione dello stato di un ordine o l'invio di notifiche sono tutte operazioni che vengono gestite dal **Model**.

In questo modo, il **Model** rappresenta una parte fondamentale per il corretto funzionamento dell'applicazione, poiché gestisce i dati e assicura che tutte le operazioni vengano eseguite correttamente in base alle regole aziendali.

3.2.2 View

La **View** è una delle componenti principali del pattern architetturale **Model-View-Controller (MVC)** ed è responsabile della presentazione dell'interfaccia utente (UI) e dell'interazione visiva con l'utente. Il ruolo della **View** è quello di mostrare i dati contenuti nel **Model** all'utente in una forma comprensibile e interattiva, senza però occuparsi della gestione dei dati o della logica di business. In altre parole, la **View** si concentra sul come visualizzare i dati e come consentire all'utente di interagire con il sistema.

3.2.2.1 Ruolo e Funzione della View

In un'applicazione di food delivery come quella descritta, la **View** è la parte che consente agli utenti di interagire con il sistema. Ad esempio, quando un cliente accede alla piattaforma, la **View** mostrerà l'elenco dei ristoranti, i menu, gli ordini effettuati e le opzioni per tracciare le consegne. Allo stesso modo, il ristoratore avrà una vista per gestire il proprio menu e gli ordini ricevuti, mentre il rider vedrà una dashboard per gestire le consegne e aggiornare lo stato della sua disponibilità.

La **View** in un'applicazione web è generalmente costituita da una serie di file di template (HTML) che vengono renderizzati dal framework di backend (in questo caso Flask) e vengono inviati al browser dell'utente per la visualizzazione. Questi template possono includere anche codice per la gestione degli stili (CSS) e del comportamento dinamico (JavaScript).

3.2.2.2 Struttura della View

- **Template HTML:** Il cuore della **View** è rappresentato dai file HTML, che definiscono la struttura di ogni pagina del sito web o dell'applicazione. In questi file, vengono inseriti i dati dinamici provenienti dal **Model**. Ad esempio, nella vista della pagina di un ristorante, verranno mostrati i dettagli del ristorante e del menu, che provengono direttamente dal **Model**.
- **CSS (Cascading Style Sheets):** Per migliorare l'aspetto visivo dell'applicazione, la **View** può includere anche file CSS che definiscono il layout, i colori, i font e altre proprietà stilistiche. In un'applicazione di food delivery, la UI dovrebbe essere accattivante e facilmente navigabile, quindi l'uso del CSS per la formattazione della pagina è cruciale.
- **JavaScript:** La **View** può includere anche script JavaScript, che consentono di aggiungere interattività dinamica alle pagine. Ad esempio, JavaScript può essere utilizzato per inviare richieste asincrone al server (tramite AJAX), aggiornare dinamicamente le informazioni della pagina senza ricaricarla o gestire il comportamento degli elementi della UI, come pulsanti e form.

3.2.2.3 Funzioni della View nell'applicazione di Food Delivery

Nel contesto di un'applicazione di food delivery, la **View** svolge diversi ruoli cruciali per i vari tipi di utenti:

- **Vista per i Clienti:**

- **Sfogliare i ristoranti e i menu:** Il cliente vedrà una lista dei ristoranti disponibili, con informazioni come nome, tipo di cucina, valutazione, e i dettagli del menu (articoli, descrizioni, prezzi).
- **Effettuare un ordine:** La vista permetterà al cliente di selezionare gli articoli dal menu e procedere con l'acquisto, visualizzando il carrello e il totale.
- **Monitorare gli ordini:** Una volta effettuato l'ordine, la vista mostrerà lo stato in tempo reale (ad esempio "In preparazione", "In consegna") e permetterà al cliente di tracciare la consegna.
- **Gestire il profilo:** La vista consente anche di modificare le informazioni personali del cliente, come l'indirizzo di consegna, e di visualizzare la cronologia degli ordini.

- **Vista per i Ristoratori:**

- **Gestire il menu:** I ristoratori potranno visualizzare e modificare il proprio menu, aggiungendo, rimuovendo o modificando gli articoli.
- **Gestire gli ordini:** La vista per i ristoratori permetterà di vedere gli ordini ricevuti, segnare un ordine come "in preparazione" o "completato", e interagire con lo stato dell'ordine.
- **Gestire il profilo:** Ogni ristorante avrà una vista per aggiornare le informazioni del proprio profilo, come l'indirizzo, gli orari di apertura, le informazioni di contatto e altro.

- **Vista per i Rider:**

- **Gestire le consegne:** I rider avranno una vista per visualizzare gli ordini da consegnare, ricevere informazioni sul percorso ottimale e aggiornare lo stato della consegna.
- **Aggiornare la disponibilità:** I rider potranno aggiornare il proprio stato di disponibilità, indicando quando sono pronti per accettare nuovi ordini di consegna.
- **Monitorare lo stato delle consegne:** La vista mostrerà informazioni in tempo reale sulle consegne, permettendo ai rider di tenere traccia delle loro attività quotidiane.

- **Vista per gli Amministratori:**

- **Monitorare le performance:** Gli amministratori avranno una vista per monitorare le performance del sistema, ad esempio visualizzando statistiche sugli ordini, la flotta di rider e i ristoranti più popolari.
- **Gestire gli account:** Gli amministratori potranno visualizzare, aggiungere, modificare o eliminare account di utenti, ristoratori e rider.

- **Gestire le impostazioni del sistema:** Gli amministratori avranno accesso a configurazioni più avanzate del sistema, come la gestione delle categorie di ristoranti o le politiche di pagamento.

3.2.3 Controller

Il **Controller** è l'elemento chiave nel pattern architetturale **Model-View-Controller** (MVC) e rappresenta il ponte tra il **Model** (che gestisce i dati e la logica di business) e la **View** (che si occupa della presentazione). La funzione principale del **Controller** è quella di gestire le richieste dell'utente, elaborarle e determinare quale **Model** utilizzare per ottenere i dati necessari. Successivamente, il **Controller** invia i dati alla **View** affinché vengano presentati all'utente.

3.2.3.1 Ruolo e Funzione del Controller

Il **Controller** ha il compito di fungere da intermediario tra l'interfaccia utente (UI) e la logica di business, gestendo le interazioni tra il **Model** e la **View**. Quando l'utente interagisce con l'applicazione (ad esempio, cliccando su un pulsante, inviando un modulo o effettuando una selezione), la richiesta dell'utente viene indirizzata al **Controller**, che esegue le operazioni necessarie per soddisfare la richiesta.

In un'applicazione di food delivery, i **Controller** gestiranno le interazioni tra i vari tipi di utenti (clienti, ristoratori, rider e amministratori) e il sistema. Ad esempio, quando un cliente effettua un ordine, il **Controller** si occupa di raccogliere le informazioni dal **Model** (come i dettagli degli articoli ordinati, il prezzo totale e la disponibilità) e di inviarle alla **View** per la visualizzazione.

3.2.3.2 Struttura del Controller

1. Routing

- Il **Controller** gestisce le rotte, cioè le URL richieste dall'utente, indirizzando la richiesta al metodo appropriato.

2. Metodi/Action

- Ogni metodo del **Controller** corrisponde a una specifica **Action** che gestisce un'operazione (come visualizzare una pagina o eseguire un'azione, ad esempio creare un ordine).

3. Input Validation

- Il **Controller** valida i dati inviati dall'utente, assicurandosi che siano nel formato corretto e completi. In caso di errore, reindirizza l'utente con un messaggio di errore.

4. Chiamata ai Model

- Il **Controller** interagisce con il **Model** per recuperare o manipolare i dati, come ottenere informazioni da un database o aggiornare un record.

5. Logica di Business

- In alcuni casi, il **Controller** gestisce la logica di business, come calcolare prezzi o applicare sconti.

6. Redirezione e Risposte

- Dopo aver completato un'operazione, il **Controller** determina se l'utente deve essere rediretto a una pagina di conferma o mostrato un errore.

7. Gestione degli errori

- Il **Controller** gestisce gli errori che si verificano durante l'esecuzione dell'azione, come errori di dati o di interazione con il **Model**.

8. Autenticazione e Autorizzazione

- Il **Controller** verifica se l'utente è autenticato e ha i permessi necessari per accedere a una risorsa o eseguire un'azione. Se necessario, può reindirizzare l'utente alla pagina di login.

9. Servizi e Helper

- Il **Controller** può utilizzare servizi o helper per delegare attività specifiche (come inviare notifiche via email o eseguire calcoli) senza includerle nella logica principale del **Controller**.

3.2.3.3 Funzioni del Controller nell'applicazione di Food Delivery

In un'applicazione di food delivery, il **Controller** svolge numerosi compiti, che variano in base al tipo di utente e all'azione intrapresa. Ecco alcune delle funzioni principali:

1. Per i Clienti:

- **Visualizzazione dei ristoranti:** Quando un cliente visita la piattaforma, il **Controller** interroga il **Model** per ottenere un elenco di ristoranti disponibili, inclusi i menu e le informazioni di ciascun ristorante.
- **Effettuare un ordine:** Quando un cliente seleziona articoli dal menu e procede con un ordine, il **Controller** raccoglie le informazioni sugli articoli, calcola il totale e invia i dati al **Model** per la registrazione dell'ordine.
- **Monitoraggio degli ordini:** Se un cliente desidera tracciare lo stato di un ordine, il **Controller** invia una richiesta al **Model** per ottenere informazioni sullo stato attuale dell'ordine (ad esempio, se è in preparazione, in transito o completato) e le invia alla **View**.

2. Per i Ristoratori:

- **Gestire il menu:** Quando un ristoratore desidera aggiungere o modificare un elemento del menu, il **Controller** riceve la richiesta dalla **View**, aggiorna il **Model** e infine aggiorna la **View** con il menu aggiornato.

- **Gestire gli ordini:** I ristoratori possono visualizzare e aggiornare lo stato degli ordini ricevuti. Il **Controller** recupera gli ordini dal **Model** e li invia alla **View** per la visualizzazione. Inoltre, quando lo stato dell'ordine cambia (ad esempio, da "In preparazione" a "Completato"), il **Controller** aggiorna il **Model** e notifica la **View**.

3. Per i Rider:

- **Gestire le consegne:** Quando un rider accetta un ordine, il **Controller** aggiorna lo stato nel **Model** per riflettere che il rider sta effettuando una consegna. Il **Controller** invia anche informazioni al **Model** per ottenere il percorso ottimale per la consegna e fornisce questi dati alla **View** per essere visualizzati dal rider.
- **Aggiornare la disponibilità:** Quando un rider cambia il proprio stato di disponibilità, il **Controller** aggiorna il **Model** con la nuova disponibilità e la **View** riflette il cambiamento.

4. Per gli Amministratori:

- **Gestire gli account:** Gli amministratori hanno il compito di gestire gli utenti (clienti, ristoratori, rider). Quando un amministratore crea, modifica o elimina un account, il **Controller** invia la richiesta al **Model** per aggiornare il database e successivamente aggiorna la **View** con le informazioni più recenti.
- **Monitorare le performance:** Gli amministratori possono visualizzare le statistiche e i report sul sistema, come il numero di ordini completati o la performance della flotta di rider. Il **Controller** raccoglie queste informazioni dal **Model** e le invia alla **View** per essere visualizzate sotto forma di grafici o tabelle.

3.2.4 Database

Il **database** è una parte fondamentale di qualsiasi applicazione che necessita di memorizzare, recuperare e gestire dati. Nel contesto di un'applicazione basata su architettura **Model-View-Controller (MVC)**, il database è essenziale per il **Model**, in quanto rappresenta la fonte dei dati che il modello gestisce e con cui interagisce.

3.2.4.1 Ruolo del Database

Nel pattern MVC, il **Model** è responsabile della gestione dei dati e della logica di business. Pertanto, il database si occupa di memorizzare i dati persistenti che l'applicazione utilizza, come le informazioni sugli utenti, ordini, ristoranti, piatti, ecc. Il **Model** interagisce direttamente con il database per recuperare, inserire, aggiornare e cancellare i dati, a seconda delle necessità dell'applicazione.

3.2.4.2 Struttura del Database

La struttura del database dipende dal tipo di dati che l'applicazione gestisce, ma generalmente include le seguenti componenti:

1. Tabelle:

- Ogni tabella nel database rappresenta un'entità del mondo reale, come "Utenti", "Ristoranti", "Ordini", "Menu", ecc.
- Ogni tabella è composta da **colonne**, che rappresentano gli attributi di queste entità (ad esempio, un'entità "Utenti" potrebbe avere colonne come nome, email, indirizzo, ecc.).

2. Relazioni:

- Le tabelle sono collegate tra loro tramite **relazioni**. Le relazioni possono essere di tipo:
 - **Uno-a-uno**: Ogni record di una tabella è correlato a un singolo record di un'altra tabella.
 - **Uno-a-molti**: Un record in una tabella può essere correlato a molti record in un'altra tabella (ad esempio, un ristorante può avere molti ordini).
 - **Molti-a-molti**: Due tabelle possono essere correlate da molteplici record reciproci, gestito generalmente da una **tabella di giunzione**.

3. Primary Key (PK):

- Ogni tabella ha una **chiave primaria** che identifica univocamente ogni record nella tabella. Ad esempio, un ID univoco per ogni ordine o utente.

4. Foreign Key (FK):

- Le **chiavi esterne** sono utilizzate per collegare record tra tabelle, stabilendo relazioni. Ad esempio, un record di un ordine potrebbe avere una chiave esterna che fa riferimento a un ristorante o un utente.

3.2.4.3 Tipi di Database

Esistono due principali tipi di database utilizzati nelle architetture moderne:

1. Database Relazionali (RDBMS):

- I **database relazionali** (come MySQL, PostgreSQL, SQLite, Oracle) sono basati su tabelle che si collegano tra loro tramite relazioni.
- Utilizzano il linguaggio **SQL** (Structured Query Language) per interagire con i dati.
- Sono ideali per applicazioni che necessitano di dati strutturati e relazioni complesse tra i dati.
- **Vantaggi**: Coerenza dei dati, supporto per transazioni, query complesse e integrità referenziale.

2. Database NoSQL:

- I **database NoSQL** (come MongoDB, Cassandra, Firebase) non sono basati su tabelle e sono più flessibili nella gestione dei dati.
- Possono essere documentali, a colonne, a grafo o a chiave-valore.
- Sono ideali per applicazioni che devono gestire grandi volumi di dati non strutturati o semi-strutturati, come contenuti web o social media.
- **Vantaggi:** Scalabilità orizzontale, flessibilità nei dati non strutturati, alta disponibilità.

3.2.4.4 Gestione delle Transazioni

Le **transazioni** sono essenziali per mantenere la coerenza dei dati nel database. Se un'operazione che coinvolge più passaggi fallisce (ad esempio, l'aggiornamento dell'ordine e il pagamento), la transazione può essere annullata, evitando che il database rimanga in uno stato incoerente.

I database relazionali utilizzano il concetto di **ACID** (Atomicità, Coerenza, Isolamento, Durabilità) per garantire che le transazioni siano gestite correttamente.

3.3 Interazione tra i Componenti

Nel pattern architetturale **Model-View-Controller (MVC)**, l'interazione tra i componenti gioca un ruolo cruciale nel separare le responsabilità e mantenere una struttura organizzata nell'applicazione. Questa separazione permette di sviluppare e mantenere l'applicazione in modo modulare, semplificando l'evoluzione del software e l'adattamento alle nuove esigenze. Ogni componente ha un ruolo ben definito e interagisce con gli altri componenti solo attraverso interfacce predefinite, evitando che un componente sia troppo dipendente dagli altri.

La **View**, che rappresenta l'interfaccia utente, è il componente responsabile della presentazione e dell'interazione diretta con l'utente. Quando l'utente interagisce con l'interfaccia (ad esempio, inserendo dei dati in un modulo o cliccando su un pulsante), la View raccoglie queste informazioni e le invia al **Controller**. Tuttavia, la View non è coinvolta nella logica dell'applicazione; si limita a mostrare i dati e a rispondere agli input dell'utente. La sua funzione principale è quella di presentare in modo chiaro e comprensibile i dati che vengono inviati dal Controller.

Il **Controller**, d'altra parte, è il "regista" del flusso dell'applicazione. Si occupa di gestire gli eventi, come gli input dell'utente provenienti dalla View, e di determinare quale logica applicativa eseguire in base a queste informazioni. Quando il Controller riceve un comando o una richiesta, decide quale azione intraprendere. Se la richiesta implica operazioni sui dati, come il recupero o la modifica di informazioni, il Controller interagisce con il **Model**. In altre parole, il Controller è il mediatore tra la View e il Model, ricevendo input dalla View, eseguendo operazioni sulla base della logica di business del Model e poi aggiornando la View con i risultati.

Il **Model** è il cuore dell'applicazione, in quanto gestisce i dati e la logica di business. Può trattare informazioni complesse, eseguire calcoli, e interagire con il database per leggere o scrivere dati. Quando il Controller necessita di accedere o aggiornare i dati, invoca il Model per eseguire le operazioni appropriate. Una volta che il Model ha completato la sua funzione, restituisce i dati al Controller, che poi li invia alla View per aggiornare l'interfaccia utente.

La separazione delle responsabilità tra questi tre componenti rende il sistema altamente modulare. In questo modo, ogni componente può essere sviluppato e modificato in modo indipendente. Per esempio, i cambiamenti nel design dell'interfaccia utente non influiscono sulla logica di business o sulla gestione dei dati. Allo stesso modo, modifiche alla logica di business possono essere implementate senza alterare la parte che si occupa dell'interfaccia. La separazione consente anche di testare e debugare ciascun componente in modo isolato, migliorando l'affidabilità e la manutenibilità del sistema.

In sintesi, l'interazione tra il **Model**, la **View** e il **Controller** segue un flusso ben definito che permette a ciascun componente di concentrarsi su un aspetto specifico dell'applicazione. La **View** si occupa della presentazione, il **Controller** gestisce la logica applicativa e il flusso, mentre il **Model** gestisce i dati e la logica di business. Questo approccio modulare garantisce una gestione più semplice del codice e una maggiore flessibilità nell'adattare l'applicazione alle necessità future.

3.3.1 Interazione tra Utente e View

L'interazione tra l'utente e la View è una delle componenti chiave nell'architettura Model-View-Controller (MVC). La View funge da interfaccia utente, cioè è ciò che l'utente vede e con cui interagisce direttamente.

1. Presentazione dei Dati:

La View è responsabile per la visualizzazione dei dati provenienti dal Model. Quando il Controller riceve i dati dal Model (come ad esempio informazioni sugli ordini di un ristorante, la disponibilità dei prodotti, o le preferenze dell'utente), la View si occupa di presentare questi dati in un formato visivamente comprensibile e interattivo. Ad esempio, in un'applicazione di food delivery, la View potrebbe mostrare la lista dei ristoranti disponibili, le descrizioni dei piatti e i prezzi, oppure visualizzare i dettagli di un ordine in corso. In generale, la View traduce i dati in elementi visivi come tabelle, elenchi, immagini e pulsanti.

2. Ricezione degli Input dell'Utente:

Quando l'utente interagisce con l'interfaccia (cliccando su un pulsante, compilando un modulo, facendo una selezione da un menu a discesa, ecc.), la View è il punto di ingresso per questi input. L'utente non interagisce direttamente con il Controller o il Model; la sua interazione avviene esclusivamente attraverso la View. Esempi di interazione includono:

- L'utente che seleziona un ristorante dalla lista.
- L'utente che inserisce un indirizzo di spedizione in un modulo.
- L'utente che clicca su "Paga" per concludere un ordine.

3. Trasferimento degli Input al Controller:

Una volta che la View riceve l'input dell'utente, la comunica al Controller. Questo passaggio avviene tramite degli event handler o dei listener che sono collegati agli elementi interattivi della View (ad esempio, bottoni, campi di testo, etc.). In sostanza, la View funge da "canale" che invia l'input dell'utente al Controller per l'elaborazione. Ad esempio, se un utente clicca su un pulsante per inviare un ordine, la View invia questa richiesta al Controller, che decide come gestirla (ad esempio, validando i dati, creando un ordine, e interagendo con il Model).

4. Feedback Visivo all'Utente:

La View è anche responsabile per dare feedback visivo all'utente in base alle azioni che sono state intraprese. Dopo che il Controller ha elaborato l'input dell'utente e ha interagito con il Model, la View aggiorna l'interfaccia per riflettere lo stato corrente del sistema. Ad esempio, se un ordine è stato completato correttamente, la View potrebbe mostrare un messaggio di conferma. Se un errore si è verificato (ad esempio, il pagamento non è andato a buon fine), la View mostrerà un messaggio di errore. Questi feedback sono fondamentali per l'esperienza utente, in quanto aiutano l'utente a comprendere lo stato del sistema e a prendere decisioni successive.

5. Comportamento Dinamico:

Le moderne View sono spesso interattive e dinamiche, cioè possono aggiornarsi senza necessitare di ricaricare completamente la pagina (questo si chiama rendering

dinamico). Quando un utente interagisce con un elemento della View, come un pulsante o una casella di ricerca, la View può aggiornare alcune parti dell'interfaccia in tempo reale (per esempio, mostrando i risultati di una ricerca senza ricaricare la pagina). In un'applicazione di food delivery, quando l'utente filtra i ristoranti per tipo di cucina, la View potrebbe mostrare immediatamente i ristoranti che corrispondono al filtro, senza dover ricaricare tutta la pagina.

6. Semplificazione dell'Interazione:

La View si occupa anche di semplificare e ottimizzare l'interazione per l'utente. Ad esempio, offre un design chiaro e intuitivo, con un layout organizzato che guida l'utente attraverso il flusso dell'applicazione. La View spesso include anche delle funzionalità di feedback visivo, come animazioni, transizioni o highlights, per segnalare all'utente quando qualcosa è andato a buon fine o quando c'è un errore.

7. Rendendo l'Interazione Accessibile:

Oltre a gestire gli input e le risposte, la View si occupa anche dell'accessibilità dell'interfaccia utente. Ciò include la gestione di elementi visivi come la leggibilità del testo, il contrasto cromatico, la navigabilità tramite tastiera e l'uso di tecnologie assistive come gli screen reader per utenti con disabilità visive.

In sintesi, l'interazione tra l'utente e la View è il cuore dell'esperienza dell'utente, poiché la View rappresenta la parte con cui l'utente ha il contatto diretto. La View non solo visualizza i dati e riceve gli input, ma è anche responsabile per aggiornare dinamicamente l'interfaccia e fornire feedback chiari e coerenti in base alle azioni dell'utente. La separazione della View dal Model e dal Controller consente di mantenere l'applicazione modulare, facilmente manutenibile e scalabile.

3.3.2 Interazione tra View e Controller

L'interazione tra View e Controller è un aspetto cruciale nel pattern Model-View-Controller (MVC) e definisce come l'interfaccia utente (View) comunica con la logica applicativa (Controller).

1. Flusso di interazione:

L'interazione tra View e Controller avviene in più fasi, che seguono un flusso di eventi ben definito:

- **Fase 1: L'utente interagisce con la View**

Quando l'utente interagisce con la View (ad esempio, clicca su un pulsante o seleziona una voce in un menu), la View cattura questo evento e lo invia al Controller. Questo può essere fatto tramite eventi come click, submit di form, o altre azioni utente.

- **Fase 2: Il Controller riceve l'input dalla View**

Il Controller riceve l'input (o evento) dalla View. In base a ciò che l'utente ha fatto (ad esempio, ha cliccato su un pulsante per ordinare un cibo), il Controller elabora l'azione e decide come gestirla. In questo momento, il Controller può decidere di aggiornare il Model (ad esempio, aggiungere un ordine) o recuperare dati dal Model per visualizzarli nella View.

- **Fase 3: Il Controller aggiorna la View**

Una volta che il Controller ha completato la sua logica (ad esempio, ha ottenuto i dati necessari o ha modificato lo stato del Model), passa i dati alla View per aggiornarla. Questo passaggio può includere la visualizzazione di un nuovo stato dell'interfaccia utente, l'aggiornamento di una lista, o il rendering di nuovi dati, come una conferma dell'ordine effettuato o una lista aggiornata dei ristoranti.

- **Fase 4: La View visualizza i risultati**

La View aggiorna l'interfaccia utente con le informazioni ricevute dal Controller. A questo punto, l'utente può vedere i risultati della sua interazione (ad esempio, il carrello della spesa aggiornato, la lista dei piatti disponibili, o la conferma dell'ordine).

2. Tipi di comunicazione:

La comunicazione tra View e Controller può avvenire in due direzioni principali:

- **Comunicazione dalla View al Controller:** La View invia un'azione al Controller quando l'utente interagisce con l'interfaccia. Ad esempio, cliccando su un pulsante, inserendo un dato nel modulo, o selezionando un'opzione.
- **Comunicazione dal Controller alla View:** Una volta che il Controller ha completato la logica e i calcoli necessari, fornisce alla View i dati da visualizzare. Questi dati possono essere un risultato di un'operazione del Model, come l'elenco di piatti disponibili, l'ordine in fase di preparazione, o una notifica di successo dell'operazione.

3. Tecniche di gestione dell'interazione:

- **Event-driven:** In un'applicazione moderna, l'interazione tra View e Controller è spesso basata su eventi. Ad esempio, in un'applicazione web con JavaScript, l'interazione tra View e Controller avviene tramite eventi come click, submit, keydown, che innescano funzioni del Controller.
- **Binding dei dati:** In alcune implementazioni MVC, il binding dei dati permette di legare i dati tra il Controller e la View in modo che ogni cambiamento nei dati si rifletta automaticamente nella vista. Questo è tipico in framework come Angular o React, dove la vista si aggiorna automaticamente quando i dati cambiano nel Controller.

4. Separazione delle preoccupazioni:

L'interazione tra View e Controller aiuta a mantenere separato il livello dell'interfaccia utente dalla logica dell'applicazione. In questo modo, il Controller è responsabile per la logica di business, mentre la View è solo un mezzo per presentare i dati e raccogliere gli input. Questa separazione facilita la manutenzione del codice, permettendo di modificare la View senza toccare la logica del Controller, e viceversa.

In generale, l'interazione tra View e Controller è continua e dinamica. La View cattura gli input dell'utente e invia eventi al Controller, che gestisce la logica e aggiorna la View

con i dati necessari. Il Controller funge da coordinatore che gestisce le azioni dell'utente e decide come rispondere, mentre la View è sempre aggiornata in base ai dati passati dal Controller.

3.3.3 Interazione tra Controller e Model

L'interazione tra il Controller e il Model è fondamentale nel pattern Model-View-Controller (MVC), poiché il Controller gestisce la logica dell'applicazione e si occupa di orchestrare le operazioni tra l'interfaccia utente (View) e i dati (Model).

1. Flusso di interazione tra Controller e Model:

L'interazione tra Controller e Model si svolge in modo sequenziale attraverso diverse fasi:

- **Fase 1: Ricezione della richiesta dall'utente**

Il flusso inizia quando l'utente interagisce con la View (ad esempio, cliccando su un pulsante per effettuare un ordine). La View invia un'azione al Controller in base all'input dell'utente (come la selezione di un piatto o l'invio di un modulo).

- **Fase 2: Elaborazione del Controller**

Una volta che il Controller riceve l'input dell'utente, inizia a elaborarlo. Se l'azione richiede operazioni sui dati, il Controller interagisce con il Model. Ad esempio, se un utente ordina un piatto, il Controller potrebbe dover aggiungere un nuovo ordine al Model. Se necessario, il Controller valida l'input dell'utente prima di inviarlo al Model.

- **Fase 3: Interazione con il Model**

Il Controller invoca metodi del Model per recuperare, modificare o salvare i dati. Ad esempio, il Controller potrebbe chiamare un metodo del Model per aggiungere un nuovo ordine, verificare la disponibilità di un prodotto, o recuperare la lista dei piatti da un database. A seconda del tipo di applicazione, il Model potrebbe interagire con una base di dati, un sistema di cache o un altro tipo di archiviazione. Alcuni esempi di operazioni che il Controller può richiedere al Model:

- **Recupero dei dati:** Il Controller può chiedere al Model di recuperare dati, come i menu dei ristoranti, gli ordini passati dell'utente o i dettagli di un singolo prodotto.
- **Creazione o aggiornamento dei dati:** Quando un utente effettua un ordine, il Controller può chiedere al Model di creare un nuovo ordine nel database.
- **Elaborazione dei dati:** Se il Controller ha bisogno di applicare regole aziendali (ad esempio, calcolare il totale di un ordine), può chiedere al Model di eseguire questi calcoli.

- **Fase 4: Risultati dal Model**

Dopo aver ricevuto la richiesta dal Controller, il Model elabora i dati, effettua le operazioni richieste e restituisce un risultato al Controller. Ad esempio, il

Model può restituire i dati di un ordine, una lista aggiornata di piatti, o una conferma di successo o errore.

- **Fase 5: Aggiornamento della View**

Dopo aver ottenuto i dati dal Model, il Controller passa i risultati alla View per la visualizzazione. Ad esempio, se il Controller ha creato con successo un ordine, invia il risultato alla View per aggiornare l'interfaccia utente e mostrare all'utente una conferma dell'ordine.

2. Tecniche di comunicazione tra Controller e Model:

Le tecniche comuni per l'interazione tra Controller e Model includono:

- **Chiamate dirette ai metodi del Model:** Il Controller può chiamare direttamente i metodi del Model per recuperare o aggiornare i dati. Questi metodi possono includere operazioni di database, calcoli o altre logiche aziendali.
- **Pattern di repository:** Un Controller può usare un pattern di repository per accedere ai dati. In questo caso, il Controller interagisce con il repository, che a sua volta comunica con il Model e il database.
- **Servizi esterni:** In applicazioni più complesse, il Controller potrebbe interagire con servizi esterni che a loro volta interagiscono con il Model. Ad esempio, un servizio di pagamento potrebbe gestire la logica di pagamento, interagendo con il Model per finalizzare l'ordine.

3. Separazione delle preoccupazioni:

Il Controller è progettato per essere separato dal Model al fine di mantenere una chiara distinzione tra la logica dell'applicazione e la gestione dei dati. Questo approccio aiuta a migliorare la manutenzione e la testabilità dell'applicazione. Inoltre, separare il Controller dal Model consente di evitare che la logica del controller dipenda troppo dai dettagli di implementazione del Model.

L'interazione tra Controller e Model è una parte fondamentale del pattern MVC. Il Controller funge da intermediario tra l'utente (tramite la View) e i dati (gestiti dal Model), eseguendo le operazioni necessarie sulla base dell'input dell'utente e restituendo i risultati. Questa separazione delle preoccupazioni tra Controller e Model consente di organizzare il codice in modo più modulare, facilitando la gestione e la manutenzione dell'applicazione.

3.3.4 Interazione tra Model e Database

L'interazione tra il Model e il Database è una componente cruciale nel pattern Model-View-Controller (MVC), poiché il Model è responsabile della gestione dei dati dell'applicazione. In questa sezione, esploreremo il processo di comunicazione tra il Model e il Database, descrivendo i meccanismi utilizzati, il flusso delle operazioni e le buone pratiche per garantire un'interazione efficace e sicura.

1. Flusso di Interazione:

- **Fase 1: Richiesta dal Model al Database**

Quando il Model riceve una richiesta dal Controller, questa potrebbe implicare una delle seguenti operazioni:

- Recuperare dati specifici (es. dettagli di un ordine).
- Inserire nuovi dati (es. salvare un ordine).
- Aggiornare dati esistenti (es. cambiare lo stato di un ordine).
- Eliminare dati (es. rimuovere un ordine annullato).

Il Model prepara una query o una richiesta al database per eseguire queste operazioni. A seconda dell'architettura, il Model potrebbe generare query SQL direttamente o utilizzare un livello di astrazione, come un Object-Relational Mapper (ORM).

- **Fase 2: Interazione con il Database**

Il Model invia la richiesta al database tramite un driver o un connettore appropriato. Durante questa fase:

- La query o la richiesta viene inviata al database.
- Il database esegue l'operazione richiesta.
- I risultati dell'operazione (ad esempio, dati recuperati, conferma di aggiornamento, o errori) vengono restituiti al Model.

- **Fase 3: Elaborazione dei Dati nel Model**

Dopo aver ricevuto i dati dal database, il Model può eseguire ulteriori elaborazioni, se necessario:

- **Conversione dei dati:** Ad esempio, trasformare i risultati della query in oggetti o strutture comprensibili per il resto dell'applicazione.
- **Validazione:** Verificare che i dati restituiti dal database siano coerenti e validi.
- **Applicazione di logica di business:** Se richiesto, il Model può arricchire o manipolare i dati per soddisfare le esigenze dell'applicazione.

- **Fase 4: Restituzione dei Dati**

Infine, il Model restituisce i dati elaborati al Controller, che li utilizza per aggiornare la View o intraprendere ulteriori azioni.

2. Tecniche di Comunicazione:

- **Utilizzo di ORM (Object-Relational Mapper)**

Un ORM è uno strumento che consente al Model di interagire con il database in modo più astratto, evitando di scrivere direttamente query SQL. Esempi di ORM popolari sono:

- SQLAlchemy (Python)
- Hibernate (Java)
- Django ORM (Python)

Con un ORM, il Model rappresenta i dati del database come oggetti. Ad esempio, un ordine nel database viene mappato a un'istanza di una classe `Order` nel Model. Questo approccio semplifica l'interazione con il database, rendendo il codice più leggibile e manutenibile.

- **Query SQL Dirette**

In alcuni casi, il Model può interagire con il database usando query SQL dirette. Questo approccio offre maggiore controllo e prestazioni ottimizzate

in scenari complessi, ma richiede una gestione attenta per evitare errori e vulnerabilità, come le SQL injection.

- **Utilizzo di Procedure Memorizzate**

In applicazioni avanzate, il Model può chiamare procedure memorizzate nel database. Queste sono blocchi di codice SQL salvati e eseguiti direttamente nel database, utili per operazioni complesse o ricorrenti.

3. Buone Pratiche

- **Separazione delle Responsabilità:** Il Model non deve contenere logica di business complessa che non sia legata ai dati. Questa separazione facilita la manutenzione.
- **Prevenzione delle Iniezioni SQL:** Quando si utilizzano query SQL dirette, è fondamentale utilizzare parametri preparati per prevenire attacchi di SQL injection.
- **Validazione dei Dati:** Prima di inviare i dati al database, il Model dovrebbe sempre validarli per garantire coerenza e correttezza.
- **Ottimizzazione delle Query:** Le query inviate dal Model devono essere ottimizzate per evitare problemi di prestazioni, specialmente in applicazioni con grandi volumi di dati.
- **Gestione degli Errori:** Il Model deve gestire correttamente gli errori restituiti dal database, come violazioni di vincoli o problemi di connessione.

L'interazione tra il Model e il Database è il fulcro della gestione dei dati in un'applicazione MVC. Il Model funge da intermediario tra il livello applicativo e il database, garantendo una separazione chiara delle responsabilità e un'astrazione che rende l'applicazione più manutenibile e scalabile. Una progettazione accurata di questa interazione è essenziale per garantire efficienza, sicurezza e affidabilità.

4 Framework, Toolkit e Protocolli Utilizzati

4.1 Flask

Flask è un framework web leggero e flessibile per Python, utilizzato per creare applicazioni web e API. È stato progettato per essere semplice e minimalista, lasciando all'utente la libertà di scegliere come strutturare e organizzare il progetto. Grazie alla sua modularità e alla vasta comunità di sviluppatori, Flask è una delle scelte più popolari per applicazioni Python-based.

4.1.1 Caratteristiche Principali

1. **Leggerezza e semplicità:** Flask è un microframework, il che significa che fornisce solo le funzionalità essenziali per creare applicazioni web. Non include funzionalità complesse come ORM o autenticazione predefiniti, ma offre la possibilità di aggiungere estensioni per espandere le sue capacità.
2. **Modularità:** È costruito in modo modulare, permettendo agli sviluppatori di aggiungere o rimuovere funzionalità secondo necessità. Questo approccio favorisce la creazione di applicazioni personalizzate e scalabili.
3. **Routing intuitivo:** Flask utilizza un sistema di routing basato su decoratori, che rende semplice definire endpoint e associarli a funzioni Python. Ad esempio, per definire una route, si usa il decoratore `@app.route`.
4. **Supporto per template:** Flask integra Jinja2, un motore di template potente e flessibile, che consente di separare la logica dal layout HTML, rendendo il codice più organizzato e leggibile.
5. **Wsgi e Werkzeug:** Flask si basa su Werkzeug, una libreria WSGI (Web Server Gateway Interface) che gestisce richieste HTTP e routing. Questo consente di creare applicazioni compatibili con i principali server web.
6. **Estendibilità:** Grazie alla sua architettura, Flask supporta numerose estensioni per aggiungere funzionalità come l'integrazione con i database, la gestione delle sessioni, l'autenticazione e altro ancora. Alcuni esempi includono Flask-SQLAlchemy, Flask-Migrate e Flask-WTF.
7. **Compatibilità API RESTful:** Flask è particolarmente adatto per creare API RESTful grazie alla sua semplicità e alla capacità di gestire facilmente richieste HTTP, dati JSON e risposte.

4.1.2 Architettura

L'architettura di Flask si basa su un'implementazione che segue il pattern WSGI. Offre un'interfaccia semplice tra server web e applicazioni Python. Flask non impone una struttura rigida, ma incoraggia il pattern Model-View-Controller (MVC) per mantenere il codice organizzato e facilmente manutenibile.

La tipica struttura di un progetto Flask include:

1. **app.py o run.py:**

- Punto di ingresso dell'applicazione.
- Contiene la creazione dell'oggetto Flask e la configurazione di base.

2. **routes/:**

- Definisce le varie route (o endpoint) che l'applicazione gestisce.
- Ogni route è associata a una funzione che restituisce una risposta.

3. **models/:**

- Contiene i modelli di dati (interfaccia con il database), spesso definiti utilizzando un ORM come SQLAlchemy.

4. **templates/:**

- File HTML che utilizzano il motore di template Jinja2 per il rendering dinamico.

5. **static/:**

- Contiene risorse statiche come file CSS, JavaScript e immagini.

6. **extensions.py o configurazioni simili:**

- File per inizializzare e configurare estensioni come il database o il gestore di migrazioni.

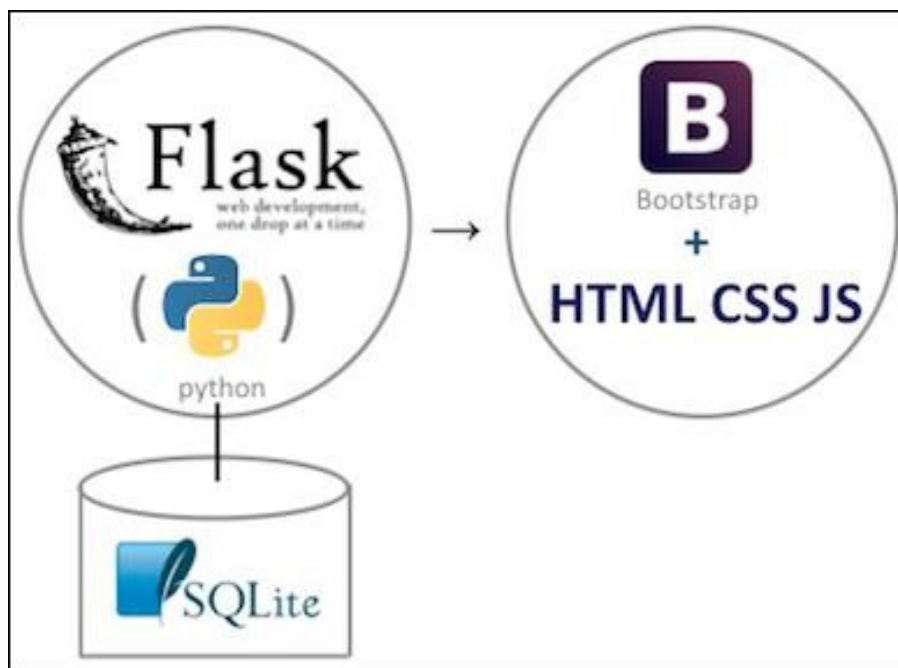


Figura 5: Flask

4.1.3 Vantaggi e Svantaggi

Flask, grazie alla sua natura minimalista e flessibile, offre numerosi vantaggi che lo rendono una scelta popolare per lo sviluppo di applicazioni web. La sua leggerezza, combinata con un'ampia gamma di estensioni e una documentazione chiara, consente agli sviluppatori di costruire progetti personalizzati e scalabili. Inoltre, la semplicità di apprendimento e l'adattabilità alle esigenze specifiche dei progetti lo rendono adatto sia per applicazioni leggere che per API RESTful. Di seguito, vengono analizzati i principali vantaggi offerti da Flask.

- **Leggerezza e semplicità:** Flask è un framework minimalista che permette agli sviluppatori di avere un controllo completo sullo sviluppo dell'applicazione senza imporre regole rigide. È ideale per progetti piccoli e medi.
- **Modularità:** Grazie alla sua natura modulare, puoi aggiungere solo ciò di cui hai bisogno utilizzando estensioni. Questo riduce l'ingombro e migliora le performance.
- **Facilità di apprendimento:** Flask è facile da imparare per chi ha già familiarità con Python. La sua curva di apprendimento è meno ripida rispetto a framework più complessi come Django.
- **Flessibilità:** Non impone scelte rigide su come organizzare il progetto. Puoi scegliere i componenti, i database e gli strumenti che preferisci.
- **Grande ecosistema di estensioni:** Esistono molte estensioni per Flask che semplificano l'integrazione di funzionalità complesse, come autenticazione, gestione di database, creazione di API RESTful, e altro.
- **Comunità attiva e documentazione chiara:** Flask ha una comunità ampia e attiva, con molte risorse, tutorial, e guide disponibili online.
- **Compatibilità con WSGI e strumenti moderni:** Può essere eseguito su qualsiasi server compatibile con WSGI (come Gunicorn) ed è facilmente integrabile con Docker e Kubernetes per la distribuzione.
- **Adatto per API RESTful:** Grazie alla sua flessibilità e leggerezza, è perfetto per creare API RESTful veloci e scalabili.

Nonostante i numerosi vantaggi, Flask presenta anche alcuni svantaggi che potrebbero influire sulla scelta di utilizzarlo, soprattutto per applicazioni di grandi dimensioni o altamente complesse. La sua natura minimalista, sebbene permetta una grande flessibilità, può richiedere più tempo e sforzi per implementare funzionalità che sono già preconfigurate in altri framework più completi. Inoltre, la gestione della crescita del progetto può diventare più difficile in caso di applicazioni complesse. Di seguito, vengono esplorati i principali svantaggi associati all'uso di Flask.

- **Maggiore responsabilità dello sviluppatore:** Flask non fornisce soluzioni preconfigurate per funzionalità comuni (come autenticazione, gestione utenti, o routing complesso). Ciò richiede più lavoro manuale e una maggiore attenzione alla progettazione.

- **Limitazioni per progetti complessi:** Sebbene Flask possa essere usato per applicazioni grandi, gestirle può diventare complicato senza una struttura chiara e modulare.
- **Dipendenza dalle estensioni:** Molte funzionalità avanzate richiedono l'uso di estensioni di terze parti, che possono avere problemi di compatibilità o mancare di aggiornamenti.
- **Minore opinione rispetto a Django:** Flask lascia molte decisioni agli sviluppatori, il che può portare a un'architettura meno consistente o a errori di design, soprattutto in team meno esperti.
- **Supporto integrato limitato per funzionalità avanzate:** Non offre funzionalità avanzate "out-of-the-box" come Django, ad esempio un ORM completo o un pannello di amministrazione.
- **Performance in ambienti ad alto traffico:** Sebbene Flask sia performante per applicazioni leggere, può non essere ideale per applicazioni con traffico molto elevato senza ottimizzazioni e scaling orizzontale.
- **Mancanza di tool integrati:** Al contrario di framework più grandi, Flask non offre strumenti integrati per task come migrazioni di database o validazione di moduli (a meno che non si usino estensioni).

4.2 AJAX

AJAX (Asynchronous JavaScript and XML) è una tecnologia che consente alle applicazioni web di essere più interattive e reattive, migliorando l'esperienza dell'utente. La principale innovazione offerta da AJAX è la possibilità di inviare e ricevere dati dal server senza dover ricaricare l'intera pagina web. Questo approccio migliora la velocità e l'efficienza dell'applicazione, riducendo i tempi di attesa e rendendo l'interazione con l'utente molto più fluida.

4.2.1 Funzionamento

Il funzionamento di AJAX si basa su una serie di passaggi che permettono a una pagina web di inviare richieste al server in modo asincrono e di ricevere risposte senza ricaricare l'intera pagina. Questo approccio offre una grande flessibilità e un'esperienza utente più fluida. Vediamo nel dettaglio come funziona:

1. L'evento di attivazione in JavaScript

Il processo di invio della richiesta AJAX inizia con un evento che viene generato dall'utente. Può trattarsi di un'azione come il clic su un pulsante, il cambiamento di un valore in un campo di input, lo scrolling della pagina o qualsiasi altro evento che desideri monitorare. Quando si verifica questo evento, un'applicazione JavaScript avvia la richiesta al server senza ricaricare la pagina.

2. Creazione della richiesta (XHR - XMLHttpRequest)

Una volta che si verifica l'evento, il browser invia una richiesta al server utilizzando l'oggetto XMLHttpRequest (XHR). Questo oggetto è un'interfaccia JavaScript che permette di inviare richieste HTTP o HTTPS a un server web e ricevere una risposta. In pratica, crea una connessione di rete con il server, ma senza ricaricare la pagina.

L'oggetto XHR consente di specificare:

- Il tipo di richiesta (GET, POST, PUT, DELETE, ecc.)
- L'URL di destinazione a cui inviare la richiesta
- Eventuali parametri o dati da inviare al server (specialmente per le richieste POST)
- La modalità di invio asincrono, che permette al browser di continuare a funzionare mentre attende la risposta dal server

3. Elaborazione della richiesta da parte del server

Una volta che la richiesta viene inviata dal client (browser) al server, il server elabora la richiesta, esegue le operazioni necessarie (ad esempio recuperare dati da un database, eseguire calcoli, ecc.) e prepara una risposta. Il server può restituire diversi tipi di dati, tra cui HTML, JSON, XML, o semplicemente un messaggio di testo.

4. Ricezione della risposta

Dopo che il server ha inviato la risposta, il client (browser) la riceve tramite l'oggetto XMLHttpRequest. Il browser non ricarica l'intera pagina, ma solo i dati necessari vengono aggiornati. Il JavaScript, che ha avviato la richiesta, può elaborare la risposta del server, analizzarla (specialmente se i dati sono in formato JSON o XML) e aggiornare la pagina senza che l'utente se ne accorga. Quando la risposta è pronta, viene eseguito un callback che permette di processare il contenuto della risposta. In base ai dati ricevuti, la pagina può essere aggiornata in modo dinamico, ad esempio aggiungendo nuovi elementi HTML, modificando il contenuto esistente, visualizzando o nascondendo informazioni, ecc.

5. Aggiornamento dinamico della pagina

L'ultimo passaggio consiste nell'uso dei dati ricevuti dal server per modificare dinamicamente il contenuto della pagina. Questo avviene tramite l'uso di JavaScript, che manipola il DOM (Document Object Model) del documento HTML. A seconda dei dati ricevuti, il browser può aggiungere nuovi elementi (come una lista di prodotti), aggiornare quelli esistenti (come il prezzo totale in un carrello) o addirittura rimuovere elementi dalla pagina. AJAX permette di modificare solo le parti di pagina che richiedono aggiornamenti, mantenendo intatta l'esperienza dell'utente, senza la necessità di ricaricare tutta la pagina. Questo riduce il traffico di rete e migliora la reattività dell'applicazione.

6. Feedback all'utente

In molte applicazioni AJAX, è comune includere un meccanismo di feedback per l'utente, come un'animazione di caricamento o un'icona che indica che l'applicazione è in attesa di una risposta dal server. Ciò contribuisce a mantenere l'utente informato sullo stato dell'operazione e migliora l'esperienza complessiva.

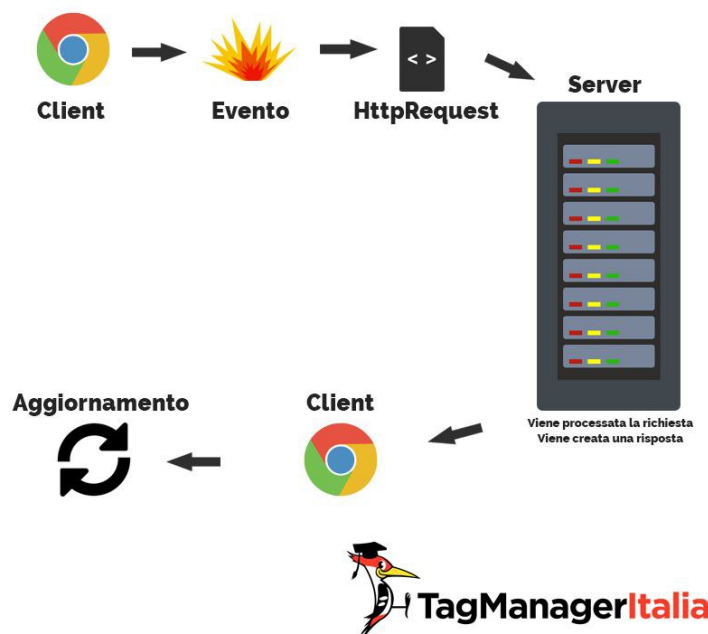


Figura 6: AJAX

4.2.2 Formati dei Dati

In AJAX, i dati possono essere scambiati tra il client (browser) e il server in vari formati. La scelta del formato dipende dal tipo di dati che devono essere trasmessi e dalla tecnologia in uso. I formati più comuni sono JSON, XML, HTML, testo semplice e, in alcuni casi, immagini o file binari. Ogni formato ha vantaggi e svantaggi a seconda del contesto dell'applicazione. Ecco una panoramica dei formati principali:

1. JSON (JavaScript Object Notation)

JSON è il formato più comunemente utilizzato per il trasferimento di dati in applicazioni web moderne. È leggero, facilmente leggibile e scrivibile, e può essere facilmente analizzato sia da JavaScript (lato client) che da molti linguaggi di programmazione lato server. È particolarmente utile quando i dati devono essere strutturati, come nel caso di oggetti complessi o array di valori. Caratteristiche:

- Leggero e conciso.
- Facile da leggere e scrivere per le macchine e gli esseri umani.
- Supportato nativamente in JavaScript tramite `JSON.parse()` e `JSON.stringify()`.
- Utilizzato per scambiare strutture di dati (oggetti e array) in modo efficiente.

2. XML (eXtensible Markup Language)

XML è un altro formato molto utilizzato per lo scambio di dati, soprattutto in contesti più legacy o in sistemi che richiedono una struttura più rigorosa. A differenza di JSON, XML è più verboso e può risultare più difficile da leggere e manipolare, ma è molto potente per rappresentare dati complessi e strutturati. Caratteristiche:

- Più verboso rispetto a JSON.
- Utilizza tag per definire la struttura dei dati.
- Supportato da vari linguaggi di programmazione tramite librerie di parsing XML.
- Utile in contesti dove è necessario un controllo più preciso sulla validità e sulla struttura dei dati (ad esempio, con DTD o schema XSD).

3. HTML

In alcuni casi, la risposta del server può essere un frammento di HTML, che può essere utilizzato per aggiornare una parte della pagina web senza ricaricarla. Questo è tipico nelle applicazioni in cui si vogliono aggiungere nuovi contenuti dinamicamente, come il caricamento di nuovi articoli, post o commenti in una sezione di una pagina. Caratteristiche:

- Utilizzato per restituire direttamente contenuti HTML al client.
- Utilizzato per modificare dinamicamente il DOM (ad esempio, aggiungendo nuove righe a una tabella).
- Spesso è combinato con JavaScript per ottenere un comportamento dinamico.

4. Testo Semplice (Plain Text)

Il formato di testo semplice è utilizzato quando la risposta del server non richiede una struttura complessa, ma solo una stringa di testo. È molto utile quando si desidera restituire informazioni di stato o semplici messaggi di conferma o errore, come un messaggio di successo dopo aver inviato un modulo. Caratteristiche:

- Formato molto semplice e diretto.
- Non ha una struttura complessa, quindi è utile per messaggi di testo semplice.
- Veloce da processare, ma limitato rispetto ad altri formati quando si trattano dati strutturati.

5. File Binari (Immagini, Video, ecc.)

Sebbene non sia comune come gli altri formati, a volte AJAX viene utilizzato per caricare o scaricare file binari, come immagini, PDF o altri tipi di file. Questo richiede una gestione speciale per l'invio e la ricezione dei dati, tipicamente utilizzando la codifica base64 o l'invio diretto tramite un'operazione di tipo "multipart/form-data". Caratteristiche:

- Permette di inviare e ricevere file come immagini, video o documenti.
- Richiede il corretto encoding e il tipo di contenuto appropriato (come multipart/form-data per caricare file tramite moduli HTML).
- Può comportare un maggiore consumo di banda e risorse.

La scelta del formato dipende dalle esigenze specifiche dell'applicazione. JSON è generalmente il formato preferito per lo scambio di dati strutturati in applicazioni moderne, mentre XML è più indicato per scenari che richiedono una struttura formale e rigida. HTML è perfetto per caricare dinamicamente contenuti visibili, mentre il testo semplice è utile per risposte rapide e messaggi di stato. Infine, i file binari sono utilizzati quando è necessario caricare o scaricare contenuti non testuali, come immagini o documenti. La gestione efficace dei formati dei dati è fondamentale per garantire un'interazione fluida e reattiva tra il client e il server nelle applicazioni AJAX.

4.2.3 Vantaggi e Svantaggi

I vantaggi di AJAX sono numerosi e contribuiscono significativamente a migliorare l'esperienza dell'utente e le prestazioni delle applicazioni web. Utilizzando AJAX, è possibile ottenere interfacce più interattive, ridurre il carico sul server e migliorare la velocità di navigazione, offrendo aggiornamenti parziali della pagina senza doverla ricaricare completamente. Tuttavia, questi vantaggi vanno bilanciati con alcune complessità tecniche che potrebbero sorgere durante lo sviluppo dell'applicazione.

- **Aggiornamenti Parziali della Pagina:** Uno dei principali vantaggi di AJAX è la capacità di aggiornare solo una parte della pagina web senza ricaricare l'intera pagina. Questo migliora notevolmente l'esperienza dell'utente, riducendo i tempi di attesa e il carico sul server.

- **Esperienza Utente Migliorata:** Grazie all'aggiornamento dinamico dei contenuti, le applicazioni web AJAX sono più interattive e rapide. L'utente può interagire con l'interfaccia senza dover aspettare il ricaricamento completo della pagina, creando un'esperienza simile a quella delle applicazioni desktop.
- **Riduzione del Carico del Server:** Poiché solo una parte della pagina viene aggiornata, le richieste AJAX riducono il carico sul server rispetto alle richieste tradizionali, in cui l'intera pagina viene ricaricata ogni volta. Ciò può migliorare le prestazioni complessive del sistema.
- **Interazione Asincrona:** AJAX consente l'interazione asincrona, il che significa che l'utente può continuare a interagire con l'interfaccia mentre i dati vengono caricati o inviati in background. Questo rende le applicazioni più reattive e veloci.
- **Compatibilità con Tecnologie Esistenti:** AJAX può essere integrato con tecnologie web esistenti, come HTML, CSS, e JavaScript, senza richiedere modifiche drastiche all'infrastruttura di un'applicazione web. È compatibile con la maggior parte dei browser moderni.
- **Supporto per Applicazioni Web Interattive:** AJAX è una tecnologia fondamentale per la realizzazione di applicazioni web moderne come social network, piattaforme di e-commerce e applicazioni di chat in tempo reale, che richiedono aggiornamenti continui e interazioni fluide.

Nonostante i numerosi vantaggi, l'uso di AJAX comporta anche alcuni svantaggi che devono essere considerati durante lo sviluppo di applicazioni web. La gestione della compatibilità tra diversi browser, la gestione di errori e il potenziale impatto sulle prestazioni in caso di richieste troppo frequenti sono solo alcune delle sfide. Inoltre, la logica di aggiornamento asincrono può rendere più complesso il debugging e la gestione dello stato dell'applicazione.

- **Compatibilità con i Browser Più Vecchi:** Sebbene la maggior parte dei browser moderni supporti AJAX, alcune versioni precedenti potrebbero non essere compatibili. Ciò può causare problemi di accessibilità per gli utenti che utilizzano browser più datati.
- **Difficoltà nel Mantenere lo Stato della Pagina:** Poiché AJAX aggiorna solo una parte della pagina, può risultare più difficile gestire lo stato della pagina, specialmente quando si tratta di navigazione tra le pagine o di aggiungere funzionalità di "Back" del browser. Il sistema di navigazione tradizionale basato su URL potrebbe non funzionare correttamente con AJAX, e potrebbe essere necessario implementare soluzioni personalizzate.
- **Problemi con la Sicurezza:** AJAX invia e riceve richieste dal server, il che significa che le vulnerabilità di sicurezza, come le iniezioni SQL o il cross-site scripting (XSS), devono essere attentamente gestite. La gestione della sicurezza è più complessa rispetto alle applicazioni che non utilizzano AJAX.
- **Maggiore Complessità nello Sviluppo:** L'uso di AJAX può aggiungere complessità allo sviluppo di un'applicazione, soprattutto quando si deve gestire la comunicazione asincrona, il caricamento dei dati in background e la sincronizzazione del

contenuto della pagina. Richiede competenze avanzate di JavaScript, soprattutto quando si devono gestire scenari complessi.

- **Consumo di Banda:** Sebbene AJAX riduca il carico sul server, può aumentare il traffico di rete, specialmente se vengono effettuate molte richieste al server in breve tempo. Ogni richiesta di aggiornamento dei dati o di caricamento di contenuti può richiedere una connessione separata, consumando risorse di rete.
- **Debugging Complesso:** Il debugging delle applicazioni AJAX può risultare difficile, in quanto le richieste sono asincrone e il flusso di dati può essere complicato da tracciare. Gli sviluppatori devono gestire correttamente il flusso di controllo tra client e server per garantire che le risposte vengano gestite correttamente.
- **Problemi con il SEO:** Poiché AJAX modifica dinamicamente il contenuto della pagina senza ricaricarla, i motori di ricerca potrebbero non indicizzare correttamente i contenuti. Ciò può compromettere la visibilità del sito nei risultati di ricerca, specialmente se il contenuto caricato tramite AJAX non è reso disponibile agli spider dei motori di ricerca.

In sintesi, AJAX offre numerosi vantaggi in termini di miglioramento dell'esperienza utente, prestazioni e interattività delle applicazioni web. Tuttavia, introduce anche alcune sfide, come la compatibilità con i browser, la gestione dello stato della pagina, la sicurezza e la complessità nello sviluppo e nel debugging. La sua implementazione deve essere bilanciata con l'architettura complessiva dell'applicazione e le necessità specifiche del progetto.

4.3 SQLAlchemy

SQLAlchemy è una libreria Python di mappatura degli oggetti relazionali (ORM - Object-Relational Mapping), che fornisce una soluzione potente e flessibile per interagire con database relazionali. SQLAlchemy consente agli sviluppatori di interagire con il database in modo più naturale, usando oggetti Python anziché scrivere query SQL a mano. Questo approccio facilita la gestione dei dati e migliora la manutenibilità del codice.

4.3.1 Caratteristiche Principali

1. **Mapping tra oggetti Python e tabelle del database:** SQLAlchemy consente di definire modelli Python che rappresentano le tabelle del database, rendendo l'interazione con i dati più fluida. Utilizzando le classi Python, è possibile mappare ogni oggetto a una riga di una tabella nel database, e ogni attributo dell'oggetto corrisponderà a una colonna della tabella.
2. **Gestione delle query:** SQLAlchemy offre un linguaggio di query ad alto livello che astrae il lavoro con le query SQL. L'API di query consente di scrivere query in modo molto simile alla sintassi Python, riducendo la necessità di interagire direttamente con il linguaggio SQL. Questo rende il codice più pulito e leggibile.
3. **Supporto per le transazioni:** SQLAlchemy fornisce supporto completo per la gestione delle transazioni, consentendo operazioni atomicamente consistenti. Si possono eseguire più operazioni sul database come parte di una singola transazione, che viene committata solo se tutte le operazioni sono riuscite.
4. **Gestione avanzata delle relazioni:** SQLAlchemy gestisce diversi tipi di relazioni tra tabelle, come uno-a-molti, molti-a-molti, e uno-a-uno, con meccanismi che rendono facile lavorare con dati relazionati. Le relazioni tra tabelle vengono rappresentate attraverso oggetti, il che semplifica notevolmente la gestione delle operazioni di join e la navigazione dei dati.
5. **Compatibilità con vari database:** SQLAlchemy è compatibile con una vasta gamma di sistemi di gestione di database relazionali (RDBMS), come MySQL, PostgreSQL, SQLite, Oracle, SQL Server, e altri. Questo lo rende molto versatile e adatto a progetti che potrebbero migrare tra diversi sistemi di database.

4.3.2 Architettura

SQLAlchemy segue un'architettura che può essere descritta come un misto tra l'approccio di ORM e quello di query builder. Può essere utilizzato in due modalità principali:

1. **ORM (Object-Relational Mapping):** In questa modalità, SQLAlchemy mappa le tabelle del database su classi Python. Ogni classe è un oggetto che rappresenta una riga della tabella, e gli attributi della classe sono le colonne della tabella. La comunicazione con il database avviene automaticamente attraverso la manipolazione degli oggetti Python. SQLAlchemy ORM gestisce anche le relazioni tra tabelle e consente operazioni di query complesse senza scrivere SQL direttamente.

2. **Core (SQL Expression Language):** Il Core è un sistema a basso livello che offre una sintassi simile a SQL per la creazione di query. Con il Core, gli sviluppatori possono creare query SQL in modo più esplicito, ma senza il bisogno di scrivere SQL grezzo. Le query possono essere manipolate come oggetti Python e poi convertite in SQL effettivo.

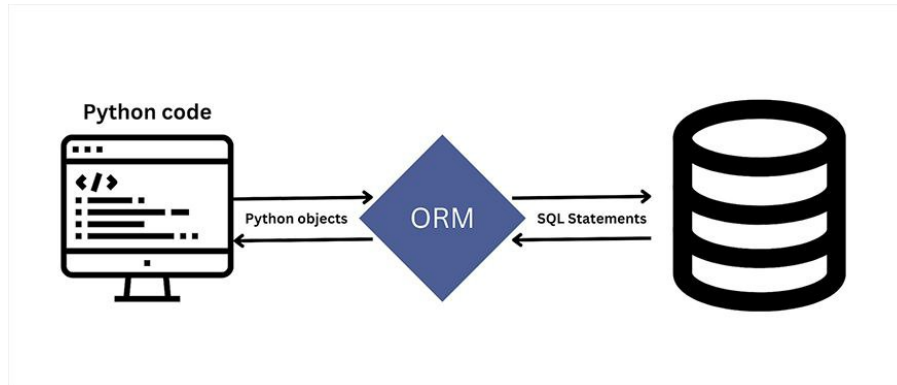


Figura 7: SQLAlchemy

4.3.3 Vantaggi e Svantaggi

I vantaggi di SQLAlchemy sono numerosi e riflettono la sua versatilità e potenza come framework di mappatura oggetto-relazionale (ORM). Tra i principali benefici, SQLAlchemy semplifica notevolmente la gestione dei dati, rendendo più facile l'interazione con il database attraverso oggetti Python anziché complesse query SQL. Inoltre, offre un elevato livello di astrazione, supporta una varietà di database e facilita la gestione delle relazioni tra le tabelle, migliorando l'efficienza e la manutenzione del codice nel lungo periodo. Tuttavia, come ogni tecnologia, presenta anche alcune sfide e limitazioni che è importante considerare.

- **Astrazione del Database:** SQLAlchemy fornisce un livello di astrazione che consente agli sviluppatori di interagire con il database utilizzando oggetti Python invece di scrivere direttamente SQL. Questo rende il codice più leggibile, manutenibile e meno propenso a errori.
- **Indipendenza dal Database:** Grazie alla sua architettura, SQLAlchemy offre una separazione tra la logica applicativa e il tipo di database utilizzato. Questo significa che è relativamente facile migrare da un sistema di database a un altro (ad esempio da SQLite a PostgreSQL) senza dover riscrivere gran parte del codice.
- **Flessibilità nella gestione delle Query:** SQLAlchemy è molto flessibile in quanto permette di scegliere tra l'approccio ORM, che automatizza la gestione degli oggetti e delle relazioni tra le tabelle, e il Core, che consente di scrivere query in un linguaggio simile a SQL. Ciò consente di adattarsi a scenari diversi, combinando il meglio dei due mondi.
- **Gestione delle Relazioni:** Una delle caratteristiche più potenti di SQLAlchemy è la gestione delle relazioni tra le tabelle. Supporta diversi tipi di relazioni come uno-a-molti, molti-a-molti, e uno-a-uno, semplificando notevolmente il lavoro con dati relazionati.

- **Supporto alle Transazioni:** SQLAlchemy offre una gestione robusta delle transazioni, che permette di eseguire diverse operazioni come parte di una singola transazione. Questo garantisce l'integrità dei dati e riduce il rischio di corruzione dei dati in caso di errore o interruzione delle operazioni.
- **Ampia Compatibilità con i Database:** SQLAlchemy è compatibile con un ampio range di database relazionali, tra cui PostgreSQL, MySQL, SQLite, Oracle e SQL Server. Questo lo rende ideale per progetti che potrebbero essere eseguiti su diversi ambienti di database o che potrebbero richiedere migrazioni tra sistemi di gestione del database.
- **Gestione delle Migrazioni:** SQLAlchemy è spesso utilizzato con strumenti come Alembic per la gestione delle migrazioni del database. Questo permette di tenere traccia delle modifiche al modello dei dati nel tempo, garantendo che il database venga aggiornato correttamente.

Nonostante i numerosi vantaggi, SQLAlchemy presenta anche alcuni svantaggi che potrebbero influenzare la scelta di utilizzarlo in determinati progetti. La sua curva di apprendimento può essere piuttosto ripida, specialmente per chi è nuovo al concetto di ORM o alle sue funzionalità avanzate. Inoltre, a causa del suo livello di astrazione, in alcuni casi potrebbe essere meno performante rispetto all'uso diretto di query SQL, soprattutto in scenari che richiedono operazioni molto complesse o ottimizzazioni specifiche. È anche importante considerare la possibilità di dover gestire casi particolari in cui l'ORM non è il più adatto.

- **Curva di Apprendimento Iniziale:** SQLAlchemy, pur essendo molto potente, ha una curva di apprendimento piuttosto ripida. Gli sviluppatori devono comprendere sia il funzionamento dell'ORM che le sue componenti più avanzate (come le relazioni, i tipi di dati complessi, la gestione delle transazioni). Se non si ha familiarità con i concetti di ORM, l'approccio può sembrare complesso inizialmente.
- **Prestazioni:** L'uso dell'ORM può comportare un overhead rispetto alla scrittura di query SQL dirette, specialmente per operazioni complesse o con grandi quantità di dati. Il fatto che l'ORM traduca le operazioni in SQL può causare una perdita di efficienza rispetto a scrivere manualmente query SQL ottimizzate.
- **Non Ottimale per Applicazioni Semplici:** Se un'applicazione non richiede la gestione complessa di dati o relazioni tra tabelle, utilizzare SQLAlchemy potrebbe risultare eccessivo. In scenari in cui sono necessarie solo semplici query, un approccio più leggero come l'uso di query SQL dirette o l'utilizzo di librerie più semplici può essere preferibile.
- **Complessità nella Configurazione:** La configurazione di SQLAlchemy e dei suoi componenti (come il mapping tra oggetti e tabelle o la gestione delle sessioni) può essere complicata, soprattutto in progetti di grandi dimensioni. Questo può richiedere più tempo per essere configurato correttamente rispetto ad altre soluzioni più dirette.
- **Gestione delle Query Complesse:** Sebbene SQLAlchemy offra un linguaggio di query potente, per query particolarmente complesse o ad alte prestazioni, potrebbe essere necessario tornare a scrivere SQL manualmente. In alcuni casi, l'ORM

può generare query non ottimizzate, e questo può avere un impatto negativo sulle performance dell'applicazione.

- **Dipendenza dalla Libreria:** Sebbene SQLAlchemy astrae molte complessità del database, ciò implica una certa dipendenza dalla libreria stessa. Questo può rappresentare un problema se si desidera passare a una libreria diversa o implementare soluzioni personalizzate senza l'uso di un ORM.
- **Debugging Difficile:** Poiché SQLAlchemy traduce le operazioni Python in SQL, può essere difficile per gli sviluppatori comprendere esattamente quali query vengono eseguite sul database, specialmente quando si verificano problemi di prestazioni. Sebbene ci siano strumenti di logging per tracciare le query, il debugging di query complesse può essere meno immediato rispetto alla scrittura diretta di SQL.

In sintesi, SQLAlchemy è uno strumento molto potente e versatile, ma come qualsiasi libreria avanzata, ha anche delle complessità che potrebbero non essere necessarie in tutte le situazioni. I vantaggi di un ORM come SQLAlchemy sono evidenti quando si gestiscono applicazioni con complessi modelli di dati e relazioni, ma per progetti più semplici o ad alte prestazioni, potrebbe essere utile considerare soluzioni alternative o limitarsi a SQL grezzo.

4.4 SQLite

SQLite è un sistema di gestione di basi di dati relazionali (RDBMS) che si distingue per la sua leggerezza, portabilità e facilità d'uso. È progettato per essere incorporato direttamente nelle applicazioni, evitando la necessità di un server separato. La sua natura embedded lo rende ideale per applicazioni che richiedono un database integrato, come applicazioni desktop, mobile o progetti di sviluppo web che non necessitano di una soluzione complessa.

SQLite memorizza i dati in un singolo file di archivio, riducendo il sovraccarico di gestione rispetto ai sistemi di database tradizionali. Non richiede una configurazione o un'installazione complessa e può essere facilmente distribuito insieme all'applicazione.

4.4.1 Caratteristiche Principali

1. **Database incorporato (Embedded):** SQLite è un database "embedded", cioè non richiede un server di database separato per funzionare. Il database viene integrato direttamente all'interno dell'applicazione, e tutto il sistema di gestione del database è contenuto in un singolo file di archivio. Questo elimina la necessità di configurare e gestire un server di database esterno, rendendo il processo di sviluppo e distribuzione molto più semplice.
2. **Semplicità e leggerezza:** Una delle principali caratteristiche di SQLite è la sua leggerezza. È progettato per essere utilizzato in applicazioni che non richiedono un database complesso. Non è necessario installare e configurare un server separato. L'intero sistema di gestione del database può essere distribuito come parte dell'applicazione. Questo lo rende ideale per applicazioni desktop, mobile o web leggere.
3. **Portabilità:** SQLite è molto portatile. Il database è archiviato in un singolo file, che può essere facilmente copiato tra diverse piattaforme, come sistemi operativi Windows, Linux, macOS o dispositivi mobili. Questo lo rende particolarmente utile per applicazioni che devono funzionare su piattaforme diverse senza necessità di adattamenti significativi.
4. **Transazioni ACID:** SQLite supporta completamente le transazioni ACID (Atomicity, Consistency, Isolation, Durability). Questo significa che le operazioni di lettura e scrittura nel database sono sicure e affidabili. Le transazioni garantiscono che anche in caso di errori, come un'interruzione del sistema o una perdita di alimentazione, il database rimanga in uno stato coerente.
5. **Zero configurazione:** SQLite non richiede alcuna configurazione. Poiché è un database embedded, una volta che è integrato nell'applicazione, non è necessaria alcuna configurazione complessa. Non sono richiesti utenti o permessi per accedere al database, e non sono necessarie altre operazioni di gestione tipiche di un database server.
6. **Supporto per linguaggi di programmazione comuni:** SQLite è compatibile con la maggior parte dei linguaggi di programmazione, tra cui Python, PHP, C/C++, Java, e molti altri. Questo lo rende versatile e facilmente integrabile in una vasta gamma di applicazioni e progetti.

7. **Ottimizzazione per operazioni di lettura:** SQLite è particolarmente ottimizzato per operazioni di lettura. Grazie alla sua natura embedded e alla gestione di piccole e medie quantità di dati, è molto veloce nelle operazioni di lettura rispetto ai sistemi di database più complessi.
8. **Supporto per SQL completo:** SQLite implementa un subset completo del linguaggio SQL, consentendo di eseguire query complesse, join, operazioni di ordinamento, filtri, e altre operazioni avanzate sui dati. Anche se non offre tutte le funzionalità avanzate di database server come la gestione delle partizioni o degli indici distribuiti, il supporto per SQL è sufficientemente robusto per le esigenze di molte applicazioni.
9. **Sicurezza tramite crittografia (opzionale):** Mentre SQLite di base non offre funzionalità di crittografia, è possibile implementare un livello di sicurezza tramite librerie esterne come SQLCipher. Questo permette di criptare il file di database, proteggendo i dati in caso di accesso non autorizzato.
10. **Dimensione ridotta:** SQLite è progettato per avere un'impronta molto piccola. La libreria che implementa SQLite è solitamente molto compatta, con un file di dimensioni minime che lo rende ideale per dispositivi con risorse limitate, come dispositivi mobili o microcontroller.
11. **Supporto per il locking e il controllo della concorrenza:** Anche se SQLite non è progettato per gestire enormi volumi di traffico e accesso concorrente, supporta il locking a livello di database per prevenire conflitti nelle operazioni di scrittura. Questo è adeguato per molte applicazioni che non necessitano di un accesso simultaneo da più utenti.
12. **Backup e ripristino semplici:** Essendo un file monolitico, il backup e il ripristino di un database SQLite è estremamente semplice. È sufficiente copiare il file del database o utilizzare i comandi di backup nativi di SQLite per creare copie di sicurezza e ripristinarle facilmente.
13. **Supporto per viste e trigger:** SQLite supporta le viste (views) e i trigger, che permettono di creare strutture più avanzate per l'interazione con il database, permettendo operazioni come il monitoraggio automatico di modifiche e la creazione di visioni personalizzate dei dati.

Queste caratteristiche fanno di SQLite una scelta eccellente per progetti con requisiti di database leggeri e semplici, come applicazioni web e mobile a bassa intensità di traffico, prototipi o applicazioni embedded.

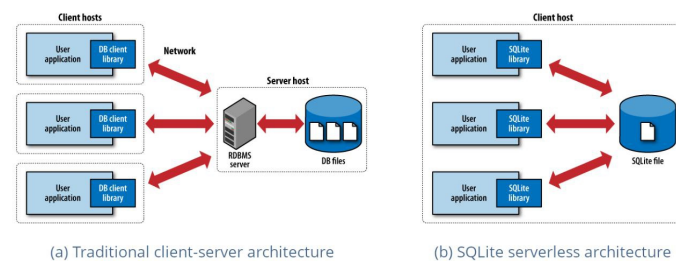


Figura 8: SQLite

4.4.2 Vantaggi e Svantaggi

I vantaggi di SQLite lo rendono una scelta popolare per molte applicazioni, soprattutto in contesti dove la semplicità e la leggerezza sono fondamentali. Grazie alla sua struttura minimale e alla facilità di integrazione, SQLite è ideale per progetti che richiedono un database veloce, portatile e facile da gestire. Tuttavia, nonostante la sua efficienza nelle operazioni di lettura e nella gestione di database di dimensioni contenute, presenta anche alcune limitazioni.

- **Semplicità e Facilità di Utilizzo:** SQLite è estremamente facile da configurare e utilizzare. Non richiede server separati o configurazioni complicate. Una volta che il database è integrato nell'applicazione, gli sviluppatori possono iniziare subito a lavorare senza preoccuparsi di configurazioni complesse o amministrazione.
- **Portabilità:** Poiché SQLite memorizza l'intero database in un singolo file, può essere facilmente spostato tra piattaforme e dispositivi. Questo è particolarmente utile per le applicazioni che devono essere eseguite su diverse piattaforme (Windows, macOS, Linux, Android, iOS), dove la portabilità del database è fondamentale.
- **Bassa Impronta di Memoria:** SQLite è progettato per avere una dimensione ridotta, il che lo rende ideale per dispositivi con risorse limitate, come dispositivi embedded, microcontroller, o applicazioni mobili con limitata memoria e potenza di calcolo.
- **Zero Configurazione:** Non è necessario alcun setup di server o configurazioni di rete, poiché SQLite è un database incorporato. La gestione del database è trasparente all'utente, riducendo la complessità delle operazioni di amministrazione e facilitando la distribuzione e l'uso.
- **Velocità nelle Operazioni di Lettura:** SQLite è altamente ottimizzato per le operazioni di lettura, il che lo rende veloce per applicazioni con una bassa frequenza di scrittura ma alta frequenza di lettura.
- **Completamente ACID Compliant:** SQLite supporta completamente le proprietà ACID (Atomicità, Coerenza, Isolamento, Durabilità), il che significa che le operazioni di database sono sicure e affidabili, garantendo l'integrità dei dati anche in caso di crash o interruzioni impreviste.
- **Facilità di Backup e Ripristino:** Poiché il database è memorizzato in un unico file, il backup e il ripristino sono semplicissimi: basta copiare il file del database. Inoltre, SQLite fornisce funzionalità di backup nativi che rendono facile automatizzare queste operazioni.
- **Supporto per SQL Completo:** SQLite supporta la maggior parte delle funzionalità di SQL standard, incluse le query, le join, le transazioni, le viste, i trigger e altre caratteristiche avanzate, che lo rendono una scelta solida per molte applicazioni.

Nonostante i numerosi vantaggi, SQLite presenta alcuni svantaggi che potrebbero renderlo meno adatto per applicazioni di grandi dimensioni o con requisiti complessi. Le sue limitazioni emergono principalmente in scenari ad alta concorrenza, in cui le prestazioni potrebbero non essere all'altezza delle necessità, e nella gestione di operazioni che richiedono un supporto avanzato per la gestione di grandi volumi di dati o complessi

schemi di dati. È importante valutare questi aspetti in relazione alle esigenze specifiche del progetto per determinare se SQLite sia la scelta giusta.

- **Limitato Supporto per la Concorrenza:** SQLite non è progettato per gestire carichi di lavoro con accesso simultaneo massiccio da parte di molteplici client. Le operazioni di scrittura sono limitate a una singola transazione alla volta, e mentre supporta il locking a livello di database, non è adatto per applicazioni con un numero elevato di accessi concorrenti.
- **Non Adatto per Grandi Volumi di Dati:** Anche se SQLite è efficiente per piccole e medie quantità di dati, non è progettato per gestire database di grandi dimensioni o applicazioni con un numero elevato di operazioni simultanee o una quantità massiva di dati. Per questi scenari, è più adatto un sistema di database server-based come MySQL, PostgreSQL o Oracle.
- **Prestazioni nelle Operazioni di Scrittura:** Sebbene SQLite sia molto veloce nelle letture, le prestazioni delle operazioni di scrittura sono inferiori rispetto ai database server più robusti. Questo è dovuto alla modalità di gestione delle transazioni e al locking a livello di database.
- **Mancanza di Supporto per Funzionalità Avanzate:** SQLite manca di alcune funzionalità avanzate che si trovano in database server più complessi, come la gestione avanzata della concorrenza (es. transazioni distribuite, sharding) o la replicazione dei dati. Inoltre, non supporta nativamente la gestione degli utenti o la creazione di permessi complessi.
- **Limitazioni nella Gestione dei Dati Complessi:** SQLite può risultare meno efficace nella gestione di grandi quantità di dati complessi o relazioni molto articolate. Mentre è molto potente nelle applicazioni più semplici, in scenari complessi, potrebbe non essere la soluzione più adatta.
- **Non adatto per applicazioni ad alta disponibilità:** SQLite non offre meccanismi avanzati di alta disponibilità, ridondanza, o failover che i database tradizionali server-based forniscono. Questo significa che se l'applicazione necessita di alta disponibilità o scalabilità, SQLite potrebbe non essere la scelta ideale.
- **Backup e Ripristino Non Ideali in Ambienti Con Molti Client:** Sebbene SQLite abbia funzionalità di backup relativamente semplici, in ambienti con molti utenti che accedono contemporaneamente al database, la gestione del backup può causare problemi, poiché il database può essere in uno stato di scrittura durante il processo di backup.
- **Mancanza di Supporto per Carichi di Lavoro Complessi:** Per le applicazioni che necessitano di operazioni molto complesse, come l'elaborazione di query ad alte prestazioni, le funzionalità avanzate di ricerca o l'analisi di grandi volumi di dati, SQLite potrebbe risultare meno performante rispetto a soluzioni di database più robuste.

SQLite è una scelta eccellente per applicazioni leggere, prototipi, applicazioni embedded, e scenari in cui la semplicità, la portabilità e la leggerezza sono fondamentali. Tuttavia, presenta delle limitazioni in termini di concorrenza e scalabilità, che potrebbero renderlo inadeguato per sistemi con carichi di lavoro complessi e requisiti di alta disponibilità.

4.5 API RESTful

Le API RESTful (Representational State Transfer) sono un tipo di architettura di comunicazione per applicazioni web che utilizza il protocollo HTTP per scambiare dati tra client e server. REST si basa su un insieme di principi di progettazione che mirano a creare un sistema semplice, scalabile e ben strutturato. Ecco un approfondimento su cos'è e come funziona una API RESTful.

4.5.1 Caratteristiche Principali

Le API RESTful (Representational State Transfer) sono un tipo di architettura software che sfrutta il protocollo HTTP per la comunicazione tra client e server. Le caratteristiche principali di un'API RESTful sono fondamentali per garantirne la semplicità, la scalabilità e la facilità di implementazione. Ecco un'analisi dettagliata di queste caratteristiche:

1. **Stateless (Senza Stato):** Una delle caratteristiche principali delle API RESTful è che ogni richiesta che il client invia al server è indipendente dalle altre. Ciò significa che ogni richiesta deve contenere tutte le informazioni necessarie per essere elaborata, come i parametri, i dati e le credenziali di autenticazione. Il server non mantiene alcuna memoria dello stato della richiesta precedente. Ogni interazione è trattata come un'operazione completamente separata, e ogni nuova richiesta è completamente autoconclusiva. Questo approccio semplifica la gestione della sessione e migliora la scalabilità, poiché il server non deve gestire né conservare informazioni sugli utenti o sulle transazioni precedenti.
2. **Uniform Interface (Interfaccia Uniforme):** Una API RESTful ha un'interfaccia uniforme che semplifica l'interazione tra client e server. Questo significa che le risorse sono trattate in modo coerente, e tutte le operazioni (come la creazione, la lettura, l'aggiornamento e la cancellazione di dati) avvengono attraverso metodi HTTP standard. L'interfaccia uniforme riduce la complessità, poiché i client non devono preoccuparsi di dettagli specifici sul funzionamento del server. Le risorse vengono identificate tramite URL, e le azioni su queste risorse sono eseguite con metodi HTTP predefiniti:
 - **GET:** Recupera informazioni dalla risorsa.
 - **POST:** Crea una nuova risorsa.
 - **PUT:** Aggiorna una risorsa esistente.
 - **DELETE:** Rimuove una risorsa.
3. **Risorse Identificabili tramite URL:** In un'API RESTful, ogni risorsa (cioè, ogni oggetto o entità con cui il client interagisce) ha un proprio URL univoco. Questo URL funge da identificatore per la risorsa e consente al client di eseguire operazioni su di essa tramite le chiamate HTTP. Ad esempio, un'API di gestione di utenti potrebbe avere un URL per ogni utente, come `https://api.example.com/users/123`, dove 123 rappresenta l'ID univoco dell'utente.

4. **Uso dei Metodi HTTP:** Le API RESTful si basano sui metodi HTTP per interagire con le risorse. Oltre ai metodi base già menzionati (GET, POST, PUT, DELETE), vengono utilizzati anche i metodi HTTP come **PATCH** per l'aggiornamento parziale di una risorsa, e **HEAD** per recuperare solo gli header di una risposta senza il corpo del messaggio. L'uso di questi metodi consente una gestione delle risorse semantica e chiara, rispecchiando la semantica di operazioni classiche come la lettura, l'aggiunta, la modifica o la rimozione dei dati.
5. **Rappresentazione delle Risorse:** Le risorse non sono manipolate direttamente dal client, ma vengono rappresentate in un formato specifico (solitamente JSON o XML). Quando un client invia una richiesta, il server risponde con una rappresentazione della risorsa richiesta. Le rappresentazioni possono essere in vari formati, ma il formato JSON è il più comune nelle API moderne grazie alla sua leggerezza e facilità di utilizzo. Il client può manipolare questi dati (ad esempio, modificando i valori JSON) e inviarli al server per creare o aggiornare risorse.
6. **Cacheable (Cacheabile):** Le risposte fornite dal server possono essere etichettate come cacheabili o non cacheabili. Quando una risposta è cacheabile, ciò significa che può essere memorizzata in cache dal client o da un server intermedio (come un proxy o un CDN) per evitare di dover fare richieste ripetute per le stesse risorse. Questo migliora l'efficienza e riduce il carico del server. Le risorse che non cambiano frequentemente (come immagini, pagine HTML statiche o informazioni di stato) sono tipicamente cacheabili, mentre le risorse dinamiche potrebbero non esserlo.
7. **Layered System (Sistema a Strati):** Il sistema REST consente di essere strutturato in più livelli (layered system), in cui possono essere inseriti server intermedi tra il client e il server finale. Questi server intermedi possono svolgere funzioni come la gestione del traffico, il bilanciamento del carico, la gestione della cache e la protezione da attacchi. Il client non ha bisogno di sapere se sta comunicando con il server finale direttamente o tramite un server intermedio, il che favorisce la scalabilità e la sicurezza.
8. **Code on Demand (Codice su Richiesta) [Opzionale]:** Anche se non è una caratteristica obbligatoria di REST, in alcuni casi i server possono inviare codice eseguibile (come JavaScript) al client. Questo permette al client di eseguire codice dinamico che può arricchire l'interfaccia o completare l'elaborazione dei dati. Tuttavia, questa caratteristica non è ampiamente utilizzata rispetto agli altri principi di REST.

In sintesi, le API RESTful sono caratterizzate da una serie di principi e convenzioni che ne semplificano l'adozione e l'uso. Queste caratteristiche le rendono particolarmente adatte per creare applicazioni web scalabili e facili da mantenere.

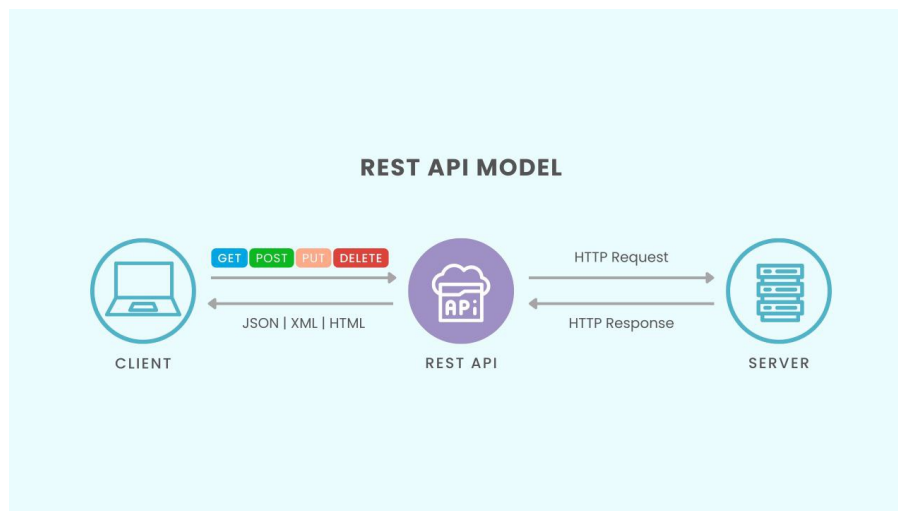


Figura 9: API RESTful

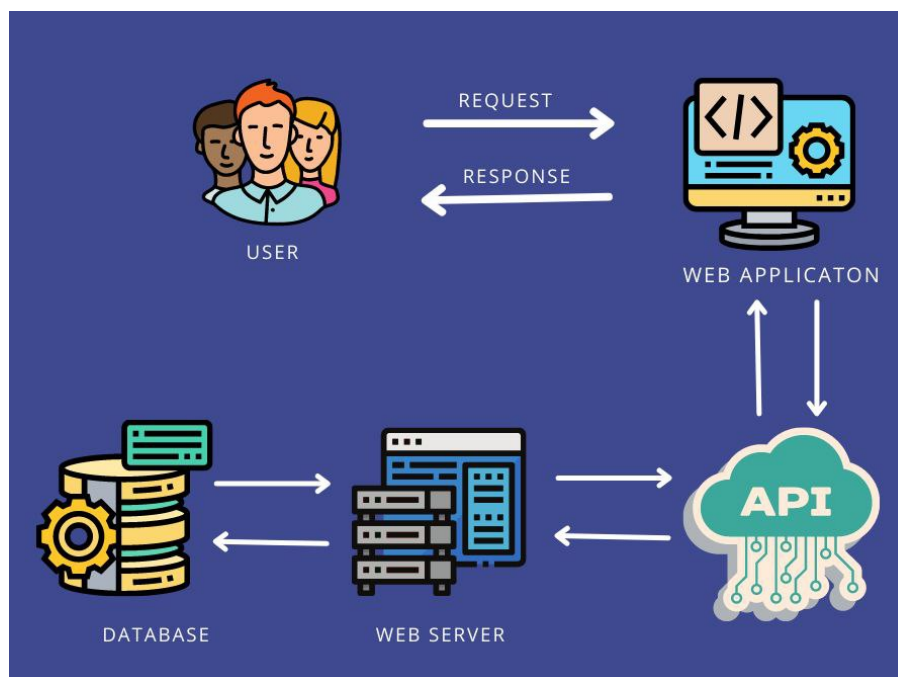


Figura 10: API RESTful: Sistema Completo

4.5.2 Vantaggi e Svantaggi

I vantaggi delle API RESTful sono numerosi e le rendono una scelta popolare per lo sviluppo di applicazioni web moderne. Grazie alla loro semplicità, flessibilità e scalabilità, le API RESTful offrono un modo efficace per facilitare la comunicazione tra diverse parti di un sistema distribuito. Inoltre, la loro capacità di sfruttare il protocollo HTTP, un linguaggio universalmente riconosciuto e supportato, le rende facilmente integrabili in una varietà di applicazioni, da quelle mobili a quelle web. Di seguito vengono esplorati i principali vantaggi che le API RESTful offrono agli sviluppatori e alle organizzazioni che le adottano.

- **Semplicità e chiarezza:** REST si basa su principi molto semplici e intuitivi. Utilizzando il protocollo HTTP e i suoi metodi standard (GET, POST, PUT, DELETE), le API RESTful sono facili da comprendere e implementare. Ogni operazione è chiaramente definita e rispetta la semantica del protocollo HTTP.
- **Scalabilità:** Le API RESTful sono ideali per sistemi distribuiti e scalabili. Poiché ogni richiesta è stateless (senza stato), è possibile gestire un grande numero di richieste simultanee senza che il server debba mantenere informazioni sui singoli client, il che riduce il carico e migliora la scalabilità.
- **Interoperabilità:** Le API RESTful sono basate su HTTP, che è un protocollo standard e ampiamente supportato da praticamente tutte le piattaforme e linguaggi di programmazione. Questo consente una comunicazione fluida tra sistemi diversi, migliorando l'interoperabilità.
- **Supporto per formati variabili:** Sebbene il formato più comune sia JSON, RESTful API può supportare diversi formati di risposta, tra cui XML, YAML, e HTML. Questo le rende flessibili e adatte a diversi casi d'uso e preferenze del client.
- **Statelessness (senza stato):** Il fatto che le API RESTful siano senza stato (stateless) è un grande vantaggio in termini di prestazioni e scalabilità. Poiché ogni richiesta è indipendente, il server non ha bisogno di mantenere informazioni tra le richieste, il che semplifica la gestione delle risorse.
- **Facile caching:** Poiché REST utilizza HTTP, è facile implementare la cache per ridurre il carico sul server e migliorare le prestazioni. Le risposte possono essere etichettate come cacheabili, e il client può memorizzare temporaneamente le risposte per evitare di effettuare richieste ripetitive.
- **Adatto per applicazioni mobile e web:** Le API RESTful sono particolarmente adatte per le applicazioni mobili e web moderne, poiché sono leggere, facili da consumare da parte di client come browser, applicazioni mobili o applicazioni desktop.

Nonostante i numerosi vantaggi, le API RESTful presentano anche alcuni svantaggi che possono influire sull'efficacia e l'efficienza di un sistema, specialmente in scenari complessi o ad alta intensità di dati. Sebbene siano relativamente semplici da implementare, l'adozione di RESTful API può portare a limitazioni in termini di prestazioni, gestione della sicurezza o capacità di scalabilità in alcune situazioni. In questa sezione, esploreremo alcuni degli svantaggi più comuni che gli sviluppatori possono incontrare quando utilizzano le API RESTful, mettendo in evidenza le aree in cui potrebbero esserci delle criticità.

- **Mancanza di operazioni complesse:** Le API RESTful sono orientate a operazioni CRUD (Create, Read, Update, Delete) che si basano sui metodi HTTP. Sebbene siano adatte per la maggior parte delle applicazioni web, per operazioni più complesse che richiedono interazioni più articolate tra client e server, REST potrebbe non essere la soluzione migliore. In questi casi, potrebbero essere necessarie soluzioni più avanzate, come GraphQL.
- **Overhead di richieste:** Ogni richiesta HTTP in un'API RESTful è completamente indipendente e richiede la trasmissione di tutti i dati necessari. In applicazioni

ad alte prestazioni o in sistemi in cui le interazioni tra client e server devono essere rapide e continuative, questo overhead di richieste può rappresentare un problema, specialmente in caso di molteplici chiamate API per ottenere dati correlati.

- **Sicurezza:** Sebbene le API RESTful possano essere sicure se correttamente implementate, la natura stateless significa che ogni richiesta deve essere autenticata separatamente, il che può portare a una gestione più complessa della sicurezza rispetto ad altre architetture come SOAP, che offre un approccio integrato alla sicurezza tramite WS-Security.
- **Manca di standardizzazione per la gestione delle risorse:** Sebbene le risorse in un'API RESTful siano identificate tramite URL, non esiste una vera e propria convenzione universale su come progettare un'API RESTful "perfetta". I vari sviluppatori o team potrebbero avere approcci diversi su come definire le risorse, rendendo la progettazione delle API più complessa e suscettibile di discrepanze tra i vari sistemi.
- **Non adatta a operazioni in tempo reale:** REST non è l'ideale per le applicazioni che richiedono comunicazione in tempo reale o a bassa latenza. Poiché ogni richiesta è un'operazione separata, non è facilmente integrabile con tecnologie che richiedono un flusso di dati continuo o l'interazione in tempo reale, come WebSockets.
- **Gestione delle versioni:** Sebbene le API RESTful siano facili da modificare, l'evoluzione delle API può causare difficoltà nella gestione delle versioni. La versione delle API deve essere gestita in modo esplicito, e modifiche non compatibili potrebbero rompere la comunicazione tra client e server, obbligando gli sviluppatori a gestire correttamente le versioni.
- **Limitazioni nelle operazioni di ricerca:** Le API RESTful, pur essendo adatte per operazioni CRUD di base, non sono particolarmente ottimizzate per operazioni di ricerca complesse o per l'interrogazione avanzata dei dati. Sebbene sia possibile aggiungere parametri di query, le ricerche sofisticate possono risultare difficili da implementare in modo efficiente.

In sintesi, le API RESTful sono un potente strumento per la creazione di applicazioni distribuite, ma non sono prive di limiti. Sebbene vantaggiose per molti scenari, potrebbero non essere la scelta ottimale per tutti i tipi di applicazioni, in particolare quando le operazioni sono complesse, in tempo reale o richiedono una gestione avanzata delle risorse.

4.6 RabbitMQ

RabbitMQ è un broker di messaggi open source che implementa il protocollo AMQP (Advanced Message Queuing Protocol). È utilizzato per facilitare la comunicazione tra sistemi distribuiti o componenti di un'applicazione. RabbitMQ consente la gestione e l'instradamento di messaggi tra produttori (che inviano messaggi) e consumatori (che li ricevono) tramite code di messaggi, permettendo una comunicazione asincrona e decoupling tra i vari sistemi.

4.6.1 Caratteristiche Principali

1. **Gestione delle code di messaggi:** RabbitMQ è progettato per gestire e memorizzare i messaggi in code, consentendo di inviare messaggi tra diverse applicazioni o componenti senza che siano direttamente connessi.
2. **Supporto per più protocolli:** Sebbene AMQP sia il protocollo principale, RabbitMQ supporta anche altri protocolli come MQTT e STOMP, il che lo rende versatile e adatto a diversi scenari di comunicazione.
3. **Affidabilità:** RabbitMQ fornisce funzionalità come il "messaggio persistente" e l'acknowledgement dei messaggi, che garantiscono che i messaggi non vadano persi anche in caso di guasti del sistema.
4. **Scalabilità:** RabbitMQ supporta una configurazione cluster, permettendo di distribuire il carico di lavoro su più nodi per migliorare la capacità di gestione dei messaggi.
5. **Routing avanzato:** RabbitMQ offre una robusta gestione del routing dei messaggi, consentendo di instradare i messaggi a destinazioni specifiche in base a criteri di routing definiti.
6. **Controllo delle priorità:** Le code di RabbitMQ supportano la gestione delle priorità per i messaggi, che consente di trattare i messaggi più urgenti prima di quelli con priorità inferiore.
7. **Exchange:** RabbitMQ utilizza "exchange" per determinare a quale coda i messaggi devono essere inviati. Gli exchange possono essere di tipo direct, fanout, topic, e headers, offrendo molta flessibilità nel routing dei messaggi.
8. **Consumer e producer:** Un "producer" è l'entità che invia i messaggi a RabbitMQ, mentre un "consumer" è l'entità che riceve e processa i messaggi dalle code. Il sistema consente la gestione di più consumer per una coda.

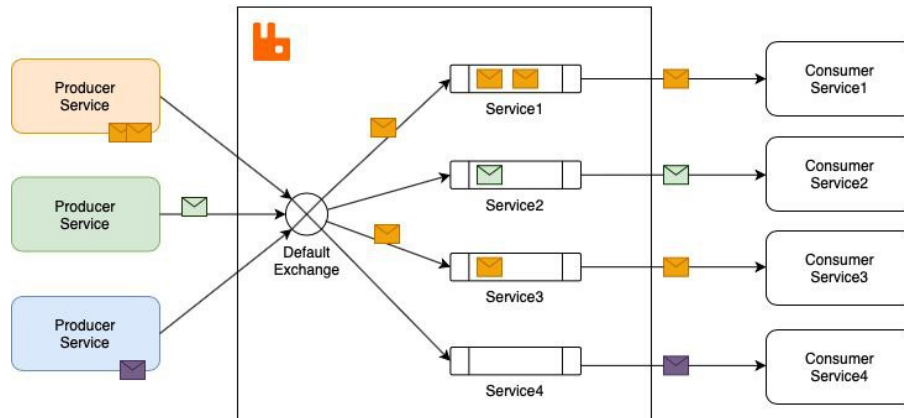


Figura 11: RabbitMQ

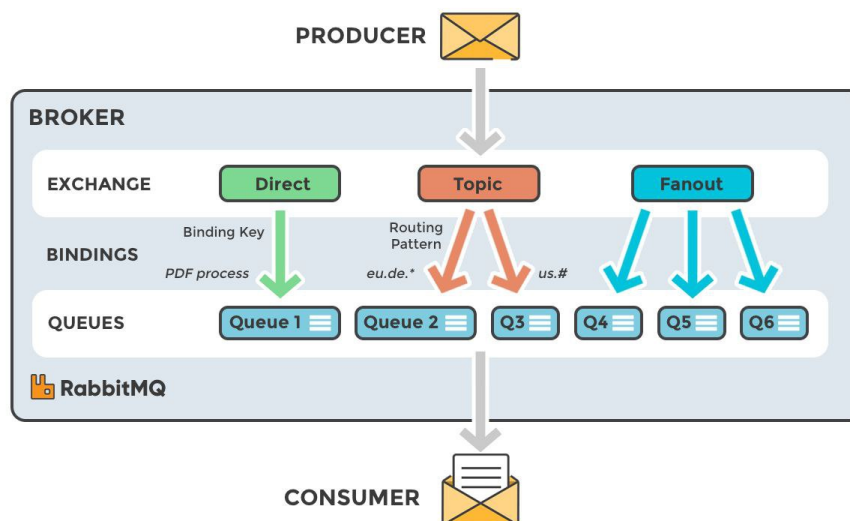


Figura 12: RabbitMQ: Un Producer ed Un Consumer

4.6.2 Advanced Message Queuing Protocol (AMQP)

AMQP (Advanced Message Queuing Protocol) è un protocollo di messaggistica avanzato che consente la comunicazione sicura, affidabile e scalabile tra applicazioni distribuite. Esso è stato progettato per risolvere diverse problematiche comuni nei sistemi distribuiti, come la gestione delle code, la persistenza dei messaggi, il routing flessibile e la garanzia di consegna anche in caso di guasti temporanei. AMQP è principalmente utilizzato per facilitare la comunicazione tra microservizi o altre componenti di sistemi complessi. Le principali caratteristiche di AMQP includono:

1. **Affidabilità:** AMQP assicura la consegna dei messaggi senza perdita di dati, anche in caso di guasti temporanei. I messaggi vengono ricevuti nell'ordine corretto, e possono essere recuperati e processati in caso di disconnessione o errori.
2. **Flessibilità nel routing:** Grazie agli "exchange", è possibile definire regole di routing dei messaggi verso le code appropriate, a seconda di parametri specifici, come il contenuto del messaggio o la sua priorità. Questo permette di costruire architetture scalabili e complesse.

3. **Transazioni:** AMQP supporta le transazioni, garantendo l'integrità dei dati e la possibilità di inviare più messaggi come parte di un'unica operazione atomica. In caso di errore, nessuno dei messaggi verrà processato, mantenendo la coerenza dei dati.
4. **Persistenza dei messaggi:** I messaggi possono essere persi se il sistema va in crash, ma AMQP permette di configurare la persistenza, assicurando che i dati vengano conservati finché non siano completamente elaborati.

Tuttavia, AMQP presenta alcuni svantaggi:

- **Overhead:** L'uso di AMQP introduce un certo overhead, che può rallentare le prestazioni, soprattutto in scenari in cui sono richieste risposte rapide o bassa latenza.
- **Complessità nella gestione:** Configurare e gestire un sistema AMQP richiede competenza, poiché il protocollo è complesso e necessita di un'infrastruttura adeguata per garantire il corretto funzionamento.
- **Non ottimale per applicazioni in tempo reale:** Poiché AMQP include meccanismi di persistenza, acknowledgment e gestione delle code, potrebbe non essere la scelta migliore per applicazioni che richiedono una comunicazione istantanea e senza latenza, come nei giochi online o nelle applicazioni di streaming in tempo reale.

Nonostante questi svantaggi, AMQP è molto utilizzato in contesti dove è fondamentale una comunicazione sicura e affidabile tra componenti distribuite. È particolarmente utile nelle architetture a microservizi, nelle piattaforme di messaggistica, e nei sistemi di gestione dei flussi di lavoro enterprise, dove la garanzia della consegna dei messaggi e la loro integrità sono requisiti fondamentali. Grazie alla sua versatilità, AMQP è uno degli strumenti più potenti per gestire la comunicazione tra componenti di sistemi complessi.

4.6.3 Vantaggi e Svantaggi

I vantaggi di RabbitMQ sono numerosi e contribuiscono a renderlo una scelta ideale per la gestione dei flussi di messaggi tra i componenti di un sistema distribuito. Grazie alla sua affidabilità, scalabilità, e flessibilità nel routing dei messaggi, RabbitMQ risponde a molte delle esigenze di comunicazione in tempo reale in ambienti complessi. Inoltre, il suo supporto per vari protocolli di messaggistica e la facilità d'uso attraverso un'interfaccia di amministrazione intuitiva lo rendono una soluzione versatile per applicazioni di diverse dimensioni e ambiti.

- **Affidabilità:** RabbitMQ garantisce la consegna dei messaggi senza perdita di dati, grazie alla persistenza dei messaggi e al meccanismo di conferma (acknowledgment). Anche in caso di guasti temporanei del server, i messaggi non andranno persi, poiché possono essere recuperati una volta che il sistema torna online.
- **Scalabilità:** RabbitMQ è progettato per scalare orizzontalmente, cioè per essere utilizzato su più nodi di server, consentendo di gestire un numero maggiore di messaggi e di utenti. Può essere configurato per supportare cluster e federazione tra più istanze.

- **Routing dei messaggi flessibile:** Grazie alla sua architettura basata su exchange e code, RabbitMQ offre un ampio controllo sul routing dei messaggi. È possibile definire politiche di routing in base a diversi criteri, come la tipologia dei messaggi o le loro priorità, consentendo una gestione dinamica dei flussi di dati.
- **Supporto per la gestione delle code:** RabbitMQ consente di gestire code multiple, assegnare priorità ai messaggi e garantire la loro elaborazione in ordine. Questo è particolarmente utile in sistemi ad alta disponibilità, in cui è necessario gestire grandi volumi di dati in modo efficiente.
- **Supporto per vari protocolli di messaggistica:** Anche se AMQP è il protocollo principale, RabbitMQ supporta anche altri protocolli di messaggistica, come MQTT, STOMP, e HTTP, permettendo a sistemi eterogenei di comunicare tra loro senza problemi.
- **Facile da usare:** RabbitMQ offre un'interfaccia web di amministrazione che permette agli sviluppatori di configurare e monitorare facilmente il sistema, visualizzando lo stato delle code e dei messaggi, e monitorando la salute del sistema in tempo reale.

Nonostante i numerosi vantaggi, RabbitMQ presenta anche alcuni svantaggi che potrebbero limitare la sua applicabilità in determinate situazioni. Sebbene sia altamente scalabile e robusto, può comportare una certa complessità nella gestione e configurazione, soprattutto in scenari molto grandi o in ambienti distribuiti. Inoltre, la gestione della latenza e delle prestazioni in sistemi ad alta richiesta può rivelarsi una sfida. Altri svantaggi riguardano l'overhead operativo e le risorse necessarie per mantenere il sistema efficiente.

- **Overhead di gestione e configurazione:** Sebbene RabbitMQ offra molte funzionalità avanzate, la sua configurazione e gestione possono risultare complesse, specialmente in ambienti di grandi dimensioni. Ciò richiede una certa esperienza nell'amministrazione di sistemi di messaggistica.
- **Prestazioni:** RabbitMQ introduce un certo overhead, specialmente se vengono utilizzati meccanismi di persistenza e conferma dei messaggi. Questo può influire sulle performance, in particolare quando si lavora con applicazioni a bassa latenza o in tempo reale.
- **Scalabilità limitata in determinate situazioni:** Nonostante la possibilità di scalare orizzontalmente, RabbitMQ potrebbe non essere la soluzione ideale per alcune applicazioni altamente distribuite che richiedono una latenza molto bassa o una gestione di milioni di messaggi al secondo, come nelle architetture altamente reattive.
- **Problemi di performance con grandi volumi di messaggi:** In scenari ad alto traffico con messaggi molto grandi o una frequenza molto elevata di invii e ricezioni, RabbitMQ potrebbe non essere in grado di gestire con la stessa efficienza di altri sistemi di messaggistica come Kafka, che è più orientato alla gestione di flussi di dati in tempo reale.

- **Mancanza di supporto nativo per lo "streaming" di dati:** Sebbene RabbitMQ possa essere utilizzato per il passaggio di messaggi tra sistemi, non è ottimizzato per applicazioni di streaming di dati ad alte prestazioni, come quelle che gestiscono big data o transazioni in tempo reale su larga scala.

Nonostante questi svantaggi, RabbitMQ è una scelta molto popolare per sistemi che necessitano di una comunicazione affidabile e scalabile tra componenti distribuiti, in particolare per applicazioni enterprise, sistemi di gestione dei flussi di lavoro e altre architetture di microservizi.

4.7 Kafka

Apache Kafka è una piattaforma di streaming distribuita che consente di gestire grandi volumi di dati in tempo reale. Nata inizialmente come un sistema di messaggistica, oggi Kafka è utilizzato in molti ambiti per la gestione e l'elaborazione di flussi di dati ad alta velocità. Kafka è progettato per essere altamente scalabile, resiliente e performante, il che lo rende ideale per applicazioni che richiedono una gestione efficiente e continua di eventi o messaggi in tempo reale, come quelle legate all'analisi dei dati, monitoraggio delle applicazioni, e log processing. Kafka è basato su un sistema di pubblicazione-sottoscrizione, dove i produttori di dati (producer) inviano messaggi a uno o più topic, e i consumatori (consumer) leggono i messaggi da questi topic. I dati sono gestiti come flussi (streams), consentendo l'elaborazione continua in tempo reale.

4.7.1 Architettura

L'architettura di Apache Kafka è progettata per essere scalabile, altamente disponibile, e resiliente. È strutturata per gestire enormi flussi di dati in tempo reale e supportare applicazioni che richiedono una gestione continua degli eventi. Kafka adotta un modello di messaggistica distribuita e si basa su una serie di componenti interconnessi che lavorano insieme per garantire l'elaborazione efficiente dei dati. Componenti principali dell'architettura di Kafka:

1. Producer

Il producer è una parte dell'applicazione che pubblica (invia) i dati (messaggi) a un topic di Kafka. I producer inviano messaggi senza preoccuparsi di come o dove vengano memorizzati, affidandosi alla logica interna di Kafka per gestire il routing e la memorizzazione dei messaggi. I producer sono responsabili anche della gestione della qualità dei messaggi inviati, come la serializzazione dei dati.

2. Consumer

Il consumer è un'applicazione che legge i messaggi da un topic. Può leggere messaggi da uno o più topic e processarli in tempo reale. Kafka consente ai consumatori di essere organizzati in gruppi di consumatori. Un gruppo di consumatori è una raccolta di istanze di consumatori che collaborano per leggere messaggi da uno o più topic. I consumatori di un gruppo leggono le partizioni del topic in modo parallelo, bilanciando il carico tra di loro.

3. Broker

I broker sono server che ricevono i messaggi dai producer e li memorizzano. Ogni broker in Kafka è responsabile per la gestione di una parte dei dati. I broker sono organizzati in cluster, che possono essere scalati orizzontalmente aggiungendo nuovi broker per gestire maggiori volumi di dati. Kafka gestisce automaticamente la distribuzione dei dati tra i broker del cluster e si occupa della replicazione per garantire la disponibilità e la resilienza dei dati.

4. Zookeeper

Zookeeper è un sistema di coordinamento distribuito utilizzato da Kafka per la gestione e il monitoraggio del cluster di broker. In particolare, Zookeeper tiene

traccia dello stato del cluster, coordina la gestione delle partizioni dei topic e gestisce il bilanciamento del carico tra i broker. È fondamentale per garantire che Kafka possa essere scalato in modo sicuro e che i dati siano sempre accessibili anche in caso di guasti.

5. Topic

Un topic è una categoria logica alla quale i producer pubblicano messaggi e i consumer si iscrivono per riceverli. I topic sono partizionati in modo che i messaggi possano essere distribuiti tra più broker e letti in parallelo dai consumatori. Ogni messaggio all'interno di un topic ha un offset, un identificatore univoco che indica la posizione del messaggio all'interno della partizione. I topic sono una parte fondamentale dell'architettura di Kafka, poiché forniscono una struttura per organizzare e indirizzare i flussi di dati.

6. Partizioni

Ogni topic in Kafka è suddiviso in più partizioni. Una partizione è una sequenza di messaggi ordinati che possono essere distribuiti tra più broker. Ogni messaggio in una partizione ha un offset che lo rende univoco. Le partizioni consentono a Kafka di gestire il parallelismo e la scalabilità, poiché ogni partizione può essere letta e scritta indipendentemente dalle altre. Le partizioni sono fondamentali per il bilanciamento del carico e la distribuzione dei messaggi tra i broker.

7. Replica

Kafka supporta la replicazione dei dati per garantire l'affidabilità e la tolleranza ai guasti. Ogni partizione di un topic può essere replicata su più broker. In caso di guasto di un broker, una delle repliche può essere utilizzata per continuare a leggere i dati senza interruzioni. La replicazione aiuta a garantire la durabilità dei messaggi anche in caso di guasti hardware o di rete.

8. Log

Kafka memorizza i messaggi come voci in un log immutabile, che è una sequenza di messaggi che viene scritta solo una volta. Questo design consente a Kafka di gestire enormi quantità di dati in modo efficiente, permettendo di scrivere e leggere i messaggi senza dover eseguire operazioni costose come l'aggiornamento dei dati. I messaggi rimangono nel log fino a quando non vengono eliminati esplicitamente, permettendo ai consumatori di rileggere e riprocessare i dati.

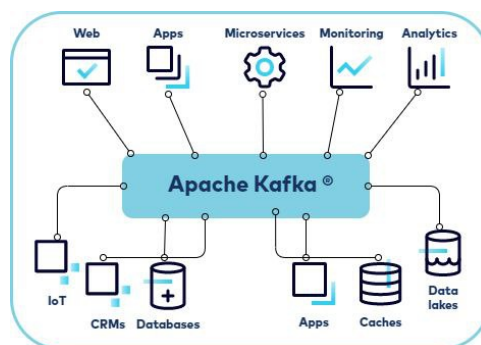


Figura 13: Kafka

4.7.2 Flusso di Dati

Il flusso di dati in Apache Kafka è progettato per garantire l'elaborazione di eventi in tempo reale in modo distribuito, affidabile e scalabile. Kafka agisce come un sistema di messaggistica in cui i produttori inviano dati (messaggi) e i consumatori li ricevono per elaborarli, mantenendo la capacità di scalare orizzontalmente. Il flusso di dati in Kafka può essere descritto in modo dettagliato attraverso diverse fasi che coinvolgono i componenti principali del sistema.

1. Producer invia messaggi

Il Producer è l'applicazione che invia i dati (messaggi) a un topic in Kafka. Un topic è una categoria logica alla quale i messaggi vengono pubblicati. Quando un producer crea un messaggio, Kafka lo invia a una delle partizioni di un topic. I producer possono inviare i messaggi in modo sincrono o asincrono, a seconda della configurazione. Kafka distribuisce i messaggi tra le partizioni in modo che i consumatori possano accedervi in parallelo. Ogni messaggio che il producer invia a Kafka include informazioni come:

- **Chiave:** opzionale, utilizzata per determinare quale partizione riceverà il messaggio. Se non viene fornita una chiave, Kafka distribuirà i messaggi in modo casuale tra le partizioni.
- **Valore:** i dati veri e propri che il producer invia (possono essere in qualsiasi formato, come JSON, Avro, o String).
- **Timestamp:** il momento in cui il messaggio è stato inviato.
- **Header:** metadati opzionali associati al messaggio.

2. Broker riceve e memorizza i messaggi

I Broker sono i server che ricevono i messaggi inviati dai producer. Ogni broker è responsabile di una o più partizioni di uno o più topic. Quando un broker riceve un messaggio, lo memorizza nel log del topic, ordinato per il offset, un identificatore univoco che rappresenta la posizione del messaggio nella partizione. Kafka memorizza i messaggi in un log immutabile, il che significa che una volta che i messaggi sono scritti nel log, non possono essere modificati. I messaggi rimangono nel log fino a quando non vengono eliminati in base alla configurazione (ad esempio, dopo un determinato periodo di tempo o una quantità specifica di dati). Ogni partizione di un topic è replicata su più broker per garantire l'affidabilità e la tolleranza ai guasti. Se un broker si guasta, un'altra replica della partizione può prendere il suo posto, mantenendo la disponibilità dei dati.

3. Consumer legge i messaggi

Un Consumer è un'applicazione che legge i messaggi da una o più partizioni di un topic. I consumatori si iscrivono ai topic di interesse e iniziano a leggere i messaggi. Kafka consente ai consumatori di far parte di gruppi di consumatori. Ogni gruppo di consumatori può leggere i messaggi di un topic in parallelo, con ogni consumer che legge una partizione del topic, garantendo un'alta disponibilità e bilanciamento del carico. Un'importante caratteristica di Kafka è che i consumatori possono leggere i messaggi in qualsiasi momento, anche in retrospettiva, in base all'offset. I consumatori memorizzano l'offset di ciascun messaggio che leggono,

consentendo loro di riprendere la lettura dal punto in cui si erano fermati, anche se si verificano guasti o riavvii.

4. Sincronizzazione tra i componenti

Nel flusso di dati di Kafka, la sincronizzazione tra producer, broker e consumer avviene tramite i topic e le partizioni. Ogni messaggio che viene inviato da un producer è distribuito tra le partizioni di un topic, dove ogni consumer in un gruppo può leggere una partizione. Il meccanismo di partizionamento permette a Kafka di distribuire i carichi di lavoro in modo bilanciato e parallelizzato, mentre la replica delle partizioni garantisce la disponibilità e la durabilità dei messaggi. Inoltre, Kafka utilizza Zookeeper per coordinare lo stato del cluster di broker, gestire la replica e monitorare i consumatori e i producer. Zookeeper assicura che i componenti del sistema possano comunicare tra loro in modo sicuro e che il flusso di dati possa proseguire senza interruzioni, anche in caso di guasti di uno o più broker.

5. Ordine dei messaggi

All'interno di una singola partizione, Kafka garantisce che i messaggi siano letti nell'ordine in cui sono stati scritti. Ogni messaggio in una partizione ha un offset, che è un identificatore numerico che rappresenta la sua posizione nel log della partizione. I consumatori possono utilizzare questi offset per leggere i messaggi in ordine, e Kafka mantiene l'integrità dell'ordine dei messaggi, permettendo ai consumatori di processarli nel giusto ordine. Tuttavia, è importante notare che Kafka non garantisce l'ordine tra le partizioni. Se un topic ha più partizioni, i messaggi inviati a ciascuna partizione possono essere letti in ordine, ma non c'è garanzia che i messaggi tra le diverse partizioni siano processati nell'ordine in cui sono stati inviati dal producer.

6. Elaborazione dei dati in tempo reale

Kafka è progettato per elaborare i dati in tempo reale. I messaggi vengono immediatamente inviati ai broker e resi disponibili ai consumatori non appena sono scritti nel log. I consumatori possono quindi elaborare i dati in tempo reale, utilizzando i messaggi per aggiornare sistemi di analisi, monitoraggio o prendere decisioni in tempo reale. Grazie alla sua architettura distribuita, Kafka è in grado di gestire flussi di dati ad alta velocità, come eventi di clic su un sito web, dati di sensori, transazioni finanziarie e altre applicazioni in tempo reale.

Il flusso di dati in Apache Kafka è progettato per essere estremamente scalabile, resiliente e performante. Il processo di invio, memorizzazione e lettura dei messaggi tra i componenti di Kafka (producer, broker e consumer) è altamente distribuito, consentendo la gestione di grandi volumi di dati in tempo reale. La replica, la gestione dei topic e delle partizioni, nonché il coordinamento tramite Zookeeper, sono essenziali per garantire che i dati siano sempre disponibili, affidabili e pronti per essere elaborati dai consumatori in tempo reale.

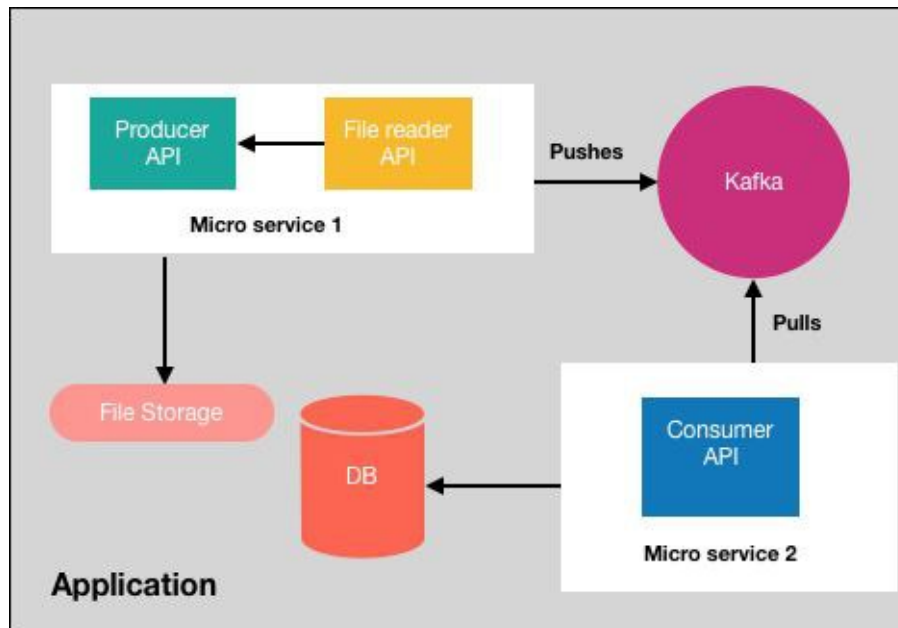


Figura 14: Kafka: Sistema Completo

4.7.3 Vantaggi e Svantaggi

I vantaggi di Kafka sono numerosi e lo rendono una delle soluzioni più robuste e scalabili per la gestione di flussi di dati in tempo reale. Grazie alla sua architettura distribuita, Kafka è in grado di gestire enormi volumi di messaggi con alta disponibilità, tolleranza ai guasti e scalabilità orizzontale. Questi punti di forza lo rendono ideale per applicazioni che necessitano di elaborazione di dati in tempo reale, come sistemi di monitoraggio, analisi, e applicazioni distribuite.

- **Alta disponibilità e tolleranza ai guasti:** Kafka è progettato per essere altamente disponibile, grazie alla replicazione delle partizioni su più broker. Se un broker si guasta, un altro broker che contiene una replica della partizione prende il suo posto, garantendo che i dati siano sempre disponibili e il sistema continui a funzionare senza interruzioni.
- **Scalabilità orizzontale:** Kafka è estremamente scalabile. Poiché è possibile aggiungere nuovi broker al cluster senza interruzioni, Kafka è in grado di gestire una quantità crescente di dati e di richieste, soddisfacendo le esigenze di applicazioni con alti volumi di traffico.
- **Elaborazione dei dati in tempo reale:** Kafka è progettato per il trattamento dei flussi di dati in tempo reale, consentendo alle applicazioni di ricevere, elaborare e agire sui dati immediatamente dopo che vengono prodotti. Questo lo rende ideale per scenari come l'elaborazione di log, eventi di monitoraggio, analisi in tempo reale e tracking di clic.
- **Durabilità dei dati:** Kafka memorizza i messaggi in modo immutabile e su disco, garantendo che i dati siano conservati per un lungo periodo di tempo. Può gestire enormi volumi di dati senza compromettere la durabilità, e offre opzioni di retention per eliminare i messaggi vecchi dopo un certo periodo di tempo.

- **Bassa latenza:** Kafka è progettato per essere veloce e a bassa latenza, in modo che i messaggi possano essere trasmessi e letti con tempi minimi di attesa. Questo è importante quando si gestiscono flussi di dati in tempo reale, dove ogni millisecondo può fare la differenza.
- **Supporto per diversi modelli di consumo:** Kafka offre diversi modelli di consumo, come il consumo da parte di singoli consumatori, gruppi di consumatori e la possibilità di riprendere la lettura dai messaggi precedenti (attraverso gli offset). Ciò consente una grande flessibilità nelle applicazioni che necessitano di consumare e processare i dati in modo diverso.
- **Integrazione con altri sistemi:** Kafka si integra facilmente con numerosi strumenti di analisi e di elaborazione dei dati, come Apache Spark, Apache Flink, Hadoop, e sistemi di data warehouse, facilitando l'elaborazione e l'analisi di grandi volumi di dati.
- **Supporto per stream processing:** Kafka può essere utilizzato insieme a Kafka Streams, una libreria di stream processing che permette di scrivere applicazioni che elaborano flussi di dati in tempo reale direttamente dentro Kafka, senza la necessità di un sistema esterno.

Nonostante i numerosi vantaggi, Kafka presenta anche alcuni svantaggi. La sua complessità di configurazione e gestione può risultare un ostacolo per alcune organizzazioni, soprattutto per chi non ha esperienza con sistemi distribuiti. Inoltre, la necessità di risorse hardware dedicate per ottenere prestazioni ottimali può rappresentare una sfida in ambienti a bassa capacità. Infine, per applicazioni più semplici o con requisiti meno esigenti, Kafka potrebbe risultare sovradimensionato rispetto a soluzioni più leggere.

- **Complessità nella gestione:** La gestione di un cluster Kafka richiede una certa esperienza e competenza. L'amministrazione e la configurazione di Kafka, specialmente in ambienti di produzione su larga scala, possono essere complesse. La gestione di broker, topic, partizioni e repliche, nonché il monitoraggio delle prestazioni, richiede attenzione costante.
- **Consumo di risorse:** Sebbene Kafka sia scalabile, la sua gestione e operatività su larga scala possono comportare un notevole consumo di risorse, inclusi spazio su disco e memoria. La necessità di memorizzare grandi volumi di dati può aumentare notevolmente i costi operativi.
- **Gestione delle transazioni:** Kafka è principalmente progettato per l'elaborazione di messaggi in modo asincrono e non offre una gestione avanzata delle transazioni come altri sistemi di messaggistica. Sebbene Kafka offra una certa forma di garanzia di consegna, la gestione di transazioni complesse tra più messaggi potrebbe richiedere un ulteriore livello di logica applicativa.
- **Dipendenza da Zookeeper:** Kafka dipende da Apache Zookeeper per il coordinamento del cluster e la gestione delle sue operazioni. La gestione di Zookeeper può aggiungere complessità al sistema e, in alcune situazioni, Zookeeper può diventare un punto di debolezza se non configurato correttamente.

- **Ordine dei messaggi limitato:** Kafka garantisce l'ordine dei messaggi solo all'interno di una singola partizione. Se un topic ha più partizioni, i messaggi inviati in ordine da un producer non vengono garantiti per essere letti nell'ordine da un consumatore che legge più partizioni contemporaneamente.
- **Overhead di configurazione per piccole applicazioni:** Sebbene Kafka sia ideale per grandi sistemi distribuiti e applicazioni con carichi di dati elevati, per piccole applicazioni o sistemi a bassa intensità di dati, la configurazione e la gestione di Kafka possono risultare eccessive. In questi casi, soluzioni più semplici come RabbitMQ o altre piattaforme di messaggistica potrebbero essere più appropriate.
- **Non progettato per la bassa latenza su singole operazioni:** Kafka è progettato per la gestione di flussi di dati su larga scala. Se la tua applicazione richiede una bassa latenza per ogni singolo messaggio o una gestione più immediata delle operazioni, Kafka potrebbe non essere la soluzione migliore, poiché la latenza può aumentare in scenari di elevato traffico.
- **Problemi con la gestione del consumo parallelo dei messaggi:** Sebbene Kafka consenta ai consumatori di leggere i dati in parallelo, la gestione di come i messaggi vengono distribuiti tra i consumatori di un gruppo può risultare complessa, soprattutto quando si cerca di ottimizzare il bilanciamento del carico o la gestione delle partizioni.

In generale, Kafka è uno strumento potente e altamente scalabile per la gestione dei flussi di dati in tempo reale, ma richiede una buona comprensione delle sue caratteristiche e dei suoi limiti per essere utilizzato al meglio.

5 Descrizione del Codice

5.0.1 `--init--.py`

Il codice definisce una parte di un'applicazione Flask, in cui vengono gestiti vari blueprint per le diverse funzionalità e un task scheduler per eseguire operazioni periodiche. Il modulo `'BackgroundScheduler'` di `APScheduler` viene utilizzato per avviare un task che può essere eseguito in background, in questo caso per aggiornare lo stato di apertura dei ristoranti ogni 5 minuti. La funzione `'update_restaurant_open_status'`, importata dalla cartella `'utils.trigger'`, è quella che esegue l'aggiornamento dello stato dei ristoranti, probabilmente per indicare se sono aperti o chiusi. La funzione `'create_app'` è il cuore della configurazione dell'applicazione Flask. All'interno di essa, viene configurato il database tramite `'db.init_app(app)'` e la gestione delle migrazioni tramite `'migrate.init_app(app, db)'`. Inoltre, viene inizializzato il supporto per la comunicazione in tempo reale con `'socketio.init_app(app)'`. I vari blueprint, come quelli per la gestione della home, dei ristoranti, dell'autenticazione, dei clienti, delle consegne e degli amministratori, vengono registrati con un URL prefix specifico per ciascuno, in modo da organizzare meglio le diverse funzionalità. Il task scheduler viene avviato dalla funzione `'start_scheduler'`, che aggiunge il job `'update_restaurant_open_status'` per essere eseguito periodicamente ogni 5 minuti. Alla fine, la funzione `'create_app'` restituisce l'oggetto `'app'`, `'socketio'` e `'orders'`, che probabilmente è un oggetto o una variabile globale per la gestione degli ordini. Il comportamento generale del codice è quello di configurare un'applicazione Flask, inizializzare le componenti necessarie come il database e la comunicazione in tempo reale, e avviare un task periodico per l'aggiornamento dello stato dei ristoranti. La struttura della funzione `'create_app'` segue un approccio tipico in Flask, dove le diverse funzionalità vengono modularizzate tramite blueprint.

5.0.2 `extensions.py`

In questo codice viene definita l'inizializzazione di un'applicazione Flask con il supporto per SQLAlchemy, SocketIO e una lista di ordini. Viene prima creato un oggetto `'Flask'` chiamato `'app'`, che è l'istanza principale dell'applicazione. Successivamente, `'SQLAlchemy'` viene inizializzato con `'db = SQLAlchemy()'`, ma non è associato direttamente all'applicazione, quindi presumibilmente sarà configurato in seguito (ad esempio, nel file di configurazione dell'app o in una funzione di inizializzazione separata). `'socketio = SocketIO(app)'` inizializza il supporto per la comunicazione in tempo reale tramite WebSockets, creando un oggetto `'SocketIO'` legato all'app `'app'`. Infine, viene definita una lista vuota `'orders'`, che potrebbe essere utilizzata per tenere traccia degli ordini, anche se in questo caso non viene ancora popolata o gestita in modo esplicito.

5.0.3 config.py

In questo codice viene definita una classe `Config` che contiene le configurazioni per l'applicazione Flask. La configurazione `SQLALCHEMY_DATABASE_URI` specifica l'URI del database da usare con SQLAlchemy. In questo caso, viene utilizzato un database SQLite locale chiamato `food_delivery.db`. La configurazione `SECRET_KEY` è una chiave segreta che Flask utilizza per proteggere le sessioni utente e i dati delle richieste. Viene generata dinamicamente con `os.urandom(24)`, che crea una sequenza casuale di 24 byte. `SQLALCHEMY_TRACK_MODIFICATIONS` è impostato su `False` per disabilitare il tracking delle modifiche agli oggetti SQLAlchemy. Questo aiuta a migliorare le prestazioni, poiché non viene mantenuta la storia delle modifiche a ogni oggetto dopo il commit. La configurazione `DEBUG` e `FLASK_DEBUG` sono impostate su `True` e `1` rispettivamente, il che abilita la modalità di debug di Flask. In questa modalità, l'applicazione mostrerà eventuali errori direttamente nel browser e ricaricherà automaticamente il server quando vengono effettuati cambiamenti al codice. Infine, `LOGGING_LEVEL` è impostato su `"DEBUG"`, il che significa che l'applicazione registrerà i messaggi di log a livello di debug, mostrando informazioni dettagliate sui processi interni dell'app.

5.0.4 run.py

In questo codice viene avviata l'applicazione Flask insieme a SocketIO. La funzione `create_app` viene chiamata per inizializzare l'applicazione, ottenendo gli oggetti `app`, `socketio` e `orders`. Questo probabilmente è il punto di partenza per la configurazione completa dell'app, come definito in precedenza. Viene poi definito un semplice endpoint `/test` che, quando visitato, restituisce un messaggio di saluto. All'interno della funzione di questo endpoint, vengono utilizzate due funzioni del modulo `threading` per ottenere informazioni sul thread corrente e sul numero totale di thread attivi. `threading.get_ident()` restituisce l'ID del thread corrente, mentre `threading.active_count()` restituisce il numero di thread attualmente attivi. Questi valori vengono poi inclusi nella risposta della route. Infine, nella parte sotto `if __name__ == '__main__':`, viene avviata l'app Flask utilizzando `socketio.run(app, debug=True, threaded=True)`. Questo avvia l'app in modalità debug e consente l'elaborazione concorrente tramite il parametro `threaded=True`. Con `threaded=True`, l'app Flask sarà in grado di gestire più richieste contemporaneamente utilizzando thread separati. Inoltre, il supporto per SocketIO è incluso, il che permette la gestione della comunicazione in tempo reale.

5.1 Templates

5.1.1 admin_dashboard.html

Il codice HTML definisce una pagina di dashboard per l'amministratore di un sistema, utilizzando il framework Bootstrap per la progettazione della UI. La struttura include una sezione '`< head >`', in cui sono definiti il set di caratteri (UTF-8) e il meta tag per la responsività, oltre al link al CSS di Bootstrap per stilizzare la pagina. Nel corpo della pagina ('`< body >`'), è presente una 'container' con il titolo "Dashboard Amministratore" e un messaggio di benvenuto personalizzato che include il nome dell'utente, presumibilmente passato dal backend tramite il template engine di Flask (Jinja2) con la sintassi '`user.name`'. Segue una sezione con due opzioni principali per l'amministratore:

- **Gestione Account:** Un bottone che reindirizza alla pagina di gestione degli account tramite il link '`url_for('admin.manage.accounts')`'. '`url_for`' è una funzione di Flask che genera l'URL per una route specificata.
- **Monitoraggio Performance e Statistiche:** Un altro bottone che porta alla pagina di monitoraggio delle performance

Infine, è presente un form per il logout, che invia una richiesta POST alla route '`auth.logout`' quando l'amministratore clicca sul bottone "Logout". La pagina è progettata per essere semplice, funzionale e responsive grazie all'uso di Bootstrap, con la possibilità di navigare facilmente tra le opzioni di amministrazione e un'uscita sicura dal sistema.

5.1.2 admin_manage_accounts.html

Questo codice HTML rappresenta la pagina di gestione degli account per l'amministratore di un'applicazione. Viene utilizzato il framework Bootstrap per stilizzare la pagina e organizzare i contenuti in modo responsivo e user-friendly. Nel dettaglio:

- **Struttura e stilizzazione:** La pagina include una sezione '`< head >`', dove viene definito il charset, la configurazione per la visualizzazione su dispositivi mobili, e il titolo della pagina "Gestione Account". Inoltre, viene incluso il foglio di stile di Bootstrap per la formattazione della pagina.
- **Corpo della pagina:**
 - Il titolo principale della pagina è "Gestione Account", centrato.
 - Un paragrafo di descrizione informa l'amministratore che questa sezione permette di eliminare gli account degli utenti.
 - Viene creata una tabella con intestazioni per visualizzare i dettagli degli utenti registrati, come ID, nome utente, nome, indirizzo, numero di telefono, email, ruolo e azioni.
 - All'interno del corpo della tabella, viene utilizzata una struttura di loop Jinja2 `for user in users` per iterare sugli utenti passati al template dal backend. Per ogni utente, vengono visualizzati i relativi dati nelle colonne e un bottone "Elimina" per ogni riga. Ogni bottone "Elimina" è associato all'ID dell'utente tramite l'attributo `data-user-id`, che viene utilizzato dal JavaScript per gestire l'eliminazione.

- **Bottoni e azioni:**

- Un bottone "Torna alla Dashboard" permette all'amministratore di ritornare alla dashboard principale.
- Un form con un bottone "Logout" permette all'amministratore di uscire dalla sessione.
- Infine, è incluso un collegamento a un file JavaScript (`admin_manage_accounts.js`), per gestire la logica di eliminazione degli utenti, tramite l'interazione con i bottoni "Elimina".

- **Funzionalità dinamica:**

- Il codice usa Jinja2 per visualizzare dinamicamente i dati degli utenti dalla variabile `users`, che si presume venga passata al template dal backend.
- La gestione della logica di eliminazione degli utenti dovrebbe essere gestita nel file JavaScript specificato, dove viene catturato l'evento di click sui bottoni "Elimina" e inviata una richiesta al server per eliminare l'utente corrispondente.

Questo design fornisce un'interfaccia chiara per gli amministratori per visualizzare, gestire e eliminare gli account degli utenti, con una navigazione facile verso la dashboard e la possibilità di effettuare il logout.

5.1.3 `admin_monitor_performance.html`

Questo codice HTML definisce una pagina per il monitoraggio delle performance, in particolare per visualizzare statistiche relative alle consegne e agli ordini, insieme a un grafico che mostra il numero di ordini per ristorante. Viene utilizzato il framework Bootstrap per la struttura della pagina e Chart.js per visualizzare il grafico. Nel dettaglio:

- **Struttura e stilizzazione:**

- Nella sezione `< head >`, vengono definiti il charset, il meta tag per la responsività, il titolo della pagina "Monitoraggio Performance", e i collegamenti per Bootstrap e la libreria JavaScript di Chart.js.

- **Corpo della pagina:**

- Il titolo principale è "Monitoraggio Performance", centrato.
- La pagina include due card che mostrano le statistiche generali:
 - * Una per il numero totale di consegne completate (`total_completed_deliveries`), passata tramite il template engine di Flask (Jinja2).
 - * Una per il numero totale di ordini (`total_orders`), anch'essa passata dinamicamente tramite Jinja2.
- Viene quindi presentato un altro titolo per il grafico: "Ordini per Ristorante". Il grafico è rappresentato da un elemento `< canvas >` con id `ordersChart`, che sarà popolato dal codice JavaScript.

- **Bottoni e azioni:**

- Un bottone "Torna alla Dashboard" permette di tornare alla dashboard principale.
- Un form di logout permette all'amministratore di uscire dalla sessione.

- **Passaggio di variabili a JavaScript:**

- Le variabili `restaurant_names` e `order_counts` vengono definite e passate dal backend (in formato JSON) al JavaScript. Queste variabili rappresentano i nomi dei ristoranti e il numero totale di ordini per ciascun ristorante. Le variabili sono ottenute utilizzando il filtro `map` di Jinja2 per estrarre gli attributi `name` e `total_orders` dalla lista `restaurant_orders`, che è una lista di oggetti che contengono queste informazioni.

- **Grafico con Chart.js:**

- Il file JavaScript esterno (`admin_monitor_performance.js`) è utilizzato per inizializzare e configurare il grafico. I dati passati tramite le variabili JavaScript vengono usati per costruire un grafico che mostrerà il numero di ordini per ogni ristorante.

- **Interazione dinamica:**

- Il grafico dinamico sarà popolato con i dati passati al template, permettendo all'amministratore di visualizzare facilmente le performance degli ordini per ristorante.

In sintesi, questa pagina fornisce una panoramica delle performance del sistema, mostrando statistiche generali sulle consegne e sugli ordini, insieme a un grafico che dettaglia gli ordini per ogni ristorante.

5.1.4 `customer_browse_restaurants.html`

Questo codice HTML definisce una pagina che permette agli utenti di sfogliare i ristoranti e visualizzare i loro menu. Viene utilizzato il framework Bootstrap per una struttura responsiva e ben stilizzata. Di seguito viene descritta la funzionalità e la struttura del codice:

- **Struttura della pagina:**

- La pagina include una navbar scura in cima con il nome "Food Delivery".
- Sotto la navbar, viene presentato il titolo "Sfoggia Ristoranti e Menu", centrato.

- **Gestione dei messaggi flash:**

- I messaggi flash (tipicamente utilizzati per notifiche come successi o errori) vengono visualizzati dinamicamente nel caso siano stati passati dal backend. Se ci sono messaggi, vengono mostrati in una lista con classi di stile (ad esempio, "alert-success" o "alert-danger").

- **Form di ricerca:**

- Un modulo di ricerca permette all'utente di cercare i ristoranti per nome. L'input viene inviato tramite un POST al server, che eseguirà la ricerca dei ristoranti in base alla query fornita. Il valore di `user.id` viene passato tramite la `url_for` per includerlo nel percorso.

- **Pulsante di ricerca del cibo:**

- Un pulsante permette di accedere alla ricerca di cibi specifici, indirizzando l'utente alla pagina per la ricerca dei piatti. Anche in questo caso, `user.id` viene passato al backend tramite l'URL.

- **Visualizzazione dei ristoranti:**

- I ristoranti vengono visualizzati in una griglia di carte (card) utilizzando Bootstrap. Per ogni ristorante, viene mostrato il nome e l'indirizzo. Ogni carta include anche un pulsante per visualizzare il menu del ristorante. Questo pulsante porta a una pagina dedicata al menu del ristorante, passando sia `user.id` che `restaurant.id` nell'URL.

- **Pulsante di ritorno alla dashboard:**

- Un pulsante permette di tornare alla dashboard del cliente, utilizzando un altro URL definito tramite `url_for`.

- **Bootstrap JS:**

- Il codice carica il file JavaScript di Bootstrap per gestire interattività come il menu a discesa o altre funzionalità dinamiche.

5.1.5 `customer_dashboard.html`

Questo codice HTML definisce la pagina della "Dashboard Cliente", che permette agli utenti di navigare attraverso diverse funzionalità della piattaforma di food delivery. Di seguito viene descritta la struttura e la logica del codice:

- **Navbar:**

- In cima alla pagina, c'è una barra di navigazione scura con il nome "Food Delivery" che serve come punto di partenza per la navigazione.

- **Benvenuto nella Dashboard:**

- Un'intestazione centrale dà il benvenuto all'utente nella sua dashboard personale.

- **Pulsanti delle Funzionalità:**

- La pagina è suddivisa in una griglia di pulsanti principali che permettono all'utente di accedere a diverse funzionalità:

- * **Visualizza Ristoranti e Menu:** Porta l'utente a una pagina per esplorare ristoranti e menù.
- * **Effettua e Personalizza Ordini:** Permette di effettuare e personalizzare ordini, probabilmente con opzioni per selezionare piatti e ristoranti.
- * **Traccia Ordini:** Consente di visualizzare lo stato attuale degli ordini in corso.
- * **Gestisci Profilo:** Permette all'utente di aggiornare i propri dati personali, come indirizzo, numero di telefono e preferenze.

- **Logout:**

- Un pulsante di logout è presente per consentire all'utente di uscire dalla propria sessione. Questo invia una richiesta POST al backend per chiudere la sessione dell'utente.

- **Responsività:**

- La pagina è progettata per essere responsive, adattandosi a schermi più piccoli grazie alla struttura basata su colonne di Bootstrap (ad esempio, col-md-6 col-lg-3), che ridistribuisce i pulsanti in modo ordinato su schermi di diverse dimensioni.

- **Caricamento di Bootstrap JS:**

- Il codice include il file JavaScript di Bootstrap per gestire eventuali interazioni dinamiche come dropdown, modali o altri componenti interattivi.

5.1.6 customer_place_order.html

Il codice HTML rappresenta una pagina per visualizzare il carrello di un cliente in una piattaforma di food delivery. Ecco una panoramica della struttura e delle funzionalità di questa pagina:

- **Navbar:**

- Una barra di navigazione in alto con il nome "Food Delivery", che può essere il punto di partenza per la navigazione su altre pagine della piattaforma.

- **Titolo della Pagina:**

- La pagina ha come titolo "Il Tuo Carrello", indicando che l'utente sta visualizzando gli articoli che ha aggiunto al carrello.

- **Messaggi Flash:**

- Viene gestita la visualizzazione di messaggi di feedback per l'utente, come conferme o errori (ad esempio, un messaggio che informa l'utente che il carrello è vuoto).

- **Tabella del Carrello:**

- Se ci sono piatti nel carrello (foods_in_cart), viene mostrata una tabella con:

- * Nome del Piatto
- * Descrizione
- * Prezzo Unitario
- * Quantità (un campo input di tipo "number" che permette di aggiornare la quantità di un piatto)
- * Prezzo Totale (calcolato come prezzo unitario * quantità)

- **Aggiornamento Quantità:**

- Un form permette di inviare una richiesta per aggiornare la quantità dei piatti nel carrello.

- **Conferma Ordine:**

- Se ci sono piatti nel carrello, un pulsante permette di inviare la richiesta per confermare l'ordine.

- **Messaggio per Carrello Vuoto:**

- Se il carrello è vuoto, viene mostrato un messaggio informativo.

- **Pulsanti di Navigazione:**

- **Torna al Menu:** Un pulsante che porta l'utente al menu del ristorante, se è stato selezionato un ristorante specifico.
 - **Sfoglia i Ristoranti:** Un pulsante che permette all'utente di tornare alla pagina per esplorare i ristoranti.
 - **Visualizza Ordini:** Un pulsante che porta alla pagina per visualizzare gli ordini effettuati.

- **Bootstrap JS:**

- Il codice include la libreria JavaScript di Bootstrap per gestire dinamicamente gli elementi della pagina.

5.1.7 `customer_search_foods.html`

Questo codice HTML crea una pagina di ricerca per piatti in un'applicazione di food delivery, utilizzando il framework Bootstrap per la stilizzazione. Ecco una panoramica delle funzionalità e struttura della pagina:

- **Navbar:**

- La pagina include una barra di navigazione scura con il titolo "Food Delivery" e un'area per la navigazione tra le sezioni dell'applicazione.

- **Titolo della Pagina:**

- La pagina ha come titolo "Cerca Piatto", indicato chiaramente nell'intestazione.

- **Form di Ricerca:**

- Un campo di input permette agli utenti di cercare un piatto specifico. Il form invia la query al server tramite un metodo POST per cercare i piatti nel database. L'input può essere una parte del nome del piatto o una parola chiave.

- **Elenco dei Piatti:**

- Se vengono trovati dei piatti corrispondenti alla ricerca, vengono visualizzati in una griglia di carte (card) responsive. Ogni carta include:
 - * **Nome del Piatto:** Visualizzato come un titolo.
 - * **Descrizione:** Una breve descrizione del piatto.
 - * **Prezzo:** Il prezzo unitario del piatto.
 - * **Pulsante "Aggiungi al Carrello":** Ogni piatto ha un pulsante che consente di aggiungere il piatto al carrello dell'utente. L'azione invia una richiesta POST per aggiungere il piatto selezionato al carrello.

- **Pulsante di Navigazione:**

- Un pulsante che consente all'utente di tornare alla pagina dei "Ristoranti e Menu", nel caso voglia esplorare altri ristoranti o menu disponibili.

- **Bootstrap JS:**

- La libreria JavaScript di Bootstrap è inclusa per gestire eventuali componenti dinamici (ad esempio, per l'espansione delle card o altre interazioni).

5.1.8 customer_track_orders.html

Questo codice HTML crea una pagina che permette agli utenti di tracciare i propri ordini, utilizzando la libreria Leaflet per visualizzare una mappa interattiva. Ecco una panoramica dettagliata:

- **Titolo della Pagina:**

- La pagina ha come titolo "Traccia Ordini", personalizzato con il nome dell'utente (`user.name`).

- **Sezione di Ordini:**

- Se ci sono ordini (`if orders`), vengono visualizzati in una lista con dettagli come:
 - * **ID Ordine:** Mostra il numero dell'ordine.
 - * **Status dell'Ordine:** Visualizza lo stato dell'ordine con un badge di colore differente.
 - * **Data dell'Ordine:** La data di creazione dell'ordine.
 - * **Totale dell'Ordine:** Il totale da pagare per l'ordine.

- * **Mappa di Tracciamento:** Una mappa che mostra la posizione dell'ordine, utilizzando la libreria Leaflet per tracciare il percorso. La mappa si popola con le coordinate di partenza (ristorante), la posizione attuale (rider) e la destinazione (cliente).

- **Sezione di Nessun Ordine:**

- Se non ci sono ordini da tracciare, viene mostrato un messaggio di avviso con il testo "Non hai ordini tracciabili al momento."

- **Navigazione:**

- Un pulsante che permette all'utente di tornare alla propria dashboard.
- Un pulsante di logout per uscire dal sistema.

- **Integrazione con Leaflet:**

- Le mappe sono caricate tramite la libreria Leaflet, che visualizza la posizione geografica dei vari ordini.
- Ogni ordine ha una mappa dedicata, che verrà aggiornata dinamicamente usando i dati di latitudine e longitudine passati tramite i dati dell'ordine.

- **Dati degli Ordini:**

- I dati degli ordini vengono passati nel formato JSON tramite il tag `<script type="application/json" id="orders-data">`, che permette di utilizzare questi dati nel codice JavaScript per popolare le mappe.

- **Bootstrap JS e Leaflet JS:**

- Bootstrap viene utilizzato per la gestione dell'interfaccia utente, inclusi gli stili per le carte e i pulsanti.
- Leaflet viene utilizzato per la creazione delle mappe, mentre le coordinate geografiche degli ordini sono utilizzate per tracciare il percorso sulla mappa.

- **JavaScript Personalizzato:**

- Un file JavaScript esterno (`customer_track_orders.js`) viene caricato per gestire il comportamento dinamico della pagina (ad esempio, la visualizzazione delle mappe).
- Viene eseguito un log dei dati degli ordini per il debug, utilizzando `console.log`.

5.1.9 customer_view_menu.html

Questo codice HTML crea una pagina che mostra il menu di un ristorante, utilizzando Bootstrap per il layout e la presentazione. Ecco una panoramica delle sezioni principali:

- **Titolo della Pagina:**

- La pagina ha come titolo dinamico "Menu di restaurant.name", dove restaurant.name viene sostituito con il nome del ristorante.

- **Navbar:**

- Una barra di navigazione in alto che consente di navigare facilmente nel sito. Ha un titolo "Food Delivery" e si mantiene sempre visibile nella parte superiore della pagina.

- **Visualizzazione del Menu:**

- Il menu del ristorante viene presentato sotto forma di card per ogni piatto. Ogni piatto ha:
 - * **Nome del Piatto:** Visualizzato come titolo della card.
 - * **Descrizione:** Una breve descrizione del piatto.
 - * **Prezzo:** Il costo del piatto.
 - * Un pulsante "**Aggiungi al Carrello**" che, al clic, aggiunge il piatto selezionato al carrello dell'utente (grazie alla funzione `url_for` che permette di inviare l'azione al backend).

Ogni piatto è visualizzato in una colonna di una griglia di Bootstrap (3 piatti per riga su schermi medi o più grandi).

- **Pulsante di Navigazione:**

- Un pulsante per tornare alla lista dei ristoranti, che utilizza `url_for` per indirizzare l'utente alla pagina di navigazione dei ristoranti.

- **Bootstrap JS:**

- Viene incluso il file JavaScript di Bootstrap per gestire dinamicamente il comportamento dell'interfaccia, come la visualizzazione dei menu a tendina, modali, e altre interazioni.

5.1.10 `customer_view_order_details.html`

Questo codice HTML mostra i dettagli di un ordine effettuato dall'utente su una piattaforma di food delivery. Ecco una panoramica delle sezioni principali della pagina:

- **Titolo della Pagina:**

- Il titolo della pagina è dinamico e visualizza "Dettagli Ordine", indicando che l'utente sta visualizzando i dettagli di un ordine specifico.

- **Navbar:**

- Una barra di navigazione in alto con il nome "Food Delivery", che funge da punto di riferimento per l'applicazione.

- **Dettagli dell'Ordine:**

- Una card che mostra i dettagli principali dell'ordine:
 - * **ID Ordine:** Un identificativo unico per l'ordine.
 - * **Data dell'Ordine:** La data in cui è stato effettuato l'ordine.

- * **Ristorante:** Il nome del ristorante dove è stato effettuato l'ordine.
- * **Stato dell'Ordine:** Lo stato attuale dell'ordine (ad esempio, "In preparazione", "In consegna", ecc.).
- * **Totale dell'Ordine:** Il costo totale dell'ordine.

- **Dettaglio dei Piatti Ordinati:**

- Una lista che mostra i piatti ordinati, con i seguenti dettagli per ogni piatto:
 - * Nome del piatto.
 - * Quantità ordinata.
 - * Prezzo unitario del piatto.
 - * Ogni piatto viene visualizzato come un elemento in una lista (ul), con ciascun piatto come un elemento della lista (li).

- **Pulsante di Navigazione:**

- Un pulsante che permette all'utente di tornare alla lista degli ordini, indirizzandolo alla vista degli ordini.

5.1.11 customer_view_orders.html

Questa pagina HTML permette agli utenti di visualizzare una lista dei propri ordini effettuati sulla piattaforma di food delivery. Ecco una panoramica delle sezioni principali e funzionalità:

- **Navbar:**

- Una barra di navigazione in cima alla pagina con il nome "Food Delivery", che serve come punto di riferimento per la piattaforma.

- **Intestazione:**

- Il titolo della pagina è dinamico e visualizza "I tuoi Ordini", indicando che l'utente sta visualizzando i propri ordini effettuati.

- **Tabella Ordini:**

- Se ci sono ordini, viene mostrata una tabella che elenca:
 - * **Data Ordine:** La data in cui è stato effettuato l'ordine.
 - * **Ristorante:** Il nome del ristorante dove è stato effettuato l'ordine.
 - * **Totale:** Il costo totale dell'ordine.
 - * **Stato:** Lo stato attuale dell'ordine (ad esempio, "In preparazione", "Completato", ecc.).
 - * **Dettagli:** Un pulsante che porta l'utente alla pagina dei dettagli dell'ordine, dove possono vedere maggiori informazioni.

- **Messaggio di Avviso:**

- Se non ci sono ordini, viene visualizzato un messaggio che informa l'utente che non ha ancora effettuato ordini.

- **Pulsanti di Navigazione:**

- Un pulsante che riporta l'utente al carrello per completare un ordine.
- Un altro pulsante che riporta l'utente alla dashboard.

5.1.12 customer_view_profile.html

Questa pagina HTML consente agli utenti di gestire e aggiornare il proprio profilo all'interno della piattaforma di food delivery. Ecco una panoramica delle sezioni e delle funzionalità principali:

- **Navbar:**

- La navbar include il nome della piattaforma "Food Delivery", rendendo facile per l'utente identificare su quale sito si trova.

- **Intestazione:**

- Un titolo centrale che indica "Gestisci il tuo Profilo", per informare l'utente che sta modificando le proprie informazioni personali.

- **Messaggi di Successo o Errore:**

- La pagina include la logica per mostrare messaggi di successo o errore utilizzando la funzione `get_flashed_messages`. Questi messaggi appaiono in una finestra di alert per informare l'utente, ad esempio, se l'aggiornamento del profilo è riuscito o se ci sono stati errori.

- **Form di Modifica Profilo:**

- **Username:** Un campo per aggiornare il nome utente.
- **Nome:** Un campo per il nome reale dell'utente.
- **Email:** Un campo opzionale per modificare l'email dell'utente.
- **Numero di Telefono:** Un campo opzionale per aggiungere o modificare il numero di telefono.
- **Indirizzo:** Un campo per aggiornare l'indirizzo dell'utente.
- Ogni campo è precompilato con le informazioni esistenti dell'utente (ad esempio, `user.username`, `user.email`, ecc.).

- **Pulsante di Aggiornamento:**

- Un pulsante di invio che consente di inviare le modifiche al backend per aggiornare il profilo dell'utente.

- **Pulsanti di Navigazione:**

- Un pulsante che permette di tornare alla dashboard del cliente.
- Un altro pulsante che consente all'utente di fare il logout dalla piattaforma.

5.1.13 home.html

Questa pagina HTML consente all'utente di selezionare un ruolo dalla piattaforma di food delivery e proseguire con il relativo processo. Ecco una panoramica delle sue funzionalità principali:

- **Header:**
 - Un'intestazione che accoglie l'utente con il messaggio "Benvenuto alla Piattaforma di Food Delivery!" su uno sfondo scuro con testo bianco.
- **Form di Selezione Ruolo:**
 - Un modulo che permette all'utente di selezionare uno dei seguenti ruoli:
 - * Ristorante
 - * Rider
 - * Cliente
 - * Amministratore
 - L'utente può selezionare il proprio ruolo tramite un select dropdown.
 - Il campo è obbligatorio e l'utente non può inviare il modulo senza aver scelto un ruolo.
- **Campo per l'ID del Ristorante:**
 - Se l'utente seleziona il ruolo di Ristorante, viene visualizzato un campo di input per inserire l'ID del ristorante.
 - Questo campo è nascosto di default e viene mostrato solo se "Ristorante" è selezionato dal dropdown.
- **Pulsante "Continua":**
 - Un pulsante che invia il modulo e consente di proseguire con la selezione del ruolo e l'eventuale inserimento dell'ID del ristorante.
- **JavaScript:**
 - Un file JavaScript (home.js) è incluso per gestire l'interattività, ad esempio, per mostrare o nascondere il campo dell'ID del ristorante in base alla selezione del ruolo.

5.1.14 login.html

La pagina di login ha un'interfaccia semplice e funzionale, con l'uso di Bootstrap per garantire una buona esperienza utente. Ecco una panoramica delle sue caratteristiche principali:

- **Header:**
 - Un'intestazione con sfondo scuro che accoglie l'utente con il messaggio "Benvenuto nella Piattaforma di Food Delivery". Il testo è bianco per contrastare con lo sfondo.

- **Form di Login:**

- Il modulo di login è centrato nella pagina ed è contenuto all'interno di un pannello bianco con bordi arrotondati e un'ombra leggera (rounded shadow-sm).
- Il modulo ha due campi principali:
 - * **Nome utente:** un campo di input per il nome utente.
 - * **Password:** un campo di input per la password.
- È presente anche un campo nascosto (hidden input) per passare l'ID del ristorante se è presente nella query string della URL.

- **Messaggi di errore o successo:**

- Se ci sono messaggi di errore (per esempio, se l'utente inserisce credenziali sbagliate), vengono visualizzati sopra il modulo in un alert con il tipo alert-warning. Questi messaggi sono gestiti utilizzando la funzione `get_flashed_messages()` di Flask, che consente di mostrare messaggi di sistema temporanei.

- **Pulsante di login:**

- Il pulsante "Login" è largo quanto il contenitore e di colore primario (btn-primary), per essere facilmente visibile e facilmente cliccabile.

- **Link per la registrazione:**

- Sotto il modulo c'è un link per la pagina di registrazione, per chi non è ancora registrato sulla piattaforma. Questo link è stilizzato come un "btn btn-link", che lo rende visibile ma non invadente.

5.1.15 register.html

La pagina di registrazione è ben strutturata e fornisce un'interfaccia user-friendly per l'utente. Ecco un'analisi e alcune considerazioni:

- **Header:**

- Un'intestazione semplice e pulita, con sfondo scuro e testo bianco, che accoglie l'utente con il messaggio "Registrati alla Piattaforma di Food Delivery".

- **Messaggi Flash:**

- Se sono presenti messaggi flash (ad esempio, errori o conferme), vengono visualizzati sopra il modulo in un elenco con classe `alert alert-warning`. Questo è utile per informare l'utente di eventuali problemi o successi.

- **Modulo di Registrazione:**

- Il modulo è centrato nella pagina, con una larghezza adeguata per garantire leggibilità su desktop e dispositivi mobili.

- Campi principali:
 - * **Nome utente** (username)
 - * **Password** (password)
 - * **Nome completo** (name)
 - * **Indirizzo** (address, opzionale)
 - * **Ruolo** (role), con un elenco a discesa per selezionare tra i ruoli disponibili (Ristorante, Rider, Cliente, Amministratore).
- Campo nascosto (hidden input) per l’ID del ristorante se passato come parametro nella query string.
- **Pulsante di Registrazione:**
 - Un pulsante ben visibile, che invita l’utente a completare la registrazione.
- **Link di Login:**
 - Fornisce un’opzione per tornare alla pagina di login, con un link che include il parametro `restaurant_id` (se presente), utile per mantenere il contesto.

5.1.16 restaurant_dashboard.html

La pagina di registrazione è ben strutturata e fornisce un’interfaccia user-friendly per l’utente. Ecco un’analisi e alcune considerazioni:

- **Header:**
 - Un’intestazione semplice e pulita, con sfondo scuro e testo bianco, che accoglie l’utente con il messaggio ”Registrati alla Piattaforma di Food Delivery”.
- **Messaggi Flash:**
 - Se sono presenti messaggi flash (ad esempio, errori o conferme), vengono visualizzati sopra il modulo in un elenco con classe `alert alert-warning`. Questo è utile per informare l’utente di eventuali problemi o successi.
- **Modulo di Registrazione:**
 - Il modulo è centrato nella pagina, con una larghezza adeguata per garantire leggibilità su desktop e dispositivi mobili.
 - Campi principali:
 - * **Nome utente** (username)
 - * **Password** (password)
 - * **Nome completo** (name)
 - * **Indirizzo** (address, opzionale)
 - * **Ruolo** (role), con un elenco a discesa per selezionare tra i ruoli disponibili (Ristorante, Rider, Cliente, Amministratore).
 - Campo nascosto (`hidden input`) per l’ID del ristorante se passato come parametro nella query string.

- **Pulsante di Registrazione:**

- Un pulsante ben visibile e largo al 100%, che invita l'utente a completare la registrazione.

- **Link di Login:**

- Fornisce un'opzione per tornare alla pagina di login, con un link che include il parametro `restaurant_id` (se presente), utile per mantenere il contesto.

5.1.17 `restaurant_edit_item.html`

La struttura della pagina "Modifica Piatto" è ben organizzata e segue un layout semplice ed efficace. Di seguito un'analisi della struttura e alcune considerazioni:

- **<head>:**

- Include le basi per una pagina moderna:
 - * Charset UTF-8 per supportare i caratteri italiani.
 - * Meta tag per la responsività (`viewport`).
 - * Bootstrap CSS per lo styling rapido e coerente.

- **<header>:**

- Include:
 - * Un titolo ben visibile ("Modifica Piatto") centrato.
 - * Pulsanti di navigazione:
 - **Torna alla Dashboard:** Link che reindirizza alla dashboard del ristorante.
 - **Logout:** Form con un pulsante per gestire la disconnessione.

- **<main>:**

- **Sezione centrale:**

- * Racchiude il modulo di modifica del piatto in un contenitore con ombreggiatura e bordi arrotondati.
- * Elementi del modulo:
 - Nome del piatto (`name`): Campo di testo obbligatorio.
 - Descrizione (`description`): Textarea opzionale.
 - Prezzo (`price`): Campo numerico obbligatorio con supporto per i decimali.
 - Disponibilità (`available`): Menu a tendina per selezionare "Sì" o "No".

- **Pulsante per inviare il modulo:**

- * Grande, visibile e a larghezza completa.

- **Script:**

- Include Bootstrap JS per le funzionalità dinamiche del framework.
- Collega un file JavaScript personalizzato (`restaurant_edit_item.js`) per la logica specifica della pagina.

5.1.18 restaurant_manage_menu.html

La pagina Gestione Menu Ristorante è ben strutturata e contiene due sezioni principali: una per visualizzare e gestire i piatti esistenti e una per aggiungere nuovi piatti. Di seguito, analizziamo in dettaglio la struttura:

- **<header>**: Contiene:
 - Un titolo ben visibile ("Gestione Menu Ristorante").
 - Pulsanti di navigazione:
 - * **Dashboard**: Link per tornare alla dashboard del ristorante.
 - * **Logout**: Form per la disconnessione, con pulsante ben visibile.
- **<main>**: Contiene due sezioni principali:
 - **Visualizzazione del menu**:
 - * Lista dei piatti esistenti, ognuno mostrato come un elemento di una lista con dettagli:
 - Nome del piatto, descrizione, prezzo e stato di disponibilità.
 - Pulsanti per modificare o eliminare il piatto.
 - * Ogni elemento è strutturato con un layout flessibile grazie a Bootstrap:
 - **Descrizione e dettagli**: Div flessibile con colonne.
 - **Azioni**: Pulsanti "Modifica" ed "Elimina", ben visibili e separati.
 - **Modulo per aggiungere/modificare piatti**:
 - * Campi di input per:
 - Nome del piatto (**name**): Testo obbligatorio.
 - Descrizione (**description**): Campo textarea opzionale.
 - Prezzo (**price**): Campo numerico obbligatorio con supporto per decimali.
 - Disponibilità (**available**): Menu a tendina con opzioni "Sì" e "No".
 - * Pulsante "Aggiungi Piatto" con stile evidenziato.
- **Script**: Include:
 - **Bootstrap JS**: Per funzionalità dinamiche e interattive.
 - **Script personalizzato** (`restaurant_manage_menu.js`): Presumibilmente per gestire l'invio del modulo o l'interazione con il backend tramite AJAX.

5.1.19 restaurant_order_details.html

La pagina Dettagli Ordine è ben strutturata, chiara e offre un'interfaccia user-friendly per visualizzare i dettagli dell'ordine e modificarne lo stato. Di seguito, una panoramica degli elementi principali.

- **Titolo**:
 - Visualizza l'ID dell'ordine in maniera prominente al centro della pagina.

- **Dettagli Ordine:**

- Mostra:
 - * **Cliente:** Identificato con `order.customer_id`. Si potrebbe considerare di sostituire l'ID con il nome del cliente per maggiore leggibilità.
 - * **Data dell'ordine.**
 - * **Stato attuale.**
- Questi dettagli sono racchiusi in un contenitore elegante con sfondo bianco e ombra.

- **Articoli Ordinati:**

- Gli articoli sono elencati come elementi di una lista (`<ul>` con `list-group` di Bootstrap), che include:
 - * Nome dell'articolo.
 - * Quantità.
 - * Prezzo per unità.

- **Totale dell'Ordine:**

- Mostra il totale in maniera evidente con un elemento `<p>` separato.

- **Modifica Stato:**

- Utilizza un menu a tendina (`<select>`) per cambiare lo stato dell'ordine.
- I valori possibili sono pre-popolati con una selezione condizionale basata sullo stato corrente.
- Il form include un pulsante "Aggiorna Stato" per inviare i dati.

- **Navigazione:**

- Un pulsante "Torna alla lista ordini" permette all'utente di tornare alla pagina principale degli ordini del ristorante.

5.1.20 restaurant_view_orders.html

La pagina "Visualizza Ordini" sembra ben strutturata e funzionale, con una buona organizzazione dei dati. Ecco una panoramica dei suoi principali componenti.

- **Titolo e Descrizione:**

- Titolo chiaro: "Ordini ricevuti".
- Una breve descrizione per spiegare cosa l'utente sta per visualizzare.

- **Tabella degli Ordini:**

- La tabella contiene:
 - * **ID Ordine**

- * **ID Cliente**
- * **Data Ordine**
- * **Stato**
- * **Prezzo Totale**
- * **Dettagli:** un link per visualizzare i dettagli dell'ordine.
- La tabella è ben formattata con classi Bootstrap per migliorarne l'aspetto.
- **Stato degli Ordini:**
 - Ogni ordine ha il suo stato mostrato chiaramente nella colonna "Stato".
- **Messaggio in Caso di Mancanza di Ordini:**
 - Se non ci sono ordini, un messaggio di avviso viene mostrato con una classe di alert.
- **Navigazione:**
 - Il pulsante "Torna alla Dashboard" permette all'utente di tornare facilmente alla pagina di dashboard del ristorante.

5.1.21 restaurant_view_profile.html

La pagina "Dettagli del Ristorante" appare ben progettata e strutturata per permettere ai ristoratori di gestire il proprio profilo. Ecco una panoramica dei suoi principali componenti:

- **Titolo e Form di Modifica:**
 - Il titolo "Gestisci Profilo Ristorante" è chiaro e descrive bene la funzione della pagina.
 - Il modulo contiene tutti i campi principali per la gestione del profilo del ristorante:
 - * **Nome del Ristorante**
 - * **Indirizzo**
 - * **Telefono**
 - * **Email**
 - * **Orari di Apertura e Chiusura (JSON):** utile per gestire gli orari flessibili.
 - * **Stato di Apertura:** con un 'select' per segnare se il ristorante è aperto o meno.
 - * **Descrizione:** per aggiungere dettagli extra.
 - * **Logo:** consente di visualizzare e aggiornare l'URL del logo.
 - Il pulsante di submit "Salva Modifiche" è ben posizionato e ben visibile.

- **Visualizzazione del Logo:**

- Se è presente un logo, viene mostrato sotto il campo di input con una dimensione adeguata.

- **Pulsanti di Navigazione:**

- I pulsanti "Torna alla Dashboard" e "Logout" sono ben posizionati per permettere all'utente di navigare facilmente nella piattaforma.

5.1.22 rider_dashboard.html

La pagina "Rider Dashboard" è ben strutturata e facile da navigare. Ogni sezione della dashboard è chiara e suddivisa in card per un'esperienza utente ottimale. Ecco una panoramica:

- **Card della Dashboard:**

- Le card sono ben organizzate e ogni card fornisce una panoramica delle diverse sezioni per la gestione delle consegne. Ogni card include:
 - * **Titolo chiaro** (ad esempio, "Richieste di Consegna", "Percorsi Ottimali", ecc.)
 - * **Descrizione breve** per ogni sezione.
 - * **Pulsante di azione** per accedere direttamente alla sezione corrispondente.
- Queste card sono facili da navigare e aiutano l'utente a trovare rapidamente le informazioni o le azioni necessarie.

- **Effetto Hover:**

- L'effetto **hover** che ingrandisce leggermente le card è un'ottima aggiunta per migliorare l'interazione con la pagina. Aggiunge un tocco visivo piacevole che invoglia l'utente a cliccare.

- **Pulsante Logout:**

- Il pulsante di logout è chiaramente visibile e ben posizionato al centro della pagina, il che rende facile per l'utente uscire dalla dashboard.

5.1.23 rider_delivery_requests.html

La pagina "Richieste di Consegna" include un'area per visualizzare le richieste di consegna e un indicatore di caricamento (spinner) per informare l'utente che i dati sono in fase di caricamento.

- **Mostrare Richieste di Consegna Dinamicamente:**

- Carica le richieste dinamicamente con JavaScript, quindi il file `rider_delivery_requests.js` deve gestire l'invio della richiesta e la visualizzazione dei dati. Assicurati che il JavaScript gestisca correttamente i seguenti passaggi:

- * **Mostrare il caricamento** (spinner) mentre le richieste vengono recuperate.
- * **Visualizzare le richieste** una volta che sono state caricate.
- * **Gestire eventuali errori** (come problemi di connessione) e visualizzare un messaggio utile.

- **Pulsante per Tornare alla Dashboard:**

- Il pulsante che rimanda alla dashboard del rider è ben posizionato. La forma del pulsante è chiara e visibile. Inoltre, il modulo per il logout è efficiente e l'utente è facilmente in grado di effettuare il logout.

5.1.24 rider_track_optimal_routes.html

Questo codice HTML crea una pagina web per visualizzare gli ordini tracciabili con la possibilità di mostrare una mappa interattiva per tracciare il percorso di consegna. La pagina include:

- **Titolo e Meta Dati:** Il titolo della pagina è "Traccia Percorso Consegna" e sono presenti i meta tag per la visualizzazione su dispositivi mobili.
- **Design con Bootstrap:** Viene utilizzato Bootstrap per il layout e gli stili. La lista degli ordini e i pulsanti sono stilizzati con classi di Bootstrap.
- **Mappa Interattiva (Leaflet):** Viene inclusa la libreria Leaflet per visualizzare la mappa, che è nascosta inizialmente e viene mostrata solo quando necessario.
- **Funzionalità di Logout e Navigazione:** Pulsanti per tornare alla dashboard o effettuare il logout.
- **Caricamento Dinamico:** Gli ordini vengono caricati dinamicamente con JavaScript.

5.1.25 rider_update_availability.html

Questo codice HTML crea una pagina web per aggiornare la disponibilità di un rider. La pagina include:

- **Titolo e Meta Dati:** Il titolo è "Aggiorna Disponibilità", con un meta tag per la visualizzazione su dispositivi mobili.
- **Form per Aggiornare la Disponibilità:**
 - Un **checkbox** per impostare se il rider è disponibile o meno.
 - Due selettori a discesa per scegliere l'**orario di inizio** e l'**orario di fine** della disponibilità, con opzioni per ore e minuti.
- **Stili con Bootstrap:** La pagina usa il framework Bootstrap per una formattazione responsive e un layout coerente.

- **Funzionalità di Salvataggio:** Un pulsante per inviare i dati del form e aggiornare la disponibilità.
- **Navigazione:** Pulsanti per tornare alla dashboard e per fare il logout.
- **JavaScript Personalizzato:** Un file esterno per gestire la logica di aggiornamento della disponibilità.

5.1.26 rider_update_status.html

Questo codice HTML crea una pagina per l'aggiornamento dello stato delle consegne da parte del rider. Contiene:

- **Titolo e Meta Dati:** La pagina è intitolata "Aggiorna Stato Consegne" e ha un meta tag per la visualizzazione su dispositivi mobili.
- **Lista delle Consegne:** Ogni consegna è visualizzata come un elemento della lista con:
 - L'**ID** della consegna e l'**ID dell'ordine**.
 - Lo **stato attuale** della consegna, visualizzato in grassetto.
 - Un **menu a discesa** per aggiornare lo stato della consegna, con tre opzioni: "Accettato dal Rider", "In Consegna" e "Completato". L'opzione selezionata è evidenziata in base allo stato attuale.
- **Pulsanti di Navigazione:**
 - Un pulsante per tornare alla **Dashboard** del rider.
 - Un pulsante per fare il **logout**.
- **JavaScript Personalizzato:** Viene incluso un file esterno (`rider_update_status.js`) per gestire la logica di aggiornamento dello stato delle consegne.

5.1.27 rider_view_profile.html

Questo codice HTML crea una pagina per la gestione del profilo utente all'interno di una piattaforma di food delivery. Contiene:

- **Navbar:** Una barra di navigazione in alto con il nome della piattaforma "Food Delivery".
- **Messaggi di Successo/Errore:** Viene mostrato un messaggio (come un>alert) per informare l'utente su eventuali aggiornamenti riusciti o falliti, tramite la funzione `get_flashed_messages`.
- **Form di Modifica Profilo:**
 - **Username, Nome, Email, Numero di Telefono, Indirizzo:** Questi campi vengono precompilati con le informazioni correnti dell'utente, consentendo di aggiornarle. Alcuni campi (come l'email e il numero di telefono) sono facoltativi, mentre altri (come il nome e l'indirizzo) sono obbligatori.

- **Pulsante di Aggiornamento:** Un pulsante per inviare il form e aggiornare le informazioni del profilo.
- **Pulsante di Logout e Navigazione:**
 - Un pulsante per tornare alla **Dashboard** dell'utente.
 - Un pulsante per effettuare il **logout**.

5.2 Static

5.2.1 admin_manage_accounts.js

Questo codice JavaScript si occupa di gestire l'eliminazione degli utenti attraverso l'interfaccia di amministrazione di una piattaforma. Ecco come funziona:

- **Selezione dei Bottoni di Eliminazione:** Utilizza `document.querySelectorAll('.delete-btn')` per selezionare tutti i pulsanti con la classe `.delete-btn` sulla pagina.
- **Aggiunta dell'Evento di Click:** A ciascun pulsante viene aggiunto un **evento di click** tramite `addEventListener`. Quando un pulsante viene cliccato, viene eseguita la funzione associata.
- **Richiesta di Conferma:** Prima di procedere con l'eliminazione, viene mostrato un messaggio di conferma con `confirm()`, chiedendo se l'amministratore è sicuro di voler eliminare l'utente.
- **Invio della Richiesta di Eliminazione:** Se l'utente conferma, viene inviata una **richiesta POST** tramite `fetch` al percorso `/admin/manage_accounts` con un corpo in formato JSON contenente l'ID dell'utente (`user_id`).
- **Gestione della Risposta:** Dopo che il server ha risposto:
 - Se la risposta è positiva (`response.ok`), viene mostrato un messaggio di successo e la pagina viene ricaricata.
 - Se la risposta è negativa, viene visualizzato un errore con un messaggio appropriato.
- **Funzionalità Complessiva:** Questo codice permette di eliminare un utente dal sistema, gestendo sia la conferma dell'operazione che la risposta del server.

5.2.2 admin_monitor_performance.js

Questo codice JavaScript crea un grafico a barre utilizzando la libreria Chart.js. Ecco una spiegazione di come funziona:

- **Creazione del Grafico a Barre:**
 - **Etichette sull'asse X:** Ogni barra rappresenta un ristorante, con i nomi dei ristoranti visualizzati come etichette sull'asse orizzontale.
 - **Altezza delle barre:** La lunghezza di ciascuna barra corrisponde al numero di ordini ricevuti dal ristorante.
 - **Colore delle barre:** Le barre sono colorate di blu con una sfumatura trasparente, rendendole visivamente più leggere.
- **Opzioni di Configurazione:**
 - **Scalabilità del grafico:** Il grafico si adatta automaticamente alle dimensioni della finestra del browser, garantendo una visualizzazione ottimale su dispositivi di diverse dimensioni.

- **Asse Y:** L'asse verticale inizia da zero, assicurando che il numero di ordini sia rappresentato correttamente.
- **Utilità:** Questa configurazione è utile per visualizzare le performance di ciascun ristorante in modo chiaro e intuitivo.

5.2.3 customer_track_orders.js

Il codice si occupa di inizializzare una mappa per ciascun ordine di consegna e mostrare il percorso tra il ristorante e la destinazione. Ecco una descrizione del funzionamento:

- **Caricamento dei Dati:**
 - Quando la pagina viene caricata, il codice recupera i dati degli ordini da un elemento HTML con `id="orders-data"` che contiene informazioni sugli ordini in formato JSON.
- **Inizializzazione delle Mappe:**
 - La funzione `initializeMap` viene definita per creare una mappa per ciascun ordine, utilizzando la libreria Leaflet.
 - La funzione prende in input l'ID dell'ordine, le coordinate del ristorante (latitudine e longitudine) e quelle della destinazione.
 - Crea un contenitore di mappa per ogni ordine, utilizzando un elemento con ID dinamico basato sull'ID dell'ordine.
 - Se le coordinate del ristorante e della destinazione sono valide, viene creata una mappa centrata sulle coordinate del ristorante, con l'aggiunta di marker per il ristorante e la destinazione (se presente).
 - Una linea blu viene disegnata per collegare il ristorante alla destinazione.
- **Verifica della Presenza di Dati Validi:**
 - Se le coordinate del ristorante o della destinazione non sono valide (coordinate vuote o non definite), il codice mostra un errore e non inizializza la mappa.
- **Gestione della Visualizzazione della Mappa:**
 - Se entrambe le coordinate (ristorante e destinazione) sono valide, la mappa si adatta automaticamente per visualizzare l'intero percorso.
 - Se la destinazione non è disponibile, la mappa si centrerà solo sulla posizione del ristorante.
- **Uso delle Mappe:**
 - Le mappe vengono memorizzate in un oggetto `mapInstances`, dove l'ID dell'ordine è la chiave e il valore è l'oggetto mappa associato.
- **Funzionalità Generale:**
 - Questo codice crea e gestisce dinamicamente le mappe per ciascun ordine, tracciando il percorso dal ristorante alla destinazione e centrando correttamente la visualizzazione sulla mappa.

5.2.4 home.js

Il codice svolge due funzioni principali:

- **Gestire la visibilità del campo 'restaurant_id' in base al ruolo selezionato:**
 - **Funzionamento:** Quando viene cambiato il valore nel campo di selezione del ruolo (con ID `role`), il codice verifica se il valore selezionato è "restaurant".
 - **Se il ruolo è "restaurant":** Viene mostrato il campo con ID `restaurant-id`, che è presumibilmente un campo di input inizialmente nascosto.
 - **Se il ruolo non è "restaurant":** Il campo `restaurant-id` viene nascosto.
 - **Tecnologia utilizzata:** Questo comportamento è implementato tramite l'evento `change` legato al campo di selezione del ruolo.
- **Verifica che un ruolo valido sia selezionato prima di inviare il form:**
 - **Funzionamento:** Quando l'utente cerca di inviare il modulo (con ID `roleForm`), il codice controlla se è stato selezionato un ruolo nel campo di selezione del ruolo.
 - **Se non è selezionato nessun ruolo:** Viene visualizzato un messaggio di avviso tramite `alert()`, e l'invio del modulo viene impedito tramite la chiamata `event.preventDefault()`.
 - **Tecnologia utilizzata:** Questo controllo viene eseguito con l'evento `submit` legato al modulo.
- **Utilità:**
 - Questa logica è utile quando si vogliono personalizzare i campi di un form in base alle scelte dell'utente, migliorando l'usabilità e la validazione lato client.

5.2.5 login.js

Il codice si occupa di gestire l'URL del link di registrazione (`register-link`), aggiungendo o rimuovendo il parametro `restaurant_id` in base al valore del campo nascosto (`restaurant_id`). Ecco come funziona:

- **Verifica dell'esistenza degli elementi:**
 - **Funzionamento:** Prima di proseguire, il codice controlla se gli elementi con gli ID `restaurant_id` (campo nascosto) e `register-link` (link di registrazione) esistono nel DOM.
 - **Tecnologia utilizzata:** L'uso di `getElementById()` per recuperare gli elementi e un controllo di esistenza (`if`).
- **Gestione dell'ID ristorante:**
 - **Funzionamento:** Se entrambi gli elementi esistono, il codice prende il valore del campo nascosto `restaurant_id` e lo memorizza nella variabile `restaurantId`.

- Se il valore di `restaurantId` esiste (cioè se non è vuoto), viene aggiunto o aggiornato come parametro `restaurant_id` nell'URL del link di registrazione (`registerLink`).
- Se il campo `restaurant_id` non ha valore, il parametro `restaurant_id` viene rimosso dall'URL.
- **Modifica dell'URL:**
 - **Funzionamento:** Il codice utilizza l'oggetto `URL` per gestire l'URL in modo sicuro, aggiungendo o rimuovendo i parametri di ricerca tramite `searchParams.set()` o `searchParams.delete()`.
 - **Tecnologia utilizzata:** `URL` è un oggetto integrato in JavaScript che semplifica la manipolazione degli URL.
- **Aggiornamento del link:**
 - **Funzionamento:** Una volta modificato l'URL, il codice aggiorna l'attributo `href` del link `register-link` con il nuovo URL costruito.
- **In sintesi:**
 - Il codice verifica se il campo `restaurant_id` e il link di registrazione esistono nel DOM.
 - Se `restaurant_id` è presente, il parametro viene aggiunto o aggiornato nell'URL del link, altrimenti viene rimosso.
 - L'oggetto `URL` viene utilizzato per manipolare in modo sicuro l'URL del link.

5.2.6 restaurant_edit_item.js

Il codice gestisce l'aggiornamento di un piatto nel menu di un ristorante. Ecco come funziona passo passo:

- **Caricamento dei dati del piatto:**
 - **Quando:** Quando la pagina è completamente caricata, l'evento `DOMContentLoaded` viene attivato.
 - **Cosa fa:** Il codice esegue una richiesta `GET` al server per recuperare i dati del piatto specificato nell'URL (tramite il parametro `id` della query string).
 - **URL di richiesta:** `/restaurant/menu/{restaurantId}/{itemId}`:
 - * `restaurantId`: Rappresenta l'ID del ristorante.
 - * `itemId`: L'ID del piatto che si desidera modificare, estratto dalla query string dell'URL.
 - **Gestione della risposta:** Se la risposta del server è positiva (`status OK`), i dati del piatto vengono utilizzati per precompilare il modulo con il nome, la descrizione, il prezzo e la disponibilità del piatto.

- **Modifica dei dati del piatto:**

- **Quando:** Quando il modulo di modifica (il form) viene inviato (evento `submit`).
- **Cosa fa:** Il codice raccoglie i valori inseriti nel modulo e li organizza in un oggetto `updatedItem`:
 - * **name:** Nome del piatto.
 - * **description:** Descrizione del piatto.
 - * **price:** Prezzo del piatto (convertito in formato numerico).
 - * **available:** Disponibilità del piatto (convertita in booleano tramite una stringa `"true"` o `"false"`).
- **Richiesta di aggiornamento:** Viene inviata una richiesta `PUT` al server con i dati modificati nel corpo della richiesta. L'URL di richiesta è simile al primo, ma stavolta con il metodo `PUT` per aggiornare i dati.
- **Gestione della risposta:** Se la risposta del server è positiva, viene mostrato un messaggio di successo e la pagina viene reindirizzata al menu (`/menu`). Se c'è un errore, viene mostrato un messaggio di errore e i dettagli dell'errore vengono stampati nella console.

- **Gestione degli errori:**

- **Errori nella richiesta:** Se si verifica un errore durante la richiesta al server (ad esempio, la rete è inattiva), viene catturato e viene visualizzato un messaggio di errore generico all'utente.
- **Errori di risposta:** Se la risposta del server non è `OK`, viene mostrato un messaggio di errore specifico che viene recuperato dalla risposta `JSON`.

- **In sintesi:**

- Il codice consente di precompilare un modulo con i dati esistenti di un piatto e di aggiornarli sul server tramite una richiesta `PUT` quando l'utente invia il modulo.
- Gestisce anche gli errori di rete e le risposte del server, fornendo feedback all'utente tramite messaggi di successo o errore.

5.2.7 restaurant_manage_menu.js

Questo codice gestisce l'interazione con il menu di un ristorante. Ecco un riassunto delle funzionalità principali:

- **Caricamento del Menu:**

- **Quando:** Quando la pagina è completamente caricata (`DOMContentLoaded`), viene effettuata una richiesta al server per recuperare il menu del ristorante tramite l'endpoint `/restaurant/menu/json/{restaurantId}`.
- **Cosa fa:** Se la richiesta ha successo, viene chiamata la funzione `renderMenu` per visualizzare gli elementi del menu.

- **Visualizzazione del Menu:**

- **Come:** Gli elementi del menu sono visualizzati come una lista (`<ul>`), con ogni piatto rappresentato da un elemento `<li>`.
- Ogni piatto ha due pulsanti:
 - * **Modifica:** Per modificare un piatto esistente.
 - * **Elimina:** Per eliminare un piatto dal menu.
- Ogni pulsante ha un `data-id` che contiene l'ID del piatto, utile per fare riferimento al piatto specifico durante le operazioni.

- **Modifica di un Piatto:**

- **Quando:** Quando l'utente clicca su "Modifica" per un piatto.
- **Cosa fa:** Viene inviata una richiesta `GET` per ottenere i dettagli del piatto dal server.
- I dati del piatto (nome, descrizione, prezzo, disponibilità) vengono inseriti nel modulo di aggiunta del piatto (`addItemForm`), trasformandolo in un modulo di modifica.
- Il titolo del modulo viene cambiato in "Modifica il Piatto" per informare l'utente.

- **Aggiunta o Modifica di un Piatto:**

- **Quando:** Quando il modulo di aggiunta (`addItemForm`) viene inviato.
- **Cosa fa:** I dati vengono raccolti e trasformati in un oggetto `newItem`.
- Se il form è in modalità modifica (`isEditing`), viene inviata una richiesta `PUT` per aggiornare il piatto esistente. Se non è in modalità modifica, viene inviata una richiesta `POST` per aggiungere un nuovo piatto.
- Dopo l'operazione, il menu viene ricaricato e il modulo viene ripristinato per aggiungere nuovi piatti.

- **Eliminazione di un Piatto:**

- **Quando:** Quando l'utente clicca sul pulsante "Elimina".
- **Cosa fa:** Viene chiesto di confermare l'eliminazione tramite una finestra di conferma (`confirm`).
- Se l'utente conferma, viene inviata una richiesta `DELETE` al server per eliminare il piatto.
- Dopo l'eliminazione, il menu viene ricaricato per riflettere le modifiche.

- **Gestione degli Errori:**

- Se si verifica un errore durante una richiesta (per recupero, aggiornamento o eliminazione), viene mostrato un messaggio di errore nel `console.log`.
- In caso di errore nell'aggiornamento o nell'eliminazione, un `alert` avvisa l'utente.

- **In sintesi:**

- Il codice consente di visualizzare, aggiungere, modificare ed eliminare piatti dal menu del ristorante.
- La gestione delle operazioni di modifica e aggiunta viene separata tramite un flag (`isEditing`), per distinguere tra modifica di un piatto esistente e aggiunta di un nuovo piatto.
- Ogni operazione viene eseguita tramite richieste asincrone (`fetch`) e il menu viene ricaricato automaticamente dopo ogni modifica.

5.2.8 restaurant_view_orders.js

Il codice fornisce una connessione in tempo reale a un server utilizzando Socket.IO. Ecco un'analisi dettagliata:

- **Connessione al Server:**

- **Come:** La riga `var socket = io.connect('http://' + document.domain + ':' + location.port);` stabilisce una connessione con il server Socket.IO.
- **Cosa fa:** Utilizza `document.domain` e `location.port` per ottenere dinamicamente l'indirizzo e la porta del server.
- **Dove:** Solitamente la connessione avviene sul server stesso, se in ambiente locale.

- **Ricezione Nuovo Ordine:**

- **Quando:** Quando l'evento `new_order` viene emesso dal server.
- **Cosa fa:** La funzione `socket.on('new_order', function(order) ...);` ascolta l'evento `new_order` e riceve i dettagli dell'ordine tramite il parametro `order`.
- **Cosa contiene:** `order` è un oggetto che include le informazioni dell'ordine appena ricevuto.

- **Aggiornamento della Lista degli Ordini:**

- **Cosa fa:** Quando arriva un nuovo ordine, viene creata una nuova riga (`<tr>`) nella tabella degli ordini.
- La riga contiene:
 - * **ID dell'ordine**
 - * **ID del cliente**
 - * **Data dell'ordine**
 - * **Stato dell'ordine**
 - * **Totale dell'ordine** (formattato con un decimale)
 - * Un link con il pulsante "Visualizza Dettagli" per i dettagli dell'ordine
- **Dove:** La nuova riga viene aggiunta alla lista degli ordini (`orders-list`).

- **Funzione di Aggiornamento Dinamico:**

- **Come:** Ogni volta che un nuovo ordine arriva, il DOM viene aggiornato dinamicamente per aggiungere la nuova riga alla lista.
- **Vantaggio:** Non è necessario ricaricare la pagina, poiché la lista degli ordini si aggiorna in tempo reale.

- **In sintesi:**

- Socket.IO viene utilizzato per ricevere ordini dal server in tempo reale.
- Quando arriva un nuovo ordine, il codice aggiorna dinamicamente la lista degli ordini, permettendo agli utenti di vederli immediatamente senza ricaricare la pagina.

5.2.9 rider_delivery_requests.js

Il codice gestisce le richieste di consegna in una piattaforma di food delivery. Ecco un'analisi passo-passo di come funziona:

- **Funzione `loadRequests()`:**

- **Obiettivo:** Caricare tutte le richieste di consegna tramite una chiamata AJAX e renderizzarle nella pagina.
- **Dettagli:**
 - * Mostra un *loading spinner* per indicare che i dati sono in fase di caricamento.
 - * Effettua una richiesta `fetch` all'endpoint `/delivery/requests_data` per ottenere i dati delle richieste.
 - * Se non ci sono richieste, viene visualizzato un messaggio che indica “*Nessuna richiesta disponibile*”.
 - * Ogni richiesta viene renderizzata come una riga con dettagli come l'ID del ristorante, la distanza e lo stato della richiesta.
 - * Se la richiesta ha lo stato “*ready_for_pickup*”, vengono mostrati due pulsanti: *Accetta* e *Rifiuta*.
 - * Ogni stato della richiesta ha un badge di colore diverso per evidenziarlo (Accettato, Pronto per il Ritiro, In Consegna, Completato).
 - * Dopo aver caricato i dati, il *loading spinner* viene nascosto.

- **Funzione `handleRequest()`:**

- **Obiettivo:** Gestire l'azione (accettare o rifiutare) su una richiesta di consegna.
- **Dettagli:**
 - * Prende l'ID dell'ordine (`orderId`), l'azione (`accept` o `reject`), e l'elemento del pulsante come parametri.
 - * Invia una richiesta `POST` all'endpoint `/delivery/requests/{orderId}/action`, passando l'azione selezionata nel corpo della richiesta.

- * Se l'azione è completata con successo, un messaggio di successo viene visualizzato tramite `alert(data.message)`.
- * I pulsanti di accettazione/rifiuto vengono *disabilitati* dopo l'azione.
- * Il badge di stato dell'ordine viene aggiornato in base all'azione (Accettato/Rifiutato).
- * La funzione ricarica la lista delle richieste per riflettere l'aggiornamento.

- **Caricamento della Pagina:**

- Quando la pagina è completamente caricata (`window.onload`), viene chiamata la funzione `loadRequests()` per popolare la lista delle richieste.

- **Gestione degli Errori:**

- Durante il caricamento delle richieste o l'invio delle azioni, se si verifica un errore (problemi di rete o server), un messaggio di errore viene mostrato nella console e il *loading spinner* viene nascosto.

- **In sintesi:**

- La funzione carica e visualizza tutte le richieste di consegna, mostrando un *caricamento visivo* tramite uno spinner.
- Le richieste possono essere accettate o rifiutate. I pulsanti vengono disabilitati dopo l'azione e il badge di stato viene aggiornato.
- Dopo ogni azione, le richieste vengono ricaricate per riflettere l'aggiornamento.
- Il codice offre un'esperienza utente fluida senza dover ricaricare la pagina.

5.2.10 `rider_track_optimal_routes.js`

Il codice gestisce la tracciatura degli ordini di consegna in tempo reale tramite una mappa interattiva. Ecco un'analisi dettagliata di come funziona:

- **Funzione `loadOrders()`:**

- **Obiettivo:** Caricare e visualizzare gli ordini disponibili per la tracciatura tramite una richiesta AJAX.
- **Dettagli:**
 - * Effettua una richiesta `fetch` all'endpoint `/delivery/optimal-route` per ottenere i dati relativi agli ordini.
 - * Una volta ricevuti i dati, la funzione svuota la lista esistente di ordini (`ordersList.innerHTML = ''`).
 - * Se non ci sono ordini, viene mostrato il messaggio *“Nessun ordine disponibile per la tracciatura.”*
 - * Ogni ordine viene renderizzato in un elemento `div` con un ID unico per la mappa.
 - * Per ogni ordine, vengono visualizzati:
 - L'ID della consegna.

- Le coordinate di partenza e di destinazione.
- Un pulsante per tracciare il percorso.
- * Quando il pulsante “*Traccia percorso*” viene cliccato, viene invocata la funzione `trackRoute()`, passando i dati dell’ordine e l’ID unico della mappa.

- **Funzione `trackRoute()`:**

- **Obiettivo:** Visualizzare il percorso dell’ordine selezionato sulla mappa.
- **Dettagli:**
 - * Quando viene cliccato il pulsante per tracciare il percorso, la funzione riceve l’ordine (salvato nei dati dell’attributo `data-order`) e l’ID della mappa associata.
 - * Prima di visualizzare la mappa, la funzione nasconde tutte le mappe esistenti sulla pagina (`maps.forEach(map => map.style.display = 'none');`)).
 - * La mappa selezionata viene poi mostrata.
 - * **Creazione della mappa:**
 - Viene utilizzata la libreria **Leaflet** per creare la mappa.
 - La mappa viene centrata sulla posizione iniziale dell’ordine (primo punto del percorso) e viene impostato un livello di zoom (13).
 - Viene aggiunta la **TileLayer** di OpenStreetMap.
 - Vengono aggiunti due **marker** per indicare la posizione di partenza e la destinazione, con delle etichette pop-up (“*Punto di partenza*” e “*Destinazione*”).
 - Viene disegnata una **polilinea** blu che segue il percorso dell’ordine (una serie di coordinate di latitudine e longitudine).

- **Caricamento della Pagina:**

- La funzione `loadOrders()` viene invocata automaticamente quando la pagina viene caricata (`window.onload = function() { loadOrders(); };`), quindi gli ordini sono caricati subito senza bisogno di un’ulteriore interazione dell’utente.

- **Gestione degli Errori:**

- Se si verifica un errore durante il caricamento degli ordini, un messaggio di errore viene stampato nella console.

- **In sintesi:**

- La funzione carica gli ordini di consegna e li visualizza nella pagina, con la possibilità di tracciare ogni ordine sulla mappa.
- Quando l’utente clicca sul pulsante “*Traccia percorso*”, viene mostrata la mappa interattiva con il percorso dell’ordine, inclusi i marker di partenza e destinazione.
- Viene utilizzata la libreria **Leaflet** per visualizzare le mappe e tracciare il percorso.

5.2.11 rider_update_availability.js

Il codice gestisce l'aggiornamento della disponibilità di un rider (o di un altro utente del sistema) tramite un modulo HTML. Ecco una panoramica dettagliata:

- **Ascoltatore dell'evento di invio del modulo:**
 - L'evento `submit` viene associato al modulo con ID `availability-form`. Quando il modulo viene inviato (premendo il pulsante di invio), viene eseguita una funzione asincrona (`async (e) => ...`).
- **Prevenzione dell'invio predefinito del modulo:**
 - Il metodo `e.preventDefault()` impedisce il comportamento predefinito del modulo, che normalmente invierebbe una richiesta HTTP e ricaricherebbe la pagina.
- **Estrazione dei dati dal modulo:**
 - **Disponibilità:** Viene recuperato lo stato del checkbox `isAvailable` (se il rider è disponibile o meno).
 - **Orari:** I valori delle ore e dei minuti per l'inizio e la fine della disponibilità vengono letti dai rispettivi input `startHour`, `startMinute`, `endHour`, `endMinute` e combinati in una stringa di formato `HH:mm`.
- **Preparazione della richiesta POST:**
 - La funzione invia una richiesta POST all'endpoint `/delivery/availability`, inviando i dati come JSON nel corpo della richiesta:
 - * `is_available`: stato della disponibilità (vero o falso).
 - * `start_time`: orario di inizio della disponibilità.
 - * `end_time`: orario di fine della disponibilità.
- **Gestione della risposta:**
 - Dopo aver inviato la richiesta, la funzione aspetta la risposta usando `await response.json()`.
 - Se la risposta ha uno stato HTTP 200 (indicato da `response.ok`), viene mostrato un messaggio di successo tramite un `alert` e la pagina viene ricaricata (`window.location.reload()`).
 - Se c'è un errore (ad esempio, un errore di validazione o di server), viene mostrato un messaggio di errore tramite un `alert`. Il messaggio di errore viene preso dalla risposta JSON (se disponibile) o da un messaggio di errore generico.

5.2.12 rider_update_status.js

Il codice serve per aggiornare lo stato di una consegna tramite un menu a discesa (`<select>`) presente sulla pagina. Ecco una panoramica di cosa fa:

- **Ascolto degli eventi di cambio stato:**
 - Viene utilizzato `document.querySelectorAll('.status-select')` per selezionare tutti gli elementi `<select>` con la classe `status-select`.
 - A ciascuno di questi elementi viene aggiunto un ascoltatore di eventi per l'evento `change`, che viene attivato quando l'utente cambia la selezione del menu.
- **Recupero dell'ID della consegna e del nuovo stato:**
 - Quando il valore del menu a discesa viene modificato, la funzione ottiene l'ID della consegna (usando `this.getAttribute('data-id')`), che è un attributo personalizzato associato al `<select>`.
 - Inoltre, il nuovo stato selezionato viene recuperato tramite `this.value`.
- **Invio della richiesta per aggiornare lo stato:**
 - Viene inviata una richiesta HTTP PUT all'endpoint `/delivery/{deliveryId}/status`, passando il nuovo stato come corpo della richiesta in formato JSON (`{ status: newStatus }`).
 - Se la richiesta ha successo, una risposta JSON viene ricevuta. Se la risposta contiene un campo `error`, viene mostrato un messaggio di errore tramite un `alert`. Se non ci sono errori, viene mostrato un messaggio di successo.
- **Gestione degli errori:**
 - Se si verifica un errore durante la richiesta (ad esempio, problemi di rete o di server), viene stampato un errore sulla console tramite `console.error('Errore:', error)`.

5.3 Models

5.3.1 delivery.py

Il codice definisce tre modelli per il tuo sistema di delivery, utilizzando SQLAlchemy per la gestione dei dati nel database. Ecco una panoramica di ciascun modello.

- **‘Delivery‘:**
 - **Descrizione:** Modello che rappresenta una consegna.
 - **Campi:**
 - * `id`: ID univoco per la consegna.
 - * `order_id`: ID dell’ordine associato alla consegna (chiave esterna dalla tabella `orders`).
 - * `delivery_person_id`: ID della persona che sta effettuando la consegna (chiave esterna dalla tabella `users`).
 - * `status`: Stato della consegna (ad esempio, `‘pending’`, `‘accepted’`, `‘rejected’`, `‘completed’`).
 - * `current_latitude`, `current_longitude`: Posizione attuale della consegna.
 - * `created_at`, `updated_at`: Timestamp di creazione e aggiornamento.
 - **Relazioni:**
 - * `order`: Relazione con l’ordine associato.
 - * `delivery_person`: Relazione con la persona che sta effettuando la consegna.
- **‘DeliveryPersonAvailability‘:**
 - **Descrizione:** Modello per la disponibilità di un rider per le consegne.
 - **Campi:**
 - * `id`: ID univoco per la disponibilità.
 - * `delivery_person_id`: ID del rider (chiave esterna dalla tabella `users`).
 - * `is_available`: Flag che indica se il rider è disponibile per una consegna.
 - * `start_time`, `end_time`: Orari di inizio e fine della disponibilità.
 - **Relazioni:**
 - * `delivery_person`: Relazione con l’utente/rider.
- **‘OrderReject‘:**
 - **Descrizione:** Modello che registra quando un rider rifiuta un ordine.
 - **Campi:**
 - * `id`: ID univoco per il rifiuto.
 - * `order_id`: ID dell’ordine che è stato rifiutato (chiave esterna dalla tabella `orders`).
 - * `rider_id`: ID del rider che ha rifiutato l’ordine (chiave esterna dalla tabella `users`).

- * **timestamp**: Data e ora in cui l'ordine è stato rifiutato.

- **Relazioni:**

- * **order**: Relazione con l'ordine associato.
- * **rider**: Relazione con l'utente/rider che ha rifiutato l'ordine.

5.3.2 menu.py

Il modello MenuItem rappresenta un elemento del menu di un ristorante. Ecco una panoramica delle sue caratteristiche.

- **'MenuItem'**:

- **Descrizione**: Modello che rappresenta un piatto o un prodotto disponibile nel menu di un ristorante.

- **Campi**:

- * **id**: ID univoco per l'elemento del menu.
- * **restaurant_id**: ID del ristorante a cui appartiene l'elemento del menu (chiave esterna dalla tabella **restaurants**).
- * **name**: Nome dell'elemento del menu.
- * **description**: Descrizione dell'elemento del menu (facoltativa).
- * **price**: Prezzo dell'elemento del menu.
- * **available**: Flag che indica se l'elemento del menu è disponibile (default è True).

- **Relazioni**:

- * **restaurant_id**: Relazione con la tabella **restaurants**, indicando a quale ristorante appartiene l'elemento del menu.

5.3.3 order.py

I modelli Order e OrderItem rappresentano rispettivamente un ordine e un singolo articolo all'interno di un ordine in una piattaforma di food delivery. Ecco una panoramica:

- **'Order'**:

- **Descrizione**: Il modello rappresenta un ordine effettuato da un cliente presso un ristorante.

- **Campi**:

- * **id**: ID univoco per l'ordine.
- * **customer_id**: ID del cliente che ha effettuato l'ordine (chiave esterna alla tabella **users**).
- * **restaurant_id**: ID del ristorante che gestisce l'ordine (chiave esterna alla tabella **restaurants**).
- * **order_date**: Data e ora dell'ordine, con un valore predefinito come l'ora corrente.

- * **status**: Stato dell'ordine, con valori possibili come `pending`, `accepted`, `preparing`, `ready for pickup`, `completed`.
- * **total**: Totale dell'ordine.
- **Relazioni**:
 - * **customer**: Relazione con la tabella `users` (cliente).
 - * **restaurant**: Relazione con la tabella `restaurants` (ristorante).
 - * **delivery**: Relazione con la tabella `Delivery`, indicando la consegna legata all'ordine.
- **'OrderItem'**:
 - **Descrizione**: Il modello rappresenta un articolo specifico all'interno di un ordine.
 - **Campi**:
 - * **order_id**: ID dell'ordine a cui appartiene l'articolo.
 - * **item_id**: ID dell'elemento del menu associato all'articolo.
 - * **quantity**: Quantità dell'articolo nell'ordine.
 - **Relazioni**:
 - * **order**: Relazione con la tabella `Order`, indicante l'ordine a cui l'articolo appartiene.
 - * **item**: Relazione con la tabella `MenuItem`, indicando l'articolo del menu.

5.3.4 restaurant.py

Il modello `Restaurant` rappresenta un ristorante all'interno di un'applicazione di food delivery. Ecco una panoramica di come è strutturato:

- **'Restaurant'**:
 - **Descrizione**: Il modello rappresenta un ristorante all'interno di un'applicazione di food delivery. Contiene informazioni basilari come il nome, l'indirizzo, i contatti, gli orari di apertura, e la descrizione. Ha anche una relazione con il menu (la tabella `MenuItem`).
 - **Campi**:
 - * **id**: ID univoco per il ristorante.
 - * **name**: Nome del ristorante (deve essere unico).
 - * **address**: Indirizzo del ristorante (deve essere unico).
 - * **menu_items**: Relazione con la tabella `MenuItem`, che contiene gli articoli del menu del ristorante.
 - * **phone_number**: Numero di telefono del ristorante.
 - * **email**: Indirizzo email del ristorante.
 - * **is_open**: Stato che indica se il ristorante è aperto o chiuso.
 - * **opening_hours**: Orari di apertura come stringa (potresti voler usare un formato JSON per una migliore gestione).

- * **description**: Una breve descrizione del ristorante.
- * **logo_url**: URL di un logo del ristorante (per la visualizzazione nel sito/app).

– **Relazioni:**

- * **MenuItem**: Ogni ristorante può avere più voci nel suo menu, quindi c'è una relazione uno-a-molti tra **Restaurant** e **MenuItem**.
 - `menu_items = db.relationship('MenuItem', backref='restaurants', lazy=True)`: Questo permette di accedere a tutti gli articoli del menu di un ristorante tramite `restaurant.menu_items`.

5.3.5 user.py

Il modello **User** rappresenta un utente del sistema, che può essere un cliente, un ristorante (ristorante associato tramite `restaurant_id`), o un rider (connesso tramite la relazione con la tabella **Delivery** e **DeliveryPersonAvailability**).

• **'User'**:

- **Descrizione**: Questo modello rappresenta un utente generico nel sistema. Può essere utilizzato per rappresentare un cliente, un ristorante o un rider. Le informazioni incluse sono il nome utente, il nome completo, l'indirizzo, il ruolo dell'utente, le credenziali di accesso, e i dati di contatto come l'email e il numero di telefono.

– **Campi**:

- * **id**: ID univoco per l'utente.
- * **username**: Nome utente (deve essere unico).
- * **name**: Nome completo dell'utente.
- * **address**: Indirizzo dell'utente.
- * **role**: Ruolo dell'utente, che può essere `customer`, `delivery_person`, o `restaurant`.
- * **password_hash**: Hash della password dell'utente, per la gestione sicura delle credenziali.
- * **restaurant_id**: ID del ristorante associato (per i ristoratori).
- * **email**: Email dell'utente (deve essere unica).
- * **phone_number**: Numero di telefono dell'utente (deve essere unico).

– **Relazioni**

- * **Restaurant**: Un ristorante può avere più utenti (ad esempio, il ristoratore stesso, il personale). Quindi c'è una relazione uno-a-molti tra **Restaurant** e **User**.
 - `restaurant_id`: Il campo `restaurant_id` stabilisce la relazione tra l'utente e il ristorante.
- * **Deliveries**: Un rider può avere più consegne, quindi c'è una relazione uno-a-molti tra **User** e **Delivery**.

- Un rider può essere associato a molte consegne (relazione tra **User** e **Delivery** tramite la chiave esterna **user_id** nella tabella **Delivery**).
- * **Availability**: Un rider ha una disponibilità che è memorizzata nella tabella **DeliveryPersonAvailability**, quindi c'è una relazione uno-a-uno (uselist=False) tra **User** e **DeliveryPersonAvailability**.
 - Ogni rider ha una sola entry per la sua disponibilità, stabilita tramite la relazione uno-a-uno tra **User** e **DeliveryPersonAvailability**.

5.4 Routes

5.4.1 administrator.py

Il codice rappresenta due route per l'amministratore della piattaforma di food delivery, utilizzando il framework Flask. Le funzionalità fornite sono la gestione degli account degli utenti e il monitoraggio delle performance della piattaforma. Ogni route è associata a una specifica vista per interagire con l'amministratore.

- **Gestione degli Account - /manage_accounts**
 - **Descrizione:** Route che consente all'amministratore di visualizzare tutti gli utenti nel sistema e di eliminare un utente specifico (escludendo se stesso).
 - **Metodi:**
 - * **GET:** Recupera una lista di tutti gli utenti (tranne l'amministratore) e li passa al template HTML per la visualizzazione.
 - * **POST:** Permette all'amministratore di eliminare un utente. Se l'utente da eliminare è un amministratore, restituisce un errore. Se esiste, l'utente viene rimosso dal database.
 - **Funzionalità di Verifica Accesso:**
 - * **verify_admin_access():** Verifica che l'utente corrente sia un amministratore, restituendo un messaggio di accesso negato se non lo è.
 - **Flusso:**
 - * **GET:** Visualizza gli utenti.
 - * **POST:** Elimina un utente, ma non permette di eliminare l'amministratore.
- **Monitoraggio delle Performance - /monitor_performance**
 - **Descrizione:** Route che permette all'amministratore di visualizzare le performance della piattaforma, inclusi i dati sui rider, ordini completati e le statistiche relative ai ristoranti.
 - **Metodi:**
 - * **GET:** Recupera vari dati sulle performance:
 - **Performance dei rider:** Conta il numero di consegne completate da ogni rider.
 - **Totale consegne completate:** Conta il numero totale di consegne completate.
 - **Totale ordini:** Conta il numero totale di ordini.
 - **Ordini per ristorante:** Conta il numero di ordini per ogni ristorante.
 - **Flusso:**
 - * Raccoglie i dati aggregati dal database tramite SQLAlchemy e li passa al template HTML per la visualizzazione.
 - * Include la possibilità di visualizzare grafici di performance.

- **Verifica dell'Accesso Amministrativo**

- Entrambe le route utilizzano il decoratore `verify_admin_access()`, che verifica se l'utente è un amministratore. Se la verifica non ha successo, l'accesso viene negato.

- **Funzioni di Database Utilizzate**

- `db.session.query()`: Utilizzata per eseguire query sul database:
 - * `User.query.filter(User.id != session.get('user_id')).all()`: Recupera tutti gli utenti tranne l'amministratore.
 - * `db.session.query(db.func.count(Delivery.id))`: Conta il numero di consegne completate.
- `db.func.count()`: Funzione di SQLAlchemy per contare le righe di una tabella.
- `db.session.commit()`: Applica le modifiche al database, come l'eliminazione di un utente.

- **Template HTML**

- `admin_manage_accounts.html`: Template per visualizzare l'elenco degli utenti e la possibilità di eliminarli.
- `admin_monitor_performance.html`: Template per visualizzare i dati delle performance (ordini, consegne, etc.) sotto forma di tabelle o grafici.

- **Sicurezza e Gestione dell'Accesso**

- La funzione `verify_admin_access()` garantisce che solo gli amministratori possano eseguire operazioni sensibili, restituendo un messaggio di accesso negato se l'utente non è autorizzato.

5.4.2 auth.py

Il codice riguarda il sistema di autenticazione e gestione degli accessi per una piattaforma di food delivery, in cui ci sono diversi ruoli (ristorante, cliente, rider e amministratore). Esso include le funzionalità di login, registrazione, logout e dashboard specifiche per ogni tipo di utente. Vengono anche gestiti i permessi per ogni tipo di utente e l'avvio di un consumatore di ordini per i ristoranti.

- **Login - /login**

- **Descrizione:** Permette agli utenti di effettuare il login, con validazione delle credenziali. Se l'utente è un ristorante, viene richiesta anche la conferma dell'ID del ristorante.
- **Funzionalità:**
 - * Se l'utente ha il ruolo `restaurant`, il codice verifica che l'ID del ristorante fornito nel form corrisponda a quello associato all'utente.

- * Dopo una corretta autenticazione, vengono salvati i dati nella sessione (ID utente, ruolo, ID del ristorante se applicabile).
- * Se l'utente è un ristorante, viene avviato un thread che esegue il consumatore di ordini in tempo reale.
- **Redirezione:**
 - * Se il login è riuscito, l'utente viene rediretto alla dashboard corrispondente al proprio ruolo (`restaurant_dashboard`, `customer_dashboard`, `rider_dashboard`, `admin_dashboard`).
- **Registrazione - /register**
 - **Descrizione:** Permette agli utenti di registrarsi, con un modulo che consente di scegliere un nome utente, password, indirizzo e ruolo.
 - **Funzionalità:**
 - * Se il ruolo è `restaurant`, viene verificato se l'ID del ristorante esiste già nel database. Se non esiste, viene creato un nuovo ristorante.
 - * Se il ruolo è `restaurant`, l'ID del ristorante viene associato all'utente.
 - * La password dell'utente viene criptata tramite la funzione `set_password`.
 - **Redirezione:**
 - * Dopo una registrazione riuscita, l'utente viene reindirizzato alla pagina di login, con l'ID del ristorante (se presente) incluso nell'URL.
- **Logout - /logout**
 - **Descrizione:** Rimuove tutti i dati dalla sessione e reindirizza l'utente alla home page.
 - **Funzionalità:**
 - * Viene chiamata la funzione `session.clear()` per cancellare tutte le informazioni dalla sessione.
 - * L'utente viene quindi reindirizzato alla home page.
- **Dashboard per Ristorante - /restaurant_dashboard**
 - **Descrizione:** Visualizza il dashboard specifico per i ristoranti.
 - **Funzionalità:**
 - * Verifica che l'utente sia un ristorante tramite `verify_restaurant_access`.
 - * Recupera i dati dell'utente (ristorante) e li passa a `restaurant_dashboard.html` per la visualizzazione.
- **Dashboard per Cliente - /customer_dashboard**
 - **Descrizione:** Visualizza il dashboard per i clienti.
 - **Funzionalità:**
 - * Verifica che l'utente sia un cliente tramite `verify_customer_access`.
 - * Recupera i dati dell'utente (cliente) e li passa a `customer_dashboard.html` per la visualizzazione.

- **Dashboard per Rider - /rider_dashboard**

- **Descrizione:** Visualizza il dashboard per i rider.

- **Funzionalità:**

- * Verifica che l'utente sia un rider tramite `verify_rider_access`.
 - * Recupera i dati dell'utente (rider) e li passa a `rider_dashboard.html` per la visualizzazione.

- **Dashboard per Amministratore - /administrator_dashboard**

- **Descrizione:** Visualizza il dashboard per gli amministratori.

- **Funzionalità:**

- * Verifica che l'utente sia un amministratore tramite `verify_admin_access`.
 - * Recupera i dati dell'utente (amministratore) e li passa a `admin_dashboard.html` per la visualizzazione.
 - * Se l'utente non è autenticato o non ha il ruolo di amministratore, viene reindirizzato alla pagina di login.

- **Gestione dei Permessi - `verify_*_access`**

- **Descrizione:** Le funzioni `verify_restaurant_access`, `verify_customer_access`, `verify_rider_access`, e `verify_admin_access` sono utilizzate per garantire che solo gli utenti con i permessi adeguati possano accedere alle rispettive dashboard.

- **Funzionalità:**

- * Se un utente non ha il permesso di accedere, viene restituito un messaggio di accesso negato.

- **Thread per il Consumatore di Ordini**

- **Descrizione:** Quando un ristorante effettua il login, viene avviato un thread che esegue la funzione `consume_orders`, permettendo al ristorante di ricevere ordini in tempo reale.

- **Funzionalità:**

- * Il consumatore di ordini è avviato in un thread separato per evitare blocchi nel flusso principale dell'applicazione.
 - * Viene passato l'ID del ristorante per ricevere gli ordini specifici per quel ristorante.

5.4.3 customer.py

Questo blueprint di Flask gestisce le interazioni lato cliente all'interno di una piattaforma di food delivery. Le rotte e le viste corrispondenti gestiscono le funzionalità principali come la navigazione tra i ristoranti, la gestione degli ordini, la visualizzazione e il monitoraggio degli ordini, la gestione del profilo utente e l'interazione con il carrello. Ecco un riepilogo delle componenti principali del codice:

1. Rotte e Viste:

- **Browse Restaurants (/browse):** Permette all'utente di visualizzare i ristoranti nelle vicinanze, entro un raggio di 10 km. Viene utilizzata la posizione dell'utente per calcolare la prossimità, e c'è una funzionalità di ricerca per filtrare i ristoranti in base al nome.
- **Place Order (/place-order):** Gestisce l'aggiunta di articoli al carrello, l'aggiornamento delle quantità e la conclusione dell'ordine. Quando l'utente completa l'ordine, i dati dell'ordine vengono inviati a RabbitMQ e il carrello viene svuotato.
- **Add to Cart (/add-to-cart):** Aggiunge un elemento al carrello dell'utente, che è memorizzato nella sessione, assicurandosi che gli stessi articoli non vengano aggiunti più volte.
- **Track Orders (/track-orders):** Mostra lo stato e il monitoraggio in tempo reale degli ordini dell'utente, recuperando i dettagli degli ordini da un servizio esterno (presumibilmente il backend del ristorante).
- **Order Status Streaming (/order-status):** Utilizza Server-Sent Events (SSE) per inviare aggiornamenti in tempo reale sullo stato dell'ordine, simulando un monitoraggio dinamico.
- **Profile (/profile):** Permette all'utente di visualizzare e aggiornare le informazioni del proprio profilo, come nome, indirizzo e contatti.
- **Search Foods (/search-foods):** Consente all'utente di cercare articoli nei menu dei ristoranti tramite una query, utilizzando un servizio esterno.
- **View Menu (/view-menu):** Visualizza il menu di un ristorante specifico, filtrando gli articoli non disponibili.
- **View Orders (/view-orders):** Mostra tutti gli ordini passati effettuati dall'utente.
- **View Order Details (/order-details):** Visualizza i dettagli di un ordine specifico.

2. Funzioni e Utilità Aggiuntive:

- **Calcolo delle Coordinate (get_coordinates_nominatim):** Recupera le coordinate di un indirizzo utilizzando l'API Nominatim, utile per il calcolo delle distanze.
- **Controllo Accesso (verify_customer_access):** Verifica se l'utente ha i permessi necessari per accedere a determinate sezioni.
- **Calcolo Distanza (haversine):** Calcola la distanza tra due punti geografici utilizzando la formula di Haversine.

3. Gestione della Sessione:

- I contenuti del carrello, le quantità degli articoli e le preferenze dell'utente sono memorizzati nella sessione, consentendo di mantenere lo stato tra le richieste dell'utente.

4. Richieste API Esterne:

- L'applicazione interagisce con endpoint esterni per ottenere dati relativi ai ristoranti, ai menu e agli ordini. Ad esempio, recupera gli ordini, i menu e monitora lo stato di ciascun ordine attraverso richieste a un altro servizio (presumibilmente un backend del ristorante).

5. Pubblicazione degli Ordini (RabbitMQ):

- Quando un cliente effettua un ordine, i dati dell'ordine vengono pubblicati su RabbitMQ (tramite la funzione `publish_order`). Questo facilita la comunicazione con altre parti del sistema, come la notifica al ristorante o ai rider per la consegna.

6. Gestione degli Errori e Flash Messages:

- Vengono utilizzati i flash messages per dare un feedback all'utente, come messaggi di successo, avviso o errore. Questo garantisce un'esperienza utente fluida, informando l'utente sullo stato delle sue azioni (es. "Articolo aggiunto al carrello", "Profilo aggiornato", "Errore nel recupero degli ordini").

5.4.4 `delivery.py`

Questo codice rappresenta un modulo Blueprint di Flask per la gestione delle operazioni dei fattorini (riders) in una piattaforma di food delivery. Il modulo gestisce le richieste di consegna, le rotte ottimali, l'aggiornamento dello stato delle consegne, e la gestione del profilo e disponibilità del fattorino. Ecco una panoramica delle sue funzionalità principali:

- **Gestione delle richieste di consegna:**

- `/requests_data`: Ottiene le richieste di consegna per il rider corrente, calcolando la distanza tra il rider e il ristorante, e restituendo una lista di richieste di consegna pertinenti (rifiutando quelli già rifiutati dal rider).
- `/requests`: Avvia un thread che consuma i messaggi da Kafka e visualizza la pagina con le richieste di consegna.
- `/requests/{int:delivery_id}/action`: Permette al rider di accettare o rifiutare una richiesta di consegna. La risposta viene inviata a Kafka e aggiorna lo stato dell'ordine nel database.

- **Gestione delle rotte:**

- `/optimal-route`: Calcola il percorso ottimale per le consegne accettate o in transito, restituendo la rotta dal ristorante al cliente (in un formato fittizio).

- **Tracciamento e aggiornamento dello stato delle consegne:**
 - `/tracking`: Visualizza una pagina per tracciare le rotte ottimali delle consegne.
 - `/update-status`: Permette di aggiornare lo stato di una consegna, ad esempio passando a "shipped" o "completed".
 - `/int:delivery_id/status`: Permette di aggiornare lo stato di una consegna specifica.
- **Gestione della disponibilità del rider:**
 - `/availability`: Consente al rider di visualizzare e aggiornare la propria disponibilità per le consegne, specificando orari di inizio e fine.
- **Gestione del profilo del rider:**
 - `/profile`: Consente al rider di visualizzare e aggiornare le informazioni del proprio profilo (nome, indirizzo, email, telefono, ecc.).
- **Comunicazione tramite Kafka:**
 - Quando il rider accetta o rifiuta una consegna, un messaggio viene inviato tramite Kafka al topic "delivery_responses" per aggiornare lo stato nel sistema.
- **Supporto per la gestione delle richieste in tempo reale:**
 - Le richieste vengono memorizzate in una coda (`requests_queue`), e il rider può ottenere le richieste in tempo reale tramite la funzione `get_requests_data`.
- **Funzioni aggiuntive:**
 - `get_coordinates_nominatim`: Recupera le coordinate geografiche di un indirizzo tramite l'API Nominatim.
 - `verify_rider_access`: Verifica che l'utente sia autenticato come rider, altrimenti ne impedisce l'accesso.
 - `haversine`: Calcola la distanza tra due punti geograficamente separati.
- **Considerazioni tecniche:**
 - **Multithreading**: La funzione `consume_kafka_messages` viene eseguita in un thread separato per ascoltare i messaggi Kafka senza bloccare il flusso dell'applicazione.
 - **Flask e Renderizzazione dei Template**: I template HTML sono usati per visualizzare e aggiornare le informazioni del rider, e sono resi utilizzando `render_template`.
 - **Gestione dello stato della consegna**: Lo stato delle consegne viene aggiornato a vari stadi, come "accepted_by_rider", "shipped", e "completed".

5.4.5 home.py

Questo codice definisce un modulo Blueprint in Flask per gestire la home page e il reindirizzamento dell'utente in base alla selezione del ruolo.

- **Home Page:**

- Route `\`: Quando un utente accede alla home page, viene eseguito il metodo `home`, che cancella la sessione precedente (se presente) e restituisce il template `home.html`.
- Questo è utile per resettare lo stato dell'utente ogni volta che accede alla home page.

- **Selezione del Ruolo e Reindirizzamento:**

- Route `\redirect`: Quando un utente invia il modulo (tramite POST), viene acquisito il valore `restaurant_id` dal form.
- Il sistema quindi reindirizza l'utente alla pagina di login (`auth.login`), passando `restaurant_id` come parametro, se presente.
- Se il `restaurant_id` non è fornito nel modulo, il parametro viene lasciato vuoto.

- **Utilizzo:**

- Questo Blueprint è utile per implementare una pagina iniziale con una selezione del ristorante e un sistema di login legato al ristorante selezionato.
- La gestione della sessione garantisce che l'utente non mantenga dati residui da sessioni precedenti, fornendo una "pagina di ingresso" pulita.

5.4.6 restaurant.py

Questo codice definisce diverse rotte per la gestione del menu, degli ordini, e dei profili dei ristoranti in un'applicazione Flask. Di seguito sono descritte le principali funzionalità e rotte:

- **Gestione del menu:**

- `/menu/<int:restaurant_id>` (GET): Visualizza il menu del ristorante.
- `/menu/json/<int:restaurant_id>` (GET): Restituisce il menu del ristorante in formato JSON.
- `/menu/<int:restaurant_id>` (POST): Aggiunge un nuovo piatto al menu.
- `/menu/<int:restaurant_id>/<int:item_id>` (DELETE): Rimuove un piatto dal menu.
- `/menu/<int:restaurant_id>/<int:item_id>` (GET): Restituisce i dettagli di un piatto specifico.
- `/menu/<int:restaurant_id>/<int:item_id>` (PUT): Modifica un piatto nel menu.

- **Gestione degli ordini:**

- `/restaurant/<int:restaurant_id>/orders` (GET): Visualizza gli ordini di un ristorante.
- `/restaurant/order/<int:order_id>` (GET, POST): Visualizza e aggiorna i dettagli di un ordine (es. cambia stato e invia ordine a Kafka quando pronto per il ritiro).
- `/orders/customer/<int:customer_id>` (GET): Recupera gli ordini di un cliente specifico.
- `/orderstraceable/customer/<int:customer_id>` (GET): Recupera solo gli ordini tracciabili di un cliente.

- **Gestione del profilo del ristorante:**

- `/restaurant/<int:restaurant_id>/profile` (GET, POST): Visualizza e aggiorna il profilo di un ristorante (incluso nome, indirizzo, numero di telefono, orari di apertura, ecc.).

- **Ricerca menu:**

- `/search/menu/items` (GET): Cerca piatti del menu in base a una query (nome del piatto) e la distanza dal cliente (entro 10 km).

- **Consumo degli ordini:**

- La funzione `consume_orders` è avviata in background utilizzando `socketio` per ogni ristorante per monitorare gli ordini.
- La connessione tramite WebSocket è utilizzata per tenere traccia in tempo reale degli ordini.

- **Comportamento:**

- Ogni rotta è protetta da un controllo di accesso tramite `verify_restaurant_access`, che verifica se l'utente ha i permessi per accedere alle risorse del ristorante.
- In caso di errori, vengono restituite risposte JSON con codici di errore (ad esempio, 400, 404, 500).
- Le risposte con successo restituiscono i dati in formato JSON o vengono renderizzati i template HTML appropriati.

- **Utilizzo di Kafka e RabbitMQ:**

- **Kafka:** Viene utilizzato per inviare dettagli sugli ordini quando lo stato di un ordine diventa `"ready_for_pickup"`.
- **RabbitMQ:** Viene utilizzato per consumare gli ordini, monitorando continuamente gli ordini dei ristoranti e aggiornandoli tramite WebSocket.

- **Utilizzo generale:**

- Questa struttura è utile per una piattaforma di food delivery in cui i ristoranti possono gestire il loro menu, i clienti possono fare ordini e monitorare lo stato, e l'amministrazione può visualizzare e gestire profili e ordini in tempo reale.

5.5 Utils

5.5.1 consumer_kafka.py

Il codice riguarda un consumer Kafka che riceve le richieste di consegna per un rider e calcola la distanza tra il rider e il ristorante per determinare se la richiesta è pertinente. Vediamo i punti principali:

- **Configurazione del Consumer Kafka:**

- Viene creato un consumer Kafka che si connette al broker `localhost:9092`.
- Il consumer si iscrive al gruppo di consumatori `rider_user_id`.
- Il consumer legge dal primo messaggio disponibile e disabilita il commit automatico per avere un controllo completo sugli offset.

- **Assegnazione manuale delle partizioni:**

- Viene ottenuto l'elenco delle partizioni per il topic `delivery_requests`.
- Il consumer si iscrive manualmente alle partizioni del topic.

- **Recupero dell'indirizzo e delle coordinate del rider:**

- Utilizzando la funzione `get_coordinates_nominatim`, vengono ottenute le coordinate GPS del rider in base al suo indirizzo.
- Questo serve per calcolare la distanza tra il rider e i ristoranti.

- **Elaborazione delle richieste di consegna:**

- Ogni messaggio Kafka che rappresenta una richiesta di consegna viene letto e decodificato.
- La distanza tra il rider e il ristorante viene calcolata utilizzando la formula `haversine`.
- Se la distanza è inferiore o uguale a 10 km, la richiesta viene considerata pertinente e aggiunta alla lista `relevant_requests`.

- **Commit dell'offset:**

- Dopo aver elaborato una richiesta, l'offset del messaggio viene committato per evitare di rielaborarlo.

- **Inserimento nella coda:**

- Le richieste pertinenti vengono inserite in una coda `queue` per essere elaborate successivamente.

- **Gestione degli errori:**

- Gli errori durante la lettura dei messaggi vengono gestiti, e in caso di problemi, viene stampato un messaggio di errore.

5.5.2 consumer_rabbitmq.py

Il codice si occupa di consumare i messaggi dalla coda RabbitMQ, processare gli ordini e inviare gli aggiornamenti ai client connessi tramite socketio. Analizziamo i principali passaggi e funzioni:

- **Connessione a RabbitMQ:**

- La funzione stabilisce una connessione con RabbitMQ utilizzando il client `pika`.
- La coda `restaurant_restaurant_id_orders` è configurata per essere durevole, garantendo che i messaggi non vengano persi anche se il broker si riavvia.

- **Funzione `on_message`:**

- Viene chiamata ogni volta che un nuovo messaggio arriva nella coda.
- Il corpo del messaggio (un ordine in formato JSON) viene decodificato.
- L'ordine viene salvato nel database creando un oggetto `Order` con i dettagli come `customer_id`, `restaurant_id`, `order_date`.

- **Aggiunta degli articoli dell'ordine:**

- Per ogni articolo nell'ordine, viene cercato un `MenuItem` corrispondente nel database.
- Se l'articolo esiste, viene aggiunto come `OrderItem` con `order_id`, `item_id` e quantità.
- Se un articolo non viene trovato nel menu, viene stampato un messaggio di errore.

- **Completamento dell'ordine:**

- Dopo aver aggiunto gli articoli, viene effettuato il commit della transazione.
- L'ordine completo viene ricaricato dal database e preparato per l'invio ai client.

- **Invio tramite SocketIO:**

- I dettagli dell'ordine (ID, data, stato, totale) vengono inviati ai client tramite `socketio.emit('new_order', order_to_send)`.
- Questo invia l'ordine a tutti i client connessi al server.

- **Conferma del messaggio RabbitMQ:**

- Dopo aver processato il messaggio, viene confermato con `channel.basic_ack()`.
- Questo segnala a RabbitMQ che il messaggio è stato elaborato e può essere rimosso dalla coda.

Considerazioni sul flusso:

- Ogni volta che un ordine viene creato, un aggiornamento in tempo reale viene inviato ai client tramite `socketio`, il che potrebbe portare a una certa latenza se ci sono molti client connessi.
- L'uso della coda RabbitMQ è vantaggioso per gestire un alto volume di ordini, poiché consente un'elaborazione asincrona e scalabile.

5.5.3 get_coordinates.py

Si tratta di una funzione che utilizza l'API di Nominatim (OpenStreetMap) per ottenere le coordinate geografiche (latitudine e longitudine) di un indirizzo specificato. Ecco una panoramica dettagliata del funzionamento:

- **Codifica dell'indirizzo:**

- L'indirizzo passato come parametro alla funzione viene inserito nella query URL.
- Questo è il parametro `q={address}` che viene passato alla API di Nominatim per la ricerca.

- **Creazione della richiesta:**

- La richiesta GET viene effettuata a Nominatim utilizzando il modulo `requests`.
- La richiesta include un header con un `User-Agent` che identifica l'applicazione che fa la richiesta.
- Nominatim richiede che il `User-Agent` sia specificato.

- **Gestione della risposta:**

- Se la risposta è positiva (`response.raise_for_status()`), i dati vengono parsati come JSON.
- La funzione estrae la latitudine (`lat`) e la longitudine (`lon`) dal primo risultato, se presente.
- Se non ci sono risultati, la funzione restituisce `None, None`.

- **Gestione delle eccezioni:**

- Vengono gestiti errori generali relativi alla richiesta tramite `requests.exceptions.RequestException`.
- Vengono gestiti errori di parsing JSON o di struttura della risposta tramite `KeyError` o `ValueError`.

5.5.4 haversine_distance.py

La funzione `haversine` calcola la distanza tra due punti sulla superficie terrestre, dati dalla loro latitudine e longitudine, utilizzando la Formula di Haversine. Questo è un metodo matematico che consente di determinare la distanza del percorso più corto tra due punti sulla superficie di una sfera (approssimando la Terra come una sfera).

- **Converte** le coordinate da gradi a radianti.
- **Calcola** le differenze tra le latitudini e longitudini.
- **Applica** la formula di Haversine per determinare l'angolo centrale tra i due punti.
- **Moltiplica** questo angolo per il raggio medio della Terra (6371 km) per ottenere la distanza in chilometri.

5.5.5 producer_kafka.py

La funzione `send_order_to_kafka` invia un ordine a un topic Kafka chiamato `delivery_requests`, utilizzando il protocollo di Kafka con il Producer di `confluent_kafka`.

- **Creazione del Producer Kafka:** Si stabilisce una connessione con il server Kafka (`localhost:9092`).
- **Callback per la gestione degli errori:** La funzione `delivery_report` viene chiamata ogni volta che c'è un errore nel messaggio inviato. Se l'invio fallisce, viene stampato un messaggio di errore.
- **Creazione del messaggio:** I dati dell'ordine (ID ordine, ID ristorante, stato e coordinate di latitudine e longitudine) vengono organizzati in un dizionario.
- **Serializzazione in JSON:** Il messaggio viene convertito in formato JSON e codificato in `utf-8` prima di essere inviato.
- **Invio a Kafka:** Il messaggio viene inviato al topic `delivery_requests` tramite il metodo `produce`.
- **Flush del producer:** Con `flush()`, si assicura che il messaggio venga effettivamente inviato prima che la funzione termini.

5.5.6 producer_rabbitmq.py

La funzione `publish_order` si occupa di inviare un ordine a un ristorante utilizzando RabbitMQ. Ecco come funziona:

- **Connessione a RabbitMQ:** Viene stabilita una connessione a RabbitMQ tramite `pika.BlockingConnection` utilizzando i parametri di connessione predefiniti (`localhost`).
- **Creazione di un canale:** Un canale di comunicazione viene aperto per inviare i messaggi.
- **Creazione della coda:** Una coda specifica per il ristorante viene creata dinamicamente usando `restaurant.restaurant_id_orders` come nome della coda. La coda è contrassegnata come "durable", il che significa che la coda sopravvive anche ai riavvii del server RabbitMQ.
- **Pubblicazione del messaggio:** L'ordine (`order_data`), che è un dizionario contenente i dati dell'ordine, viene serializzato in formato JSON e inviato alla coda tramite `channel.basic_publish()`. La proprietà `delivery_mode=2` assicura che il messaggio sia persistente (cioè non andrà perso in caso di riavvio del broker).
- **Chiusura della connessione:** Alla fine, la connessione a RabbitMQ viene chiusa.

5.5.7 session_handler.py

La serie di funzioni presenti in questo file gestisce la verifica dell'accesso per vari ruoli utente (admin, rider, customer, ristorante) nell'applicazione Flask. Ogni funzione ha un obiettivo specifico, basato sui dati della sessione, per determinare se l'utente ha i permessi necessari per accedere a determinate risorse. Ecco un riepilogo di ciascuna funzione:

- **Funzioni di Verifica Accesso:**

- **'get_session_data()':**
 - * Recupera i dati dalla sessione Flask.
 - * Ritorna l'ID dell'utente, il ruolo dell'utente e l'ID del ristorante (se il ruolo è "restaurant").
- **'verify_admin_access()':**
 - * Verifica se l'utente ha il ruolo di "admin".
 - * Se l'utente non è un amministratore, viene visualizzato un messaggio di errore e l'utente viene reindirizzato alla pagina di login.
- **'verify_rider_access()':**
 - * Verifica se l'utente ha il ruolo di "rider".
 - * Se l'utente non è un rider, viene visualizzato un messaggio di errore e l'utente viene reindirizzato alla pagina di login.
- **'verify_customer_access()':**
 - * Verifica se l'utente ha il ruolo di "customer".
 - * Se l'utente non è un cliente, viene visualizzato un messaggio di errore e l'utente viene reindirizzato alla pagina di login.
- **'verify_restaurant_access(restaurant_id)':**
 - * Verifica se l'utente ha il ruolo di "restaurant" e se l'ID del ristorante nell'utente corrisponde a quello passato come parametro.
 - * Se l'utente non ha accesso al ristorante specificato, viene visualizzato un messaggio di errore e l'utente viene reindirizzato alla pagina di login.

- **Funzionamento:**

- Ogni funzione di verifica fa affidamento su 'get_session_data()', che estrae i dati relativi all'utente dalla sessione.
- Se l'accesso non è consentito, un messaggio di errore viene mostrato tramite `flash()`, e l'utente viene reindirizzato alla pagina di login utilizzando `redirect(url_for('auth.login'))`.

5.5.8 trigger.py

Questa parte del codice si occupa di determinare se un ristorante è aperto o chiuso in base agli orari di apertura memorizzati nel database, aggiornando automaticamente lo stato di apertura dei ristoranti nel sistema. Ecco una descrizione dettagliata:

- **Funzioni e Logiche Implementate:**

- **‘is_restaurant_open(opening_hours: str) -> bool’:**

- * **Obiettivo:** Verifica se il ristorante è aperto in base agli orari di apertura registrati nel database.
- * **Funzionamento:**
 - Riceve gli orari di apertura del ristorante come stringa JSON (nel formato `'giorno': {'apertura': 'hh:mm', 'chiusura': 'hh:mm'}`).
 - Converte la stringa in un oggetto Python (dizionario) tramite `json.loads()`.
 - Ottiene il giorno corrente della settimana (in inglese) e lo mappa al corrispondente giorno in italiano usando il dizionario `DAYS_MAP`.
 - Confronta l'orario corrente con gli orari di apertura e chiusura del ristorante.
 - Se il ristorante ha una chiusura oltre la mezzanotte (ad esempio "00:30"), gestisce questo caso e considera la fascia oraria successiva.
 - Restituisce `True` se il ristorante è aperto, altrimenti `False`.

- **‘update_restaurant_open_status()’:**

- * **Obiettivo:** Aggiorna automaticamente lo stato di apertura dei ristoranti nel database.
- * **Funzionamento:**
 - Recupera tutti i ristoranti dal database usando `Restaurant.query.all()`.
 - Per ogni ristorante, chiama la funzione `is_restaurant_open()` per determinare se è aperto o chiuso.
 - Imposta lo stato di apertura del ristorante nel database (campo `is_open`).
 - Salva l'aggiornamento nel database tramite `db.session.commit()`.
 - Se c'è un errore nel salvataggio (ad esempio una violazione del database), effettua il rollback della sessione.

- **Funzione is_restaurant_open:**

- Gestisce correttamente i vari formati di orario (es. orari normali o con chiusura oltre la mezzanotte).
- Se l'orario di apertura o di chiusura è malformato o se non è presente il giorno nella lista, la funzione restituisce `False`.

- **Funzione update_restaurant_open_status:**

- Automatizza l'aggiornamento dello stato di apertura per tutti i ristoranti, mantenendo il sistema sempre aggiornato senza intervento manuale.

Indice delle Figure

1	Food Delivery Platform	3
2	Processo di Ordinazione sulla Piattaforma	3
3	Diagrammi UML	40
4	Pattern MVC	42
5	Flask	62
6	AJAX	66
7	SQLAlchemy	72
8	SQLite	76
9	API RESTful	81
10	API RESTful: Sistema Completo	81
11	RabbitMQ	85
12	RabbitMQ: Un Producer ed Un Consumer	85
13	Kafka	90
14	Kafka: Sistema Completo	93